

LAYER: 1_Core

This file is a merged representation of a subset of the codebase, containing files not matching ignore patterns, combined into a single document by Repomix.

<file_summary>

This section contains a summary of this file.

<purpose>

This file contains a packed representation of a subset of the repository's contents that is considered the most important context.

It is designed to be easily consumable by AI systems for analysis, code review, or other automated processes.

</purpose>

<file_format>

The content is organized as follows:

1. This summary section
2. Repository information
3. Directory structure
4. Repository files (if enabled)
5. Multiple file entries, each consisting of:
 - File path as an attribute
 - Full contents of the file

</file_format>

<usage_guidelines>

- This file should be treated as read-only. Any changes should be made to the original repository files, not this packed version.
- When processing this file, use the file path to distinguish between different files in the repository.
- Be aware that this file may contain sensitive information. Handle it with the same level of security as you would the original repository.

</usage_guidelines>

<notes>

- Some files may have been excluded based on .gitignore rules and Repomix's configuration
- Binary files are not included in this packed representation. Please refer to the Repository Structure section for a complete list of file paths, including binary files
- Files matching these patterns are excluded: **/obj/**, **/bin/**, **/*.png, **/*.jpg, **/node_modules/**, **/*.Designer.cs
- Files matching patterns in .gitignore are excluded
- Files matching default ignore patterns are excluded
- Files are sorted by Git change count (files with more changes are at the bottom)

</notes>

</file_summary>

<directory_structure>

Configuration.Web/Components/_Imports.razor
Configuration.Web/Components/Layout/MainLayout.razor
Configuration.Web/Components/Layout/ReconnectModal.razor
Configuration.Web/Components/NavMenu.razor
Configuration.Web/Components/NotFound.razor
Configuration.Web/Components/Routes.razor
Configuration.Web/Configurator/IWebAppConfigurator.cs
Configuration.Web/Configurator/WebAppConfigurator.cs
Configuration.Web/Configurator/WebScanningExtensions.cs
Configuration.Web/Promatis.Net.Configuration.Web.csproj
Configuration.Web/Properties/launchSettings.json
Configuration.Web/Services/TabNavigationService/ComponentRegistry.cs
Configuration.Web/Services/TabNavigationService/TabItem.cs
Configuration.Web/Services/TabNavigationService/TabNavigationService.cs
Configuration.Web/Services/UiModuleService.cs
Configuration.Web/UiCommandBus/IUiCommandBus.cs
Configuration.Web/UiCommandBus/UiCommandBus.cs
Configuration.Web/WebAppBootstrapper.cs
Configuration/AppBootstrapper.cs
Configuration/AppConfigurator.cs
Configuration/AppInfrastructure.cs
Configuration/IAppConfigurator.cs
Configuration/Promatis.Net.Configuration.csproj
Configuration/ServiceScanningExtensions.cs
Data/Configurations/AuditLogConfiguration.cs
Data/Configurations/ModelBuilderConventions.cs
Data/DbContext/ApplicationDbContext.cs

Data/DbContext/ModelBuilderExtensions.cs
Data/Interceptors/ChangeEntryModel.cs
Data/Interceptors/DatabaseTriggerInterceptor.cs
Data/Interceptors/IUserProvider.cs
Data/Promatis.Net.Data.csproj
Data/Triggers/Castom/AuditTrigger.cs
Data/Triggers/Global/FluentValidationTrigger.cs
Data/Triggers/Global/ReferenceTreeParentTrigger.cs
Data/TriggerService/DatabaseTriggerService.cs
Data/TriggerService/EntityStateChangeEnum.cs
Data/TriggerService/IDatabaseTriggerService.cs
Data/TriggerService/PropertyChangeInfo.cs
Data/TriggerService/TriggerEvents/EntityCancelArgsBase.cs
Data/TriggerService/TriggerEvents/EntityCancelEventArgs.cs
Data/TriggerService/TriggerEvents/EntityChangedEventArgsBase.cs
Data/TriggerService/TriggerEvents/EntityChangedEventArgs.cs
Data/TriggerService/TriggerEvents/EntityEventArgsBase.cs
Data/TriggerService/TriggerInterfaces/IAfterSaveTrigger.cs
Data/TriggerService/TriggerInterfaces/IBeforeSaveTrigger.cs
Data/Validation/GlobalPolymorphicValidator.cs
Domain.Interface/Enums/EnumExtensions.cs
Domain.Interface/Interfaces/IAudit.cs
Domain.Interface/Interfaces/IDomainObject.cs
Domain.Interface/Interfaces/IDomainObjectHasKey.cs
Domain.Interface/Interfaces/IEntityCloner.cs
Domain.Interface/Interfaces/ISoftDeletable.cs
Domain.Interface/Interfaces/ITreeNode.cs
Domain.Interface/Promatis.Net.Domain.Interface.csproj
Domain/AuditLog/AuditLog.cs
Domain/DomainObject/DomainObject.cs
Domain/DomainObject/DomainObjectBase.cs
Domain/DomainObject/DomainObjectValidator.cs
Domain/EntityCloner/JsonEntityCloner.cs
Domain/Promatis.Net.Domain.csproj
Domain/ReferenceBase/ReferenceBase.cs
Domain/ReferenceBase/ReferenceBaseValidator.cs
Domain/ReferenceTreeBase/ReferenceTreeBase.cs
Domain/ReferenceTreeBase/ReferenceTreeBaseValidator.cs
Promatis.Net.UI/_Imports.razor
Promatis.Net.UI/Components/ActionContext/CrudWorkspaceContext.cs
Promatis.Net.UI/Components/ActionContext/IWorkspaceActionContext.cs
Promatis.Net.UI/Components/ActionContext/WorkspaceActionContext.cs
Promatis.Net.UI/Components/Dialogs/Buttons/CancelDialogButton.cs
Promatis.Net.UI/Components/Dialogs/Buttons/SubmitDialogButton.cs
Promatis.Net.UI/Components/Dialogs/DynamicDialog/DialogActionContextBase.cs
Promatis.Net.UI/Components/Dialogs/DynamicDialog/DialogTabConfig.cs
Promatis.Net.UI/Components/Dialogs/DynamicDialog/DynamicEditDialog.razor
Promatis.Net.UI/Components/Dialogs/DynamicDialog/DynamicEditDialog.razor.cs
Promatis.Net.UI/Components/Dialogs/DynamicDialog/IDialogActionContext.cs
Promatis.Net.UI/Components/ElementRenderBase/ButtonRenderBase.razor
Promatis.Net.UI/Components/ElementRenderBase/ButtonRenderBase.razor.cs
Promatis.Net.UI/Components/ElementRenderBase/DateRangeRenderBase.razor
Promatis.Net.UI/Components/ElementRenderBase/DateRangeRenderBase.razor.cs
Promatis.Net.UI/Components/ElementRenderBase/DividerRenderBase.razor
Promatis.Net.UI/Components/ElementRenderBase/DividerRenderBase.razor.cs
Promatis.Net.UI/Components/ElementRenderBase/RenderBase.cs
Promatis.Net.UI/Components/ElementRenderBase/SelectRenderBase.razor
Promatis.Net.UI/Components/ElementRenderBase/SelectRenderBase.razor.cs
Promatis.Net.UI/Components/References/Dialogs/ReferenceEditDialog.razor
Promatis.Net.UI/Components/References/Dialogs/ReferenceEditDialog.razor.cs
Promatis.Net.UI/Components/References/Dialogs/ReferenceEditDialogContext.cs
Promatis.Net.UI/Components/References/ReferenceGridPage.razor
Promatis.Net.UI/Components/References/ReferenceGridPage.razor.cs
Promatis.Net.UI/Components/References/ReferenceWorkspaceContext.cs
Promatis.Net.UI/Components/UIControls/BaseUiControl.cs
Promatis.Net.UI/Components/UIControls/IHasOptions.cs
Promatis.Net.UI/Components/UIControls/IHasValue.cs
Promatis.Net.UI/Components/UIControls/IUiControl.cs
Promatis.Net.UI/Components/UIControls/UiControlAlignment.cs
Promatis.Net.UI/Components/Workspaces/GridPage.razor
Promatis.Net.UI/Components/Workspaces/GridPage.razor.cs
Promatis.Net.UI/Components/Workspaces/TreePage.razor
Promatis.Net.UI/Components/Workspaces/TreePage.razor.cs
Promatis.Net.UI/Components/Workspaces/UiToolbar.razor
Promatis.Net.UI/Components/Workspaces/UiToolbar.razor.cs
Promatis.Net.UI/Components/Workspaces/WorkspacePage.razor

```

Promatis.Net.UI/Components/Workspaces/WorkspacePage.razor.cs
Promatis.Net.UI/Controls/CreateEntityButton.cs
Promatis.Net.UI/Controls/DeleteEntityButton.cs
Promatis.Net.UI/Controls/EditEntityButton.cs
Promatis.Net.UI/Controls/ToolBarDivider.cs
Promatis.Net.UI/Data/Cache/FlatOzuMutationStrategy.cs
Promatis.Net.UI/Data/Cache/HierarchicalOzuMutationStrategy.cs
Promatis.Net.UI/Data/Cache/IOzuMutationStrategy.cs
Promatis.Net.UI/Data/Cache/IUiOzuCache.cs
Promatis.Net.UI/Data/Cache/UiOzuCache.cs
Promatis.Net.UI/Data/IUiDataBroker.cs
Promatis.Net.UI/Data/UiDataBroker.cs
Promatis.Net.UI/Extensions/MudColorExtensions.cs
Promatis.Net.UI/Interfaces/IHasSelectedData.cs
Promatis.Net.UI/IUiModule.cs
Promatis.Net.UI/Pages/AuditLogs/AuditLogPage.razor
Promatis.Net.UI/Pages/AuditLogs/AuditLogPage.razor.cs
Promatis.Net.UI/Pages/AuditLogs/AuditLogWorkspaceContext.cs
Promatis.Net.UI/Pages/AuditLogs/ToolBar/AuditActionSelect.cs
Promatis.Net.UI/Pages/AuditLogs/ToolBar/AuditEntitySelect.cs
Promatis.Net.UI/Pages/AuditLogs/ToolBar/AuditExportButton.cs
Promatis.Net.UI/Pages/AuditLogs/ToolBar/AuditPeriodPicker.cs
Promatis.Net.UI/Pages/AuditLogs/ToolBar/AuditToolBarDivider.cs
Promatis.Net.UI/Pages/Dashboard.razor
Promatis.Net.UI/Pages/Dashboard.razor.css
Promatis.Net.UI/Promatis.Net.UI.csproj
Promatis.Net.UI/Theme/PromatisTheme.cs
Promatis.Net.UI/UiModule.cs
Promatis.Net.UI/wwwroot/App.css
Service/Contracts/AuditLog/AuditLogSearchRequest.cs
Service/Contracts/AuditLog/PagedResult.cs
Service/Promatis.Net.Service.csproj
Service/Services/AuditLogService/AuditLogService.cs
Service/Services/AuditLogService/IAuditLogService.cs
Service/Services/BaseService/BaseService.cs
Service/Services/BaseService/IBaseService.cs
Service/Services/ReferenceService/IReferenceService.cs
Service/Services/ReferenceService/ReferenceService.cs
Service/Services/ReferenceTreeService/IReferenceTreeService.cs
Service/Services/ReferenceTreeService/ReferenceTreeService.cs
Service/Services/TreeService/ITreeService.cs
Service/Services/TreeService/TreeService.cs
Tests/AssemblyInfo.cs
Tests/Domain/DomainLogicTests.cs
Tests/Domain/DomainObjectBaseLogicTests.cs
Tests/Domain/DomainObjectBaseTests.cs
Tests/Domain/DomainObjectExtendedTests.cs
Tests/Domain/ReferenceBaseTests.cs
Tests/Domain/ReferenceTreeTests.cs
Tests/Domain/SoftDeletableTests.cs
Tests/Interceptor/DatabaseTriggerInterceptorTests.cs
Tests/Promatis.Net.ApplicationModel.Tests.csproj
Tests/Service/BaseServiceTests_Crud.cs
Tests/Service/BaseServiceTests.cs
Tests/Service/ReferenceServiceTests_Search.cs
Tests/Service/ReferenceTreeServiceTests.cs
Tests/Service/ServiceIntegrationTests.cs
Tests/Trigger/AuditTriggerTests.cs
Tests/Trigger/DatabaseTriggerServiceTests.cs
Tests/Trigger/FluentValidationTriggerTests.cs
Tests/Trigger/FullTriggerChainTests.cs
Tests/Trigger/ReferenceTreeParentTriggerTests.cs
Tests/Trigger/TreeTriggerChainTests.cs
Tests/Validator/DomainObjectValidatorTests.cs
Tests/Validator/ReferenceBaseValidatorTests.cs
Tests/Validator/ReferenceTreeBaseValidatorTests.cs
</directory_structure>

```

<files>

This section contains the contents of the repository's files.

```

<file path="Configuration.Web/Components/_Imports.razor">
@using System.Net.Http
@using System.Net.Http.Json
@using Microsoft.AspNetCore.Components.Forms
@using Microsoft.AspNetCore.Components.Routing

```

```

@using Microsoft.AspNetCore.Components.Web
@using Microsoft.JSInterop
@using MudBlazor
@using Promatis.Net.Ui
@using Promatis.Net.Configuration.Web
@using Promatis.Net.Configuration.Web.Components
@using Promatis.Net.Configuration.Web.Components.Layout
@using static Microsoft.AspNetCore.Components.Web.RenderMode
</file>

<file path="Configuration.Web/Components/Layout/MainLayout.razor">
@inherits LayoutComponentBase
@using Promatis.Net.Ui.Theme
@
@inject UiModuleService UiService
@inject TabNavigationService TabService
@implements IDisposable
@
@* A,,!Aô A A´ Aô : B 2Dô7D´2C 5CÂ ?D >C$0C"4CT@ D$5CÄK D DC´0C4>CÂ BE <Cô>C4> D 5Cd8CÄ0 *@
<MudThemeProvider Theme="PromatisTheme.DefaultTheme" @bind-IsDarkMode="_isDarkMode" />
<MudPopoverProvider />
@
<MudDialogProvider FullWidth="false"
    MaxWidth="MaxWidth.False"
    CloseButton="false"
    BackdropClick="false"
    NoHeader="false"
    Position="DialogPosition.Center"
    CloseOnEscapeKey="true" />
@
<MudSnackbarProvider />
@
<CascadingValue Value="this">
    <MudLayout>
        @* A$5D ECô0Dò ?C =CT;DÂ ?D 8C´>Cd5Cô8Dò ¢
        <MudAppBar Color="Color.Default" Elevation="1">
            @* A,,!Aô A A´ Aô : At0CÄ5Cô5Cô> Color.Primary Cô Color.Default *@
            <MudIconButton Icon="@Icons.Material.Filled.Menu"
                Color="Color.Inherit"
                Edge="Edge.Start"
                Size="Size.Large"
                OnClick="ToggleDrawer" />
            <MudStack Spacing="-1">
                @* AD5DD>C´BCô>CR =C 7C$0Cô8CR ¢
                <MudText Typo="Typo.h6" Class="ml-2">
                    B$5D BCä2D´9 Cô@Cä5C¢B Promatis Net
                </MudText>
            </MudStack>
        @
        <MudSpacer />
        @
        <MudIconButton Icon="@(_isDarkMode ? Icons.Material.Filled.LightMode :
Icons.Material.Filled.DarkMode)"
            Color="Color.Inherit"
            OnClick="ToggleDarkMode" />
    </MudAppBar>
    @
    @* Aô0Cô5C´L CÄ5Côn *@
    <MudDrawer @bind-Open="_drawerOpen" ClipMode="DrawerClipMode.Always"
Variant="DrawerVariant.Temporary" Overlay="true" OverlayAutoClose="true" Width="220px"
Elevation="1">
        <NavMenu />
    </MudDrawer>
    @
    <MudMainContent Class="pt-16 px-4 pb-4 d-flex flex-column"
        Style="height: calc(100vh - 8px); min-height: calc(100vh - 8px); margin-
top: 10px; display: flex; flex-direction: column; flex-grow: 1; overflow: hidden;">
        @
        @* A,,!Aô A A´ Aô : A4;Cä1C ;DÄ=D´9 ErrorBoundary D41D 0Cò A C$5D ECô5C4> D4@Cä2Cô0. B BC,,;C, fÄW
        @if (TabService.OpenTabs.Count == 0)
        {
            <div class="flex-grow-1 overflow-auto">
                <ErrorBoundary>
                    <ChildContent>
                        @Body
                    </ChildContent>
                </ErrorBoundary>
            </div>
        }
    </MudMainContent>
</file>

```

```

        <ErrorContent Context="exception">@
            <MudAlert Severity="Severity.Error" Class="my-4">@
                A@C@8Ct>D,,;C :D 8D$8Dt5D :C 0 C@HC,,1C@0 C,,=D$5D DCT9D 0: @exception.Message
            </MudAlert>@
        </ErrorContent>@
    </ErrorBoundary>@
</div>@
}@
else@
{@
    <MudTabs @bind-ActivePanelIndex="TabService.ActiveTabIndex"@
        Position="Position.Top"@
        Color="Color.Default"@
        SliderColor="Color.Primary"@
        Class="mdi-tabs-container w-100 flex-grow-1 d-flex flex-column"@
        Style="height: 100%; max-height: 100%; min-height: 0; display: flex; flex-
direction: column; flex-grow: 1;"@
        TabPanelsClass="pa-0 flex-grow-1 d-flex flex-column overflow-hidden"@
        ApplyEffectsToContainer="true"@
        TabButtonsClass="rounded-t-lg shadow-sm"@
        Outlined="true">@
@
        @for (int i = 0; i < TabService.OpenTabs.Count; i++)@
        {@
            int localIndex = i;@
            TabItem tab = TabService.OpenTabs[i];@
@
            @* A,,!A@ A A´ A@ : AD>C 0C$;CT=C fÆW,ÔAD$@D4:D$CD 0 CD;D@ AC <C@3C@ BC 1-C@0C@5C´0,
            <MudTabPanel @key="tab" ID="@localIndex" Class="h-100 d-flex flex-column">@
                <TabContent>@
                    <div class="d-flex align-center gap-2" style="cursor: pointer;">@
                        @if (!string.IsNullOrEmpty(tab.Icon))@
                        {@
                            <MudIcon Icon="@tab.Icon" Size="Size.Small" />@
                        }@
                        <MudText Typo="Typo.body2" Class="font-weight-
medium">@tab.Title</MudText>@
@
                            <MudIconButton Icon="@Icons.Material.Filled.Close"@
                                Size="Size.Small"@
                                Color="Color.Default"@
                                Class="pa-0 ms-1"@
                                Style="width: 16px; height: 16px; min-width:
16px;"@
                                OnClick="@((e) =>
TabService.CloseTabByIndex(localIndex))" />@
                            </div>@
                        </TabContent>@
                    <ChildContent>@
                        @* B@BC@B C@>C@BCT9C@5D 6CTAD$:C@ 7C =C,,<C 5D" R 2D´AC@BD² 2C@;C 4C@8 C,
                        <div class="pa-2 rounded-b-lg w-100 d-flex flex-column flex-grow-1"@
                            Style="height: 100%; max-height: 100%; min-height: 0;
overflow: hidden;">@
@
                            <ErrorBoundary>@
                                <ChildContent>@
                                    @* A,,!A@ A A´ A@ : A@K C@1C@C GC,,2C 5C@ G-æ Ô-46@× @æVÇB 2 DD8C
                                    @* C@>D$>D KC´ AC,,;C@9 C$KD$0C48C$0CTB D BD 0C@8DdC C@0D,,5C4> CD5
                                    <div class="d-flex flex-column flex-grow-1 w-100"
style="height: 100%; min-height: 0;">@
                                        <DynamicComponent Type="@tab.ComponentType" />@
                                    </div>@
                                </ChildContent>@
                                <ErrorContent Context="exception">@
                                    <MudAlert Severity="Severity.Error" Class="my-4">@
                                        A@C@8Ct>D,,;C :D 8D$8Dt5D :C 0 C@HC,,1C@0 C,,=D$5D DCT9D 0 C$:
                                    </MudAlert>@
                                </ErrorContent>@
                            </ErrorBoundary>@
@
                            </div>@
                        </ChildContent>@
                    </MudTabPanel>@
                }@
            </MudTabs>@
        }@
    }@
}

```

```

        </MudMainContent>@
    </MudLayout>@
</CascadingValue>@
@
@code {@
    private bool _drawerOpen = true;@
    private bool _isDarkMode; // A@> D4<Cä;Dt@C08Dâ ?D 8C´>Cd5C08CR 7C ?D4AC@0CTBD 0 C" AC$5D$;Cä9 D$5CÄ5D
@
    protected override void OnInitialized()@
    {@
        base.OnInitialized();@
        TabService.OnTabsChanged += HandleTabsChanged;@
        _drawerOpen = false;@
    }@
@
    private void ToggleDrawer() => _drawerOpen = !_drawerOpen;@
@
    /// <summary>@
    /// A@5D 5C@;DäGC 5D" DC´0C2 BE <C0>C4> D 5Cd8CÄ0 C, 7C AD$0C$;D@5D" ×VEF†VÖU &÷f-FW" <C4=Cä2CT=C0> C05D
    /// </summary>@
    private void ToggleDarkMode() => _isDarkMode = !_isDarkMode;@
@
    private void HandleTabsChanged() => InvokeAsync(StateHasChanged);@
@
    public void CloseDrawer()@
    {@
        if (_drawerOpen)@
        {@
            _drawerOpen = false;@
            StateHasChanged();@
        }@
    }@
@
    public void Dispose() => TabService.OnTabsChanged -= HandleTabsChanged;@
}
</file>

```

```

<file path="Configuration.Web/Components/Layout/ReconnectModal.razor">
<script type="module" src="@Assets["Components/Layout/ReconnectModal.razor.js"]"></script>@
@
<dialog id="components-reconnect-modal" data-nosnippet>@
    <div class="components-reconnect-container">@
        <div class="components-rejoining-animation" aria-hidden="true">@
            <div></div>@
            <div></div>@
        </div>@
        <p class="components-reconnect-first-attempt-visible">@
            Rejoining the server...@
        </p>@
        <p class="components-reconnect-repeated-attempt-visible">@
            Rejoin failed... trying again in <span id="components-seconds-to-next-attempt"></span>
seconds.@
        </p>@
        <p class="components-reconnect-failed-visible">@
            Failed to rejoin.<br />Please retry or reload the page.@
        </p>@
        <button id="components-reconnect-button" class="components-reconnect-failed-visible">@
            Retry@
        </button>@
        <p class="components-pause-visible">@
            The session has been paused by the server.@
        </p>@
        <p class="components-resume-failed-visible">@
            Failed to resume the session.<br />Please retry or reload the page.@
        </p>@
        <button id="components-resume-button" class="components-pause-visible components-resume-
failed-visible">@
            Resume@
        </button>@
    </div>@
</dialog>
</file>

```

```

<file path="Configuration.Web/Components/NavMenu.razor">
@using System.Reflection@
@inject UiModuleService UiService@

```

```

@Inject TabNavigationService TabService
@Inject ComponentRegistry Registry
{
    <MudNavMenu>
    {
        List<(string Title, string Href, string Icon, string? Group)> allItems = UiService.Modules
            .SelectMany(m => m.GetMenuItems())
            .ToList();

        // AäBD 8D >C$KC$0CT< D0;CT<CT=D$K C$5D EC05C4> D4@Cä2C00
        IEnumerable<(string Title, string Href, string Icon, string? Group)> topLevelItems =
        allItems.Where(i => string.IsNullOrEmpty(i.Group));
        foreach ((string Title, string Href, string Icon, string? Group) item in topLevelItems)
        {
            <!-- A,,!A0 A A' A0 : A$<CTAD$> Href C,,AC0>C'LCtCCT< OnClick CD;D0 >D$:D KD$8D0 2C;C 4Cä: C 5Cr
            <MudNavLink Icon="@item.Icon" OnClick="@(() => OpenMenuAsTab(item.Title, item.Href,
            item.Icon))">
                @item.Title
            </MudNavLink>
        }

        // A4@D4?C08D CCT< D0;CT<CT=D$K C0> C00D$5C4>D 8D0<D
        IOrderedEnumerable<IGrouping<string, (string Title, string Href, string Icon, string?
        Group)>> pooledGroups = allItems
            .Where(i => !string.IsNullOrEmpty(i.Group))
            .GroupBy(i => i.Group!)
            .OrderBy(g => g.Key);

        foreach (IGrouping<string, (string Title, string Href, string Icon, string? Group)> group
        in pooledGroups)
        {
            <MudNavGroup Title="@group.Key" Icon="@Icons.Material.Filled.Folder" Expanded="false">
                @foreach ((string Title, string Href, string Icon, string? Group) item in group)
                {
                    <!-- A,,!A0 A A' A0 : B41D 0C0 ‡&VbÂ 4Cä1C 2C'5C0 ää6Æ-6² 00í
                    <MudNavLink Icon="@item.Icon" OnClick="@(() => OpenMenuAsTab(item.Title,
                    item.Href, item.Icon))">
                        @item.Title
                    </MudNavLink>
                }
            </MudNavGroup>
        }
    }
</MudNavMenu>
}

@code {
    /// <summary>
    /// A05D 5DT2C BD'2C 5CÄ AC02Cä7C0>C' MC07CT<C0;D0@ D >CD8D$5C'LD :Cä3Cä <C :CTBC
    /// </summary>
    [CascadingParameter]
    protected MainLayout? MainLayoutInstance { get; set; }

    protected override void OnInitialized()
    {
        base.OnInitialized();
        IEnumerable<Assembly> assemblies = UiService.Modules.Select(m =>
        m.GetType().Assembly).Distinct();
        Registry.RegisterModules(assemblies);
    }

    private void OpenMenuAsTab(string title, string href, string icon)
    {
        Type? componentType = Registry.GetComponentTypeByRoute(href);

        if (componentType != null)
        {
            // AäBC0@D'2C 5CÄ 2C;C 4C0C
            TabService.OpenTab(title, href, icon, componentType);
        }

        // A 2D$>CÄ0D$8Dt5D :C, 7C :D KC$0CT< C >C0>C$>CR <CT=Dä ?Cä2CT@DR >C0=C ?CäAC'5 C0;C,,:C
        MainLayoutInstance?.CloseDrawer();
    }
    else
    {
        System.Diagnostics.Debug.WriteLine($"A0CD$L {href} C05 Ct0D 5C48D BD 8D >C$0C0 2 C0>CÄ?Cä=CT=D$0D
    }
}

```

```

    }
}
</file>

<file path="Configuration.Web/Components/NotFound.razor">
@page "/not-found"
@layout MainLayout
@
<PageTitle>404 - B BD 0C08Dd0 C05 C00C"4CT=C Ã ÷ VF-FÆÍ
@
<MudContainer MaxWidth="MaxWidth.Small" Class="d-flex align-center justify-center" Style="height:
70vh;">
    <MudPaper Elevation="3" Class="pa-8 text-center" Style="width: 100%;">
        <MudIcon Icon="@Icons.Material.Filled.SearchOff" Color="Color.Error" Size="Size.Large"
Class="mb-4" />
        <MudText Typo="Typo.h5" Class="mb-2">AäHC,,1C00 404</MudText>
        <MudText Typo="Typo.body2" Class="mud-text-secondary mb-6">
            A,,7C$8C08D$5, Ct0C0@C HC,,2C 5CÄKC' 0CD@CTA C05 D CD"5D BC$CCTB, C'8C > D >CäBC$5D$AD$2D4ND"8C' <C
        </MudText>
        <MudButton Variant="Variant.Filled" Color="Color.Primary" href="/">D
            A$5D =D4BDÄAD0 =C 3C'0C$=D4ND
        </MudButton>
    </MudPaper>
</MudContainer>
</file>

<file path="Configuration.Web/Components/Routes.razor">
@using System.Reflection
@inject UiModuleService UiService
@
<Router AppAssembly="@typeof(Routes).Assembly"
    AdditionalAssemblies="@_moduleAssemblies"
    NotFoundPage="@typeof(NotFound)">
    <Found Context="routeData">
        <RouteView RouteData="@routeData" DefaultLayout="@typeof(MainLayout)" />
        <FocusOnNavigate RouteData="@routeData" Selector="h1" />
    </Found>
</Router>
@
@code {
    private Assembly[] _moduleAssemblies = [];
    protected override void OnInitialized()
    {
        _moduleAssemblies = UiService.Modules
            .Select(m => m.GetType().Assembly)
            .Distinct()
            .ToArray();
        base.OnInitialized();
    }
}
</file>

<file path="Configuration.Web/Configurator/IWebAppConfigurator.cs">
namespace Promatis.Net.Configuration.Web;
@
public interface IWebAppConfigurator : IAppConfigurator
{
    void ConfigureMiddleware(WebApplication app);
}
</file>

<file path="Configuration.Web/Configurator/WebAppConfigurator.cs">
using System.Reflection;
using Microsoft.AspNetCore.Components;
using MudBlazor.Services;
@
namespace Promatis.Net.Configuration.Web;
@
/// <summary>
/// A 0Ct>C$KC' :Cä=DD8C4CD 0D$>D 4C'0 Web-C0@C,,;Cä6CT=C,,9 C00 C 0Ct5 Blazor C, xVD&Æ |÷"í
/// </summary>
public class WebAppConfigurator<TRootComponent> : AppConfigurator, IWebAppConfigurator
    where TRootComponent : IComponent
{

```



```

    public override void ConfigureServices(IServiceCollection services, IConfiguration
configuration)ð
    {ð
        // 1. A 0Ct>C$0Dð 8C0DD 0D BD CC=BD4@C „ ABÂ !CT@C$8D K, A$0C´8CD0D$>D K, B$@C„3C45D K, B :C =CT@ DL
        base.ConfigureServices(services, configuration);ð
ð
        // 2. UI C„=DD@C AD$@D4:D$CD 0 (Að>C„AC¢ •V"ÖöGVÆR 2 D 1Cä@C=0DRÂ =C 2C„3C FC„0)ð
        services.AddWebInfrastructure(projectPrefix: "Promatis.");ð
ð
        // B B "AT A A A Aä (MDI): B 5C48D BD 8D CCT< C„=DD@C AD$@D4:D$CD C CÄ=Cä3Cä2C¢;C 4CäGCÔ>C4> C„
        services.AddSingleton<ComponentRegistry>();ð
        services.AddScoped<TabNavigationService>();ð
ð
        // 3. B ?CTFC„DC„:C &Æ |÷" „-çFW& 7F-fR 6W'fW"•
        services.AddRazorComponents()ð
            .AddInteractiveServerComponents();ð
ð
        // 4. B 5C48D BD 0Dd8Dð 8 C00D BD >C":C xVD&Æ |÷-
        services.AddMudServices(config =>ð
        {ð
            config.SnackbarConfiguration.PositionClass =
MudBlazor.Defaults.Classes.Position.BottomRight;ð
            config.SnackbarConfiguration.PreventDuplicates = false;ð
            config.SnackbarConfiguration.NewestOnTop = true;ð
            config.SnackbarConfiguration.ShowCloseIcon = true;ð
            config.SnackbarConfiguration.VisibleStateDuration = 5000;ð
        });ð
ð
        // 5. B„8C00 UI-D >C KD$8C•
        services.AddScoped<IUiCommandBus, UiCommandBus>();ð
ð
        // B 5C48D BD 0Dd8Dð AC„AD$5CÄ=Cä3Cä 0C=ACTAD >D 0 CD;Dð ?Cä4CD5D 6C=8 C´5C08C$>C4> D 0Ct@CTHCT=C„O S
        services.AddHttpContextAccessor();ð
    }ð
ð
    /// <summary>ð
    /// A00D BD >C":C :Cä=C$5C"5D 0 Cä1D 0C >D$:C, …EE Ô7C ?D >D >C" „Ö-FFÆWV &R´í
    /// </summary>ð
    public virtual void ConfigureMiddleware(WebApplication app)ð
    {ð
        // A 0Ct>C$KCR Ö-FFÆWV &W2 4C´O D 0C >D$K Web-Cô@C„;Cä6CT=C„0ð
        app.UseStaticFiles();ð
        app.UseAntiforgery();ð
ð
        // 1. Að>C´CDt0CT< Ct0D 5C48D BD 8D >C$0C0=D´5 UI-D 1Cä@C=8 C„7 DI-C= >C0BCT9C05D 0 DT>D BC
        using IServiceScope scope = app.Services.CreateScope();ð
        var uiService = scope.ServiceProvider.GetRequiredService<UiModuleService>();ð
ð
        Assembly[] moduleAssemblies = uiService.Modulesð
            .Select(m => m.GetType().Assembly)ð
            .Distinct()ð
            .ToArray();ð
ð
        // A„ A„&A„ A´ At Bd Bð A B$+ A$ A´ AD A£¢ C ?Cä;C00CT< D 5CTAD$@ D >D4BCä2 C= >CÄ?Cä=CT=D$0CÄ8 C„7
        var componentRegistry = scope.ServiceProvider.GetRequiredService<ComponentRegistry>();ð
        componentRegistry.RegisterModules(moduleAssemblies);ð
ð
        // 2. B 5C48D BD 8D CCT< C= >D =CT2Cä9 C= >CÄ?Cä=CT=D" 8 D :C =C„@D45CÄ AD$@C =C„FD² <Cä4D4;CT9 C00 D 5
        app.MapRazorComponents<TRootComponent>()ð
            .AddInteractiveServerRenderMode()ð
            .AddAdditionalAssemblies(moduleAssemblies);ð
    }ð
ð
    public override void ConfigureApp(IHost app)ð
    {ð
        // A$Kct>C" 1C 7Cä2Cä9 C„=C„FC„0C´8Ct0Dd8C, „0C$BCä@CT3C„AD$@C FC„O D$@C„3C45D >C" AB GCT@CT7 D :C =
        base.ConfigureApp(app);ð
ð
        // At4CTADÂ <Cä6C0> CD>C 0C$8D$L C0@Cä2CT@C= C Ct4Cä@Cä2DÄ0 A C„;C, =C GC ;DÄ=D´9 Seed CD0C0=D´E CD;
    }ð
}
</file>

```

```

<file path="Configuration.Web/Configurator/WebScanningExtensions.cs">
using Promatis.Net.UI;ð
using System.Reflection;ð

```

```

Ð
namespace Promatis.Net.Configuration.Web;Ð
Ð
public static class WebInfrastructureExtensionsÐ
{Ð
    public static void AddWebInfrastructure(this IServiceCollection services, string projectPrefix
= "Promatis.")Ð
    {Ð
        Console.WriteLine("[SCANNER] At0C0CD : C0@Cä3D 5C$0 D 1Cä@Cä: CÄ>CÔ>C´8D$0 (B$>C´LC= UI-CÄ>CDCC´8)..
Ð
        string binPath = AppContext.BaseDirectory;Ð
Ð
        // A,,!Aô A A´ Aô : Aô0 D4@Cä2C05 DD0C";Cä2Cä9 D 8D BCT<D² 8D"5CÄ BCä;DÄ:Cä BCR FÆÄÄ :CäBCä@D´5 CÔ0Dt
        string[] assemblyFiles = Directory.GetFiles(binPath, $"{projectPrefix}*.UI.dll",
SearchOption.TopDirectoryOnly);Ð
Ð
        foreach (string file in assemblyFiles)Ð
        {Ð
            tryÐ
            {Ð
                // Aô@C,,=D44C,,BCT;DÄ=Cä 7C 3D CCd0CT< C" 76VÖ&Ç"Æö D6öçFW‡BäFVf VÇM
                Assembly.LoadFrom(file);Ð
            }Ð
            catch (Exception ex)Ð
            {Ð
                Console.WriteLine($"[SCANNER] Aô5 D44C ;CäADÄ 7C 3D Cct8D$L DD0C"; D 1Cä@C=8 {Path.GetFileName
}Ð
            }Ð
        }Ð
Ð
        // A,,!Aô A A´ Aô : BD8C´LD$@D45CÄ FöÖ -âÄ 1CT@Dò AC >D :C, AD$@Cä3Cä A Cö@CTDC,,:D >CÄ 8 Cä:Cä=Dt0
        Assembly[] assemblies = AppDomain.CurrentDomain.GetAssemblies()Ð
        .Where(a => a.FullName != nullÐ
            && a.FullName.StartsWith(projectPrefix)Ð
            && a.GetName().Name!.EndsWith(".UI"))Ð
        .Distinct()Ð
        .ToArray();Ð
Ð
        Console.WriteLine($"[SCANNER] B :C =C,,@Cä2C =C,,5 Ct0C$5D HCT=Cää C 9CD5CÔ> {assemblies.Length} Dd5C´
Ð
        Console.WriteLine();Ð
Ð
        Console.WriteLine("[SCANNER] B 5C48D BD 0Dd8Dò T´Ô:Cä<Cô>CÔ5CÔBCä2 C, <Cä4D4;CT9 CÔ0C$8C40Dd8C, GCT@C
Ð
        // A 2D$>CÄ0D$8Dt5D :Cä5 D :C =C,,@Cä2C =C,,5 C, @CT3C,,AD$@C FC,,O C,,=D$5D DCT9D >C" •V"ööGVÆ]
        services.Scan(scan => scanÐ
            .FromAssemblies(assemblies)Ð
            .AddClasses(c => c.AssignableTo<IUiModule>().Where(t => !t.IsAbstract))Ð
            .AsImplementedInterfaces()Ð
            .WithScopedLifetime()Ð
        );Ð
Ð
        // B 5C48D BD 0Dd8Dò 2C HCT3Cä 0C4@CT3C BCä@C <Cä4D4;CT9D
        services.AddScoped<UiModuleService>();Ð
Ð
        // A´>C48D >C$0CÔ8CR >C =C @D46CT=CÔKDR ?D4=C=BCä2 CÄ5CÔNÐ
        IEnumerable<Type> uiModuleTypes = assembliesÐ
            .SelectMany(a => a.GetTypes())Ð
            .Where(t => typeof(IUiModule).IsAssignableFrom(t) && !t.IsInterface && !t.IsAbstract);Ð
Ð
        foreach (Type type in uiModuleTypes)Ð
        {Ð
            tryÐ
            {Ð
                if (Activator.CreateInstance(type) is IUiModule module)Ð
                {Ð
                    Console.WriteLine($"[SCANNER] UI ÄÄ>CDCC´L: {module.Name}");Ð
                    List<(string Title, string Href, string Icon, string? Group)> menuItems =
module.GetMenuItems()?.ToList() ?? new();Ð
Ð
                    if (!menuItems.Any())Ð
                    {Ð
                        Console.WriteLine(" % % [AôCD BCä5 CÄ5CÔN]");Ð
                        continue;Ð
                    }Ð
Ð
                    IEnumerable<(string Title, string Href, string Icon, string? Group)> rootItems

```

```

= menuItems.Where(i => string.IsNullOrEmpty(i.Group));
IEnumerable<IGrouping<string?, (string Title, string Href, string Icon, string?
Group)>> groupedItems = menuItems.Where(i => !string.IsNullOrEmpty(i.Group)).GroupBy(i => i.Group);
foreach ((string Title, string Href, string Icon, string? Group) item in
rootItems)
{
    Console.WriteLine($"          % % {item.Title} -> {item.Href}");
}
foreach (IGrouping<string?, (string Title, string Href, string Icon, string?
Group)> group in groupedItems)
{
    Console.WriteLine($"          % % [{group.Key}]");
    List<(string Title, string Href, string Icon, string? Group)> itemsInGroup
= group.ToList();
    for (int i = 0; i < itemsInGroup.Count; i++)
    {
        string prefix = (i == itemsInGroup.Count - 1) ? "          % % % " :
"          % % % ";
        Console.WriteLine($"{prefix} {itemsInGroup[i].Title} ->
{itemsInGroup[i].Href}");
    }
}
}
catch (Exception ex)
{
    Console.WriteLine($"[SCANNER] AäHC,,1C=0 DtBCT=C,,0 CÄ5CÔN CD;Dò BC,,?C .G- Räæ ÖWÓ¢ ¶W,äÖW76 v
");
}
}
Console.WriteLine("[SCANNER] B 5C48D BD 0Dd8Dò T'Ô:Cä<Cô>CÔ5CÔBCä2 D4ACô5D,,=Cä 7C 2CT@D,,5CÔ0");
Console.WriteLine();
}
}
</file>

<file path="Configuration.Web/Promatis.Net.Configuration.Web.csproj">
<Project Sdk="Microsoft.NET.Sdk.Web">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
    </PropertyGroup>
    <ItemGroup>
        <PackageReference Include="MudBlazor" Version="9.4.0" />
    </ItemGroup>
    <ItemGroup>
        <ProjectReference Include="..\Configuration\Promatis.Net.Configuration.csproj" />
        <ProjectReference Include="..\Promatis.Net.UI\Promatis.Net.UI.csproj" />
    </ItemGroup>
</Project>
</file>

<file path="Configuration.Web/Properties/launchSettings.json">
{
    "profiles": {
        "Promatis.Net.Configuration.Web": {
            "commandName": "Project",
            "launchBrowser": true,
            "environmentVariables": {
                "ASPNETCORE_ENVIRONMENT": "Development"
            },
            "applicationUrl": "https://localhost:56819;http://localhost:56820"
        }
    }
}
</file>

<file path="Configuration.Web/Services/TabNavigationService/ComponentRegistry.cs">
using System.Reflection;

```

```

using Microsoft.AspNetCore.Components;
{
namespace Promatis.Net.Configuration.Web;
{
public class ComponentRegistry
{
    private readonly Dictionary<string, Type> _routeMap = new(StringComparer.OrdinalIgnoreCase);

    /// <summary>
    /// B :C =C,,@D45D" AC >D :C, <Cä4D4;CT9 C, AD$@Cä8D" :C @D$: URL -> B Æ 72 "C,,? C@>CÄ?Cä=CT=D$0D
    /// </summary>
    public void RegisterModules(IEnumerable<Assembly> assemblies)
    {
        foreach (Assembly assembly in assemblies)
        {
            IEnumerable<Type> componentTypes = assembly.GetTypes()
                .Where(t => typeof(IComponent).IsAssignableFrom(t) && !t.IsAbstract);

            foreach (Type type in componentTypes)
            {
                // A,,ICT< C BD 8C CD" µ&÷WFT GG&- 'WfU0Â 2 C@>D$>D KC' :Cä<C08C'8D CCTBD 0 CD8D 5C@BC,,2C
                IEnumerable<RouteAttribute> routeAttributes =
                    type.GetCustomAttributes<RouteAttribute>();
                foreach (RouteAttribute attr in routeAttributes)
                {
                    _routeMap[attr.Template] = type;
                }
            }
        }
    }

    public Type? GetComponentTypeByRoute(string route)
    {
        return _routeMap.TryGetValue(route, out Type? type) ? type : null;
    }
}
</file>

<file path="Configuration.Web/Services/TabNavigationService/TabItem.cs">
namespace Promatis.Net.Configuration.Web;
{
public class TabItem
{
    public string Id { get; init; } = string.Empty; // A00D, ‡&Vm
    public string Title { get; init; } = string.Empty;
    public string Icon { get; init; } = string.Empty;
    public Type ComponentType { get; init; } = null!;
}
</file>

<file path="Configuration.Web/Services/TabNavigationService/TabNavigationService.cs">
namespace Promatis.Net.Configuration.Web;
{
public class TabNavigationService
{
    public List<TabItem> OpenTabs { get; } = new();
    public int ActiveTabIndex { get; set; }
    public event Action? OnTabsChanged;

    public void OpenTab(string title, string href, string icon, Type componentType)
    {
        TabItem? existingTab = OpenTabs.FirstOrDefault(t => t.Id.Equals(href,
            StringComparison.OrdinalIgnoreCase));

        if (existingTab != null)
        {
            ActiveTabIndex = OpenTabs.IndexOf(existingTab);
        }
        else
        {
            var newTab = new TabItem
            {
                Id = href,
                Title = title,
                Icon = icon,
                ComponentType = componentType
            }
        }
    }
}
}

```

```

        };
        OpenTabs.Add(newTab);
        ActiveTabIndex = OpenTabs.Count - 1;
    }

    NotifyStateChanged();
}

/// <summary>
/// A 5Ct>C00D =Că5 Ct0C@D`BC,,5 C$:C`0CD:C, ?Că 5E GC,,AC`>C$>CĂC C,,=CD5C=AD=
/// </summary>
public void CloseTabByIndex(int index)
{
    if (index < 0 || index >= OpenTabs.Count) return;

    OpenTabs.RemoveAt(index);

    // A,,=D$5C`;CT:D$CC ;DĂ=D`9 D 0D GCTB DD>C=CD 0 CÔ0 CăAD$0C$HC,,5D 0 C$:C`0CD:C•
    if (ActiveTabIndex >= OpenTabs.Count)
    {
        ActiveTabIndex = OpenTabs.Count - 1;
    }

    if (ActiveTabIndex < 0 && OpenTabs.Count > 0)
    {
        ActiveTabIndex = 0;
    }

    NotifyStateChanged();
}

private void NotifyStateChanged() => OnTabsChanged?.Invoke();
}
</file>

<file path="Configuration.Web/Services/UiModuleService.cs">
using Promatis.Net.UI;
namespace Promatis.Net.Configuration.Web;
// B 5D 2C,,A, C=D$>D KC' ACă1C,,@C 5D" 2D Q C$>CT4C,,=Cı
public class UiModuleService(IEnumerable<IUiModule> modules)
{
    public IEnumerable<IUiModule> Modules => modules;
}
</file>

<file path="Configuration.Web/UiCommandBus/IUiCommandBus.cs">
using MudBlazor;
namespace Promatis.Net.Configuration.Web;
/// <summary>
/// A,,=D$5D DCT9D HC,,=D² T'ÔACă1D`BC,,9D
/// </summary>
public interface IUiCommandBus
{
    /// <summary>
    /// B 5C48D BD 0Dd8D0 >C @C 1CăBDt8C=0 C$Kct>C$0 DD>D <D² ,,2D`?Că;CÔ0CTBD 0 Cô@C,,=C,,<C ND"8CĂ <Că4D4;CT<,
    /// </summary>
    void RegisterHandler(string commandName, Func<Dictionary<string, object>, Task<DialogResult?>> handler);
}

/// <summary>
/// A$7C 8CĂ>CD5C"AD$2C,,5 CĂ5Cd4D2 <Că4D4;Dô<C, 2D ;CT?D4N (C$KctKC$0CTBD 0 C,,7 MDM)D
/// </summary>
Task<DialogResult?> ExecuteAsync(string commandName, Dictionary<string, object> parameters);
}
</file>

<file path="Configuration.Web/UiCommandBus/UiCommandBus.cs">
using MudBlazor;
namespace Promatis.Net.Configuration.Web;

```

```

/// <summary>Ð
/// B,,8C00 UI-D >C KD$8C•
/// </summary>Ð
public class UiCommandBus : IUiCommandBusÐ
{Ð
    // A0>D$>C¤>C 5Ct>C00D =D´9 D ;Cä2C @DÂ 4C´O D 5C48D BD 0Dd8C, :Cä<C =CB 2 D 0C0BC 9CÄ5 SignalRÐ
    private readonly System.Collections.Concurrent.ConcurrentDictionary<string,
Func<Dictionary<string, object>, Task<DialogResult?>>> _handlers = new();Ð
    Ð
    public void RegisterHandler(string commandName, Func<Dictionary<string, object>,
Task<DialogResult?>> handler)Ð
    {Ð
        _handlers[commandName] = handler ?? throw new ArgumentNullException(nameof(handler));Ð
    }Ð
    Ð
    public async Task<DialogResult?> ExecuteAsync(string commandName, Dictionary<string, object>
parameters)Ð
    {Ð
        if (_handlers.TryGetValue(commandName, out Func<Dictionary<string, object>,
Task<DialogResult?>>? handler))Ð
        {Ð
            return await handler(parameters);Ð
        }Ð
    }Ð
    Ð
    // ATAC´8 CÄ>CDCC´L (C00C0@C,,<CT@, DCA) C05 C0>CD:C´NDt5C0 2 C´0D4=Dt5D 5, D 8D BCT<C =CR CC00CD5D"A
    return null;Ð
}Ð
}
</file>

<file path="Configuration.Web/WebAppBootstrapper.cs">
using Microsoft.AspNetCore.Components;Ð
Ð
namespace Promatis.Net.Configuration.Web;Ð
Ð
// A00D ;CT4C08C¢ &ö÷G7G& W"Â :CäBCä@D´9 D4<CT5D" 7C ?D4AC¤0D$L Web-D 5D 2CT@Ð
public abstract class WebAppBootstrapper<TRootComponent> : AppBootstrapperÐ
    where TRootComponent : IComponentÐ
{Ð
    public override void Run(string[] args)Ð
    {Ð
        // B =Cä2C 7C 3D CCd0CT< DLL (C00 D ;D4GC 9 C0@D0<Cä3Cä 7C ?D4AC¤0)Ð
        // LoadProjectAssemblies ("Promatis.") D46CR 2D´7C$0C00 C" & 6RÄ'VâÄ
        // C0> Ct4CTADÄ <D² 8D ?Cä;DÄ7D45CÄ vV$ Ä-6 F-öä'V-ÆFW"í
    }Ð
    Ð
    // Aä Bô A "AT BÄ Aä¢ CT7 D0BCä3Cä F0Ö -â =CR CC$8CD8D" <Cä4D4;C, ?D 8 Type.GetTypeÐ
    LoadProjectAssemblies("Promatis.");Ð
    Ð
    WebApplicationBuilder builder = WebApplication.CreateBuilder(args);Ð
    List<IAppConfigurator> configs = GetConfigurators(builder.Configuration).ToList();Ð
    Ð
    foreach (IAppConfigurator cfg in configs)Ð
        cfg.ConfigureServices(builder.Services, builder.Configuration);Ð
    Ð
    WebApplication app = builder.Build();Ð
    Ð
    // A00D BD >C":C Ö-FFÆWv &R <Cä4D4;CT9Ð
    foreach (IWebAppConfigurator cfg in configs.OfType<IWebAppConfigurator>())Ð
    {Ð
        cfg.ConfigureMiddleware(app);Ð
    }Ð
    Ð
    foreach (IAppConfigurator cfg in configs)Ð
        cfg.ConfigureApp(app);Ð
    Ð
    app.Run();Ð
    }Ð
}
</file>

<file path="Configuration/AppBootstrapper.cs">
using Microsoft.Extensions.Configuration;Ð
using Microsoft.Extensions.Hosting;Ð
using System.Reflection;Ð
using System.Runtime.Loader;Ð
Ð

```

```

namespace Promatis.Net.Configuration;
{
    public abstract class AppBootstrapper
    {
        public virtual void Run(string[] args)
        {
            // 1. Aö A,, B4 A,, "AT BÄ A / At A4 B4 A Dll (DtBCä1D² G- RävWEG- R 8DR 2C,,4CT;)
            LoadProjectAssemblies("Promatis.");

            HostApplicationBuilder builder = Host.CreateApplicationBuilder(args);

            // 2. B4 Aô+A' Aä B A Aô$A,, B4 A "Aä Aä A,, JSON
            List<IAppConfigurator> configurators = GetConfigurators(builder.Configuration).ToList();

            foreach (IAppConfigurator config in configurators)
            {
                config.ConfigureServices(builder.Services, builder.Configuration);
            }

            IHost host = builder.Build();

            foreach (IAppConfigurator config in configurators)
            {
                config.ConfigureApp(host);
            }

            host.Run();
        }

        protected virtual IEnumerable<IAppConfigurator> GetConfigurators(IConfiguration configuration)
        {
            string[] typeNames = configuration.GetSection("LauncherSettings:Modules").Get<string[]>()
                ?? [];

            foreach (string name in typeNames)
            {
                // A,,ICT< D$8Cò 2Cä 2D 5DR 7C 3D CCd5C0=D´E D 1Cä@C=0D]
                Type? type = AppDomain.CurrentDomain.GetAssemblies()
                    .Select(a => a.GetType(name))
                    .FirstOrDefault(t => t != null);

                if (type != null && typeof(IAppConfigurator).IsAssignableFrom(type))
                {
                    if (Activator.CreateInstance(type) is IAppConfigurator instance)
                    {
                        yield return instance;
                    }
                }
                else
                {
                    Console.WriteLine($"[BOOTSTRAPPER][WARN] B$8Cò =CR =C 9CD5C0 8C´8 C05 C$0C´8CD5C0¢ ¶æ ÖW0""½
                }
            }
        }

        protected void LoadProjectAssemblies(string prefix)
        {
            string? path = Path.GetDirectoryName(Assembly.GetExecutingAssembly().Location);
            if (path == null) return;

            foreach (string file in Directory.GetFiles(path, $"{prefix}*.dll"))
            {
                try
                {
                    AssemblyLoadContext.Default.LoadFromAssemblyPath(file);
                }
                catch { /* A,,3C0>D 8D CCT< CäHC,,1C=8 Ct0C4@D47C=8 */ }
            }
        }
    }
}
</file>

```

```

<file path="Configuration/AppConfigurator.cs">
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;

```

```

using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using System.Text.Json;
using System.Text.Json.Serialization;

namespace Promatis.Net.Configuration;

public class AppConfigurator : IAppConfigurator
{
    public virtual void ConfigureServices(IServiceCollection services, IConfiguration configuration)
    {
        // B :C =C,,@D45CÂ 2D Q (C$:C'NDt0Dò VF-EG&-vvW" 2 Cò@Cä5C▯BCR F F •
        services.AddDomainInfrastructure(projectPrefix: "Promatis.");
    }

    // B 5C48D BD 8D CCT< C 0Ct>C$KCR ACT@C$8D Kð
    services.AddScoped<DatabaseTriggerService>();
    services.AddScoped<IDatabaseTriggerService>(sp =>
sp.GetRequiredService<DatabaseTriggerService>());
    services.AddScoped<DatabaseTriggerInterceptor>();

    // Aä Bd Af¢ ¥4ôâ =C AD$@Cä9C▯8ð
    services.AddSingleton(new JsonSerializerOptions
    {
        PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
        DefaultIgnoreCondition = JsonIgnoreCondition.WhenWritingNull,
        WriteIndented = false
    });

    // B A4 B "B Bd Bò A´ B$Aä AÄ AÔ Aä Aâ A´ AÔ B B #B" Aä!B$ A•
    services.AddSingleton<IEntityCloner, JsonEntityCloner>();
}

public virtual void ConfigureApp(IHost app)
{
    if (app == null) throw new ArgumentNullException(nameof(app));

    // A,,=C,,FC,,0C´8Ct8D CCT< C4;Cä1C ;DÄ=D´9 Service Locator Cò@C, AD$0D BCR ?D 8C´>Cd5C08Dý
    AppInfrastructure.Initialize(app.Services);

    using IServiceScope scope = app.Services.CreateScope();
    scope.ServiceProvider.AutoRegisterTriggers();
}
}
</file>

<file path="Configuration/AppInfrastructure.cs">
namespace Promatis.Net.Configuration;

/// <summary>
/// A4;Cä1C ;DÄ=C 0 D 8D BCT<C00Dò BCäGC▯0 CD>D BD4?C : C,,=DD@C AD$@D4:D$CD =D´< D 5D 2C,,AC < Cò;C BDD>D <D²
/// A,,=C,,FC,,0C´8Ct8D CCTBD 0 Cä4C0>C▯@C BC0> Cò@C, AD$0D BCR ?D 8C´>Cd5C08Dò 8 C,,AC▯;DäGC 5D" @C 7CDCC$0C08CR
/// </summary>
public static class AppInfrastructure
{
    private static IServiceProvider? _provider;
    private static readonly object _lock = new();

    /// <summary>
    /// Read-only CD>D BD4? C¢ 3C´>C 0C´LCÔ>CÄC C▯>C0BCT9C05D C Ct0C$8D 8CÄ>D BCT9.D
    /// </summary>
    public static IServiceProvider Provider
    {
        get
        {
            if (_provider == null)
            {
                throw new InvalidOperationException(
                    "A▯@C,,BC,,GCTAC▯0Dò >D,,8C :C ¢ -æg& 7G'V7GW&R =CR 8C08Dd8C ;C,,7C,,@Cä2C =. " +
                    "B41CT4C,,BCTADÂÂ GD$> CÄ5D$>CB -æg& 7G'V7GW&Rä-æ-F- Æ-¡R,' 2D´7C$0C0 2 Program.cs C,,;C
                );
            }
            return _provider;
        }
    }
}

/// <summary>

```



```

    /// Að>D$>Cæ>C 5Ct>Cð0D =C 0 C,,=C,,FC,,0C`8Ct0Dd8Dð ;Cä:C BCä@C â 0´7D´2C 5D$ADð AD$@Cä3Cä >CD8C0 @C 7 C0@
    /// </summary>ð
    public static void Initialize(IServiceProvider provider)ð
    {ð
        if (provider == null) throw new ArgumentNullException(nameof(provider));ð
        if (_provider != null) return;ð

        lock (_lock)ð
        {ð
            _provider ??= provider;ð
        }ð
    }ð
}
</file>

<file path="Configuration/IAppConfigurator.cs">
using Microsoft.Extensions.Configuration;ð
using Microsoft.Extensions.DependencyInjection;ð
using Microsoft.Extensions.Hosting;ð
ð
namespace Promatis.Net.Configuration;ð
ð
public interface IAppConfiguratorð
{ð
    void ConfigureServices(IServiceCollection services, IConfiguration configuration);ð
    void ConfigureApp(IHost app);ð
}
</file>

<file path="Configuration/Promatis.Net.Configuration.csproj">
<Project Sdk="Microsoft.NET.Sdk">ð
ð
    <PropertyGroup>ð
        <TargetFramework>net10.0</TargetFramework>ð
        <ImplicitUsings>enable</ImplicitUsings>ð
        <Nullable>enable</Nullable>ð
    .
    </PropertyGroup>ð
ð
ð
ð
"Ä-FVÔw&÷W í
' Ä 6¶ vU&VfW&Væ6R -æ6ÇVFS0$0-7&÷6ögBäW†FVç6-öç2äFW VæFVæ7"-æ|V7F-öâä '7G& 7F-öç2"
Version="10.0.8" />ð
' Ä 6¶ vU&VfW&Væ6R -æ6ÇVFS0$0-7&÷6ögBäW†FVç6-öç2ä†÷7F-æ" fW'6-öä0# ä ä," óí
' Ä 6¶ vU&VfW&Væ6R -æ6ÇVFS0$0-7&÷6ögBäW†FVç6-öç2ä†÷7F-æä '7G& 7F-öç2" fW'6-öä0# ä ä," óí
' Ä 6¶ vU&VfW&Væ6R -æ6ÇVFS0$0-7&÷6ögBäW†FVç6-öç2äÆövv-æä '7G& 7F-öç2" fW'6-öä0# ä ä," óí
' Ä 6¶ vU&VfW&Væ6R -æ6ÇVFS0%67'WF÷"" fW'6-öä0#rä ä " óí
"ÄÔ-FVÔw&÷W í
ð
ð
ð
"Ä-FVÔw&÷W í
' Ä &ö;V7E&VfW&Væ6R -æ6ÇVFS0"ääÄ6W'f-6UÄ &öÖ F-2ääWBâ6W'f-6Ræ77 &ö¢" óí
"ÄÔ-FVÔw&÷W í
ð
ð
</Project>
</file>

<file path="Configuration/ServiceScanningExtensions.cs">
using FluentValidation;ð
using Microsoft.Extensions.DependencyInjection;ð
using Promatis.Net.Data;ð
using Promatis.Net.Domain;ð
using Promatis.Net.Service;ð
using System.Reflection;ð
using System.Runtime.Loader;ð
ð
namespace Promatis.Net.Configuration;ð
ð
public static class ServiceScanningExtensionsð
{ð
    public static void AddDomainInfrastructure(this IServiceCollection services, string
projectPrefix = "Promatis.")ð
    {ð

```

```

Console.WriteLine();
Console.WriteLine("[SCANNER] At0C0CD : C 2D$>CÄ0D$8Dt5D :Cä9 D 5C48D BD 0Dd8C, 2 DI...");

// 1. A0 A,, B4 A,, "AT BÄ A / At A4 B4 A B Aä Aä D
string? path = Path.GetDirectoryName(Assembly.GetExecutingAssembly().Location);
if (path != null)
{
    foreach (string file in Directory.GetFiles(path, $"{projectPrefix}*.dll"))
    {
        try
        {
            AssemblyLoadContext.Default.LoadFromAssemblyPath(file);
        }
        catch (Exception ex)
        {
            string fileName = Path.GetFileName(file);
            Console.ForegroundColor = ConsoleColor.Red;
            Console.WriteLine($"[SCANNER][ERROR] A05 D44C ;CäADÄ 7C 3D CCT8D$L D 1Cä@C C {fileName}");
            Console.ResetColor();
        }
    }
}

// A 5D 5CÄ 2D 5 Ct0C4@D46CT=C0KCR AC >D :C, A C$0D,,8CÄ AC,,AD$5CÄ=D´< C0@CTDC,,:D >Cí
Assembly[] assemblies = AppDomain.CurrentDomain.GetAssemblies()
    .Where(a => a.FullName != null && a.FullName.StartsWith(projectPrefix))
    .Distinct()
    .ToArray();

int countBefore = services.Count;

// 2. A B$ AÄ B$ Bt B Aä B A A,, Aä A A,, A, AT A,,!B$ A &A,,/ Bt B Ar 45%UDö-
services.Scan(scan =>
{
    // A´>C=C0C´LC00D0 DD4=C=FC,,0 CD;D0 @CT:D4@D 8C$=Cä9 C0@Cä2CT@C=8 C$ACT9 Dd5C0>Dt:C, =C AC´5CD>C$0
    bool IsSubclassOfRawGeneric(Type generic, Type? toCheck)
    {
        while (toCheck != null && toCheck != typeof(object))
        {
            Type cur = toCheck.IsGenericType ? toCheck.GetGenericTypeDefinition() : toCheck;
            if (generic == cur) return true;
            toCheck = toCheck.BaseType;
        }
        return false;
    }

    scan.FromAssemblies(assemblies)
        // A,,!A0 A A´ A0 : B 5C48D BD 0Dd8D0 !AT A$ B A" A C0>CD4CT@Cd:Cä9 C4;D41Cä:Cä3Cä =C AC´5CD
        .AddClasses(classes => classes.Where(type =>
            !type.IsAbstract &&
            !type.IsGenericTypeDefinition &&
            IsSubclassOfRawGeneric(typeof(BaseService<,,>), type)))
        .AsSelf()
        .AsImplementedInterfaces()
        .WithScopedLifetime()

        // B 5C48D BD 0Dd8D0 A A,, A "Aä Aä (FluentValidation)
        .AddClasses(c => c.AssignableTo(typeof(IValidator<>))
            .Where(t => !t.IsAbstract && !t.IsGenericTypeDefinition))
        .AsImplementedInterfaces()
        .WithTransientLifetime()

        // B 5C48D BD 0Dd8D0 "B A4 AT Aä (Before/After Save)
        .AddClasses(c => c.AssignableToAny(typeof(IBeforeSaveTrigger<>),
            typeof(IAfterSaveTrigger<>)).Where(t => !t.IsAbstract))
        .AsSelfWithInterfaces()
        .WithScopedLifetime();
});

// --- B A4 B "B Bd B0 A´ A A´,A0 A4 A0 A´ Aä B $A0 A4 A$ A´ AD B$ B B0 B ---
// B 5C48D BD 8D CCT< C=C0Cf >D$:D KD$KC´ 4Cd5C05D 8Cfä "CT?CT@DÄ ?D 8 Ct0C0@CäACR •f Æ-F F÷#Ä Dä1Cä9A
// DI-C=C0BCT9C05D ääUB 3C @C =D$8D >C$0C0=Cä >D$4C AD" =C H C0>C´8CÄ>D DC0KC´ 4C,,AC05D$GCT@
services.AddScoped(typeof(IValidator<>), typeof(GlobalPolymorphicValidator<>));

// 3. A,,!A0 A A´ A0 Aä A, A AT A0 AR Aä A,, Aä A A,, B At#A´,B$ B$ A-
List<ServiceDescriptor> newRegistrations = services.Skip(countBefore).ToList();

```

```

var loggedTypes = new HashSet<string>();
foreach (ServiceDescriptor reg in newRegistrations)
{
    // A 5D 5CÂ @CT0C'LC0KC' BC,,? D 5C ;C,,7C FC,,8 (C" ?D 8Cä@C,,BCTBCR 8Cr -x ÆVÖçF F-öâG- RÂ 8C00Dt5
    Type? implType = reg.ImplementationType ?? reg.ImplementationInstance?.GetType();
    if (implType == null) continue;
    // --- A A A,, A$ A' AD B$ B A" 00Ý
    bool isValidator = reg.ServiceType.IsGenericType &&
        reg.ServiceType.GetGenericTypeDefinition() == typeof(IValidator<>);
    if (isValidator && loggedTypes.Add($"Val:{implType.FullName}"))
    {
        var inheritanceChain = new List<string>();
        Type? currentType = implType;
        while (currentType != null && currentType != typeof(object))
        {
            string typeName = currentType.Name.Contains('.') ? currentType.Name.Split('.')[0] : currentType.Name;
            inheritanceChain.Add(typeName);
            currentType = currentType.BaseType;
            if (typeName == "AbstractValidator") break;
        }
        string chainDisplay = string.Join(" -> ", inheritanceChain);
        Console.WriteLine($"[SCANNER] A$0C'8CD0D$>D ¢ ¶6† -äF-7 Æ -0 ¢ •f Æ-F F÷""½
        continue; // A'>C2 4C'O D0BCä9 Ct0C08D 8 C$KC$5CD5C0Â 8CD5CÂ : D ;CT4D4ND"5C•
    }
    // --- A A A,, B B A,,!Aä (A00D ;CT4C08Cα>C" & 6U6W'f-6R' 00Ý
    // A,,!A0 A A' A0 : B41D 0C0> Cd5D BCα>CR CD ;Cä2C,,5 reg.ServiceType == reg.ImplementationType,
    // D$0C¢ :C : D 5D 2C,,AD² BCT?CT@DÂ @CT3C,,AD$@C,,@D4ND$AD0 ?C @C <C, C'0D A + A,,=D$5D DCT9D
    bool isService = false;
    var serviceChain = new List<string>();
    Type? currentServiceType = implType;
    while (currentServiceType != null && currentServiceType != typeof(object))
    {
        string typeName = currentServiceType.Name.Contains('.') ?
currentServiceType.Name.Split('.')[0] : currentServiceType.Name;
        serviceChain.Add(typeName);
        if (currentServiceType.IsGenericType &&
currentServiceType.GetGenericTypeDefinition() == typeof(BaseService<,,>))
        {
            isService = true;
            break;
        }
        currentServiceType = currentServiceType.BaseType;
    }
    if (isService && loggedTypes.Add($"Srv:{implType.FullName}"))
    {
        string chainDisplay = string.Join(" -> ", serviceChain);
        Console.WriteLine($"[SCANNER] B 5D 2C,,A: {chainDisplay}");
        continue;
    }
    // --- A A A,, B$ A,, A4 B A" 00Ý
    bool isTrigger = implType.GetInterfaces().Any(i => i.IsGenericType &&
        (i.GetGenericTypeDefinition() == typeof(IBeforeSaveTrigger<>) ||
        i.GetGenericTypeDefinition() == typeof(IAfterSaveTrigger<>)));
    if (isTrigger && loggedTypes.Add($"Tri:{implType.FullName}"))
    {
        Console.WriteLine($"[SCANNER] B$@C,,3C45D ¢ ¶-x ÄG- Rää ÖW0""½
    }
}
Console.WriteLine("[SCANNER] A 2D$>CÄ0D$8Dt5D :C O D 5C48D BD 0Dd8D0 CD ?CTHC0> Ct0C$5D HCT=C â""½
Console.WriteLine();
}

```

```

public static void AutoRegisterTriggers(this IServiceProvider serviceProvider)
{
    Console.WriteLine("[AUTOREG] At0C0CD : C 2D$>CÄ0D$8Dt5D :Cä9 D 5C48D BD 0Dd8C, BD 8C43CT@Cä2 C" F F &
    var triggerService = serviceProvider.GetRequiredService<IDatabaseTriggerService>();
    // 1. A0>C,,AC 2D 5DR :C´0D ACä2 D$@C,,3C45D >C" 2 D 1Cä@C0DR &ö F-2ä-
    IEnumerable<Type> triggerTypes = AppDomain.CurrentDomain.GetAssemblies()
        .Where(a => a.FullName?.StartsWith("Promatis.") == true)
        .SelectMany(s => s.GetTypes())
        .Where(t => t.IsClass & true, IsAbstract: false } &&
            t.GetInterfaces().Any(i => i.IsGenericType &&
                (i.GetGenericTypeDefinition() ==
typeof(IBeforeSaveTrigger<>) ||
                i.GetGenericTypeDefinition() ==
typeof(IAfterSaveTrigger<>))))
        .ToList();
    MethodInfo? registerMethod =
typeof(IDatabaseTriggerService).GetMethod(nameof(IDatabaseTriggerService.Register));
    int totalBindings = 0;
    foreach (Type triggerType in triggerTypes)
    {
        // A00DT>CD8CÄ 2D 5 C,,=D$5D DCT9D K D$@C,,3C45D >C"Â :CäBCä@D´5 D 5C ;C,,7D45D" 4C =C0KC´ :C´0D A0
        IEnumerable<Type> interfaces = triggerType.GetInterfaces()
            .Where(i => i.IsGenericType &&
                (i.GetGenericTypeDefinition() == typeof(IBeforeSaveTrigger<>) ||
                i.GetGenericTypeDefinition() == typeof(IAfterSaveTrigger<>)));
        foreach (Type @interface in interfaces)
        {
            // A0>C´CDt0CT< D$8C0 AD4IC0>D BC, ...B 8Cr "&Vf÷&U6 fUG&-vvW#ÄCä•
            Type entityType = @interface.GetGenericArguments()[0];
            // Aä?D 5CD5C´0CT< D$8C0 BD 8C43CT@C 4C´0 C@C AC,,2Cä3Cä ;Cä3C
            string triggerKind = @interface.GetGenericTypeDefinition() ==
typeof(IBeforeSaveTrigger<>)
                ? "Before"
                : "After ";
            try
            {
                // A$KctKC$0CT< Register<TEntity, TTrigger>()
                MethodInfo genericRegister = registerMethod!.MakeGenericMethod(entityType,
triggerType);
                genericRegister.Invoke(triggerService, null);
                // A´ A4 B A$ A0 AR #B AT(A0 A´ B A$/At A•
                Console.WriteLine($"[AUTOREG] [{triggerKind}] {entityType.Name} <---
{triggerType.Name}");
                totalBindings++;
            }
            catch (Exception ex)
            {
                Console.ForegroundColor = ConsoleColor.Red;
                Console.WriteLine($"[AUTOREG][ERROR] AäHC,,1C0 C0@C,,2D07C08 {triggerType.Name} C0 ¶VçF-G•
{ex.InnerException?.Message ?? ex.Message}");
                Console.ResetColor();
            }
        }
    }
    Console.WriteLine($"[AUTOREG] At0C$5D HCT=Cäâ !Cä7CD0C0> C0@C,,2D07Cä:: {totalBindings}");
    Console.WriteLine();
}
}
</file>

```

```

<file path="Data/Configurations/AuditLogConfiguration.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Promatis.Net.Domain;
namespace Promatis.Net.Data;

```

```
public class AuditLogConfiguration : IEntityTypeConfiguration<AuditLog>
{
    public void Configure(EntityTypeBuilder<AuditLog> builder)
    {
        builder.Property(e => e.EntityName).HasMaxLength(100);
        builder.Property(e => e.Action).HasMaxLength(50);
        builder.Property(e => e.ChangedBy).HasMaxLength(100);

        // ChangesJson C AD$ C$;D 5C  1CT7 C`8C 8D$  (text/jsonb), D$ C  :C : D ?C,,AC : C,,7C 5C 5C 8C' <C 6CT
        builder.Property(e => e.ChangesJson).HasColumnType("jsonb");
    }
}
</file>

<file path="Data/Configurations/ModelBuilderConventions.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata;
using System.Linq.Expressions;
using Promatis.Net.Domain.Interface;
namespace Promatis.Net.Data;
public static class ModelBuilderConventions
{
    public static void ApplyGlobalConventions(this ModelBuilder modelBuilder)
    {
        foreach (IMutableEntityType entityType in modelBuilder.Model.GetEntityTypes())
        {
            Type type = entityType.ClrType;

            // A`8C 8D$K CD;D  AD$ C : (255 C > D4<C ;Dt C 8D  
            foreach (IMutableProperty property in entityType.GetProperties())
            {
                if (property.ClrType == typeof(string))
                {
                    if (property.GetMaxLength() == null)
                        property.SetMaxLength(255);
                }
            }

            // A 2D$>-C ;D GC, 4C`O Guid (ValueGeneratedNever, D$ C  :C : Id D >Ct4C 5D$AD  2 DomainObject)
            if (typeof(IDomainObjectHasKey<Guid>).IsAssignableFrom(type))
            {
                modelBuilder.Entity(type).Property("Id").ValueGeneratedNever();
            }

            // A B$ -BD A`,B$ C, A$"A   A  AT B 4C`O SoftDelete C 1D 5C BC 2 
            if (typeof(ISoftDeletable).IsAssignableFrom(type))
            {
                IMutableEntityType? baseType = entityType.BaseType;

                // B BC 2C,,< DD8C`LD$  D$>C`LC > C   C >D 5C L C,,5D  D EC,,8 (C$:C`NDt D   C AD$ C :D$=D`9 Uni
                if (baseType == null || !typeof(ISoftDeletable).IsAssignableFrom(baseType.ClrType))
                {
                    // B4AD$ C >C$:C VW'"f- FW-
                    modelBuilder.SetSoftDeleteFilter(type);

                    // B4AD$ C >C$:C 8C 4CT:D   C   DeletedAt CD;D  CD :C @CT=C,,O Ct C @C AC 2 
                    modelBuilder.Entity(type).HasIndex("DeletedAt");
                }
            }
        }
    }

    private static void SetSoftDeleteFilter(this ModelBuilder modelBuilder, Type entityType)
    {
        // AD;D  E B E , 2C 6C > D BC 2C,,BD  DC,,;D BD =C :C @CT=D   4C 6CR 5D ;C, >C   C AD$ C :D$=D`9.D
        // B4AC`>C$8CR -46  72 4C AD$ D$>Dt=C  
        if (entityType.IsClass)
        {
            modelBuilder.Entity(entityType).HasQueryFilter(GenerateFilter(entityType));
        }
    }

    private static LambdaExpression GenerateFilter(Type type)

```

```

    {
        ParameterExpression parameter = Expression.Parameter(type, "e");
        // A,,ACô>C`LCtCCT< Property CD;Dò 4CäAD$CCô0 C¢ FVÆWFD B GCT@CT7 C,,=D$5D DCT9D 8C`8 D 2Cä9D BC$>D
        MemberExpression property = Expression.Property(parameter,
nameof(ISoftDeletable.DeletedAt));
        ConstantExpression nullConstant = Expression.Constant(null, typeof(DateTime?));
        BinaryExpression body = Expression.Equal(property, nullConstant);
    }

    return Expression.Lambda(body, parameter);
}
}
</file>

<file path="Data/DbContext/ApplicationDbContext.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Promatis.Net.Domain;
namespace Promatis.Net.Data;
public class ApplicationDbContext(
    DbContextOptions options,
    IConfiguration configuration,
    IServiceProvider? serviceProvider = null)
    : DbContext(options)
{
    protected virtual string Schema => "public";
    public DbSet<AuditLog> AuditLogs => Set<AuditLog>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.HasDefaultSchema(Schema);
        modelBuilder.ApplyModuleConfigurations(this);
        modelBuilder.ApplyGlobalConventions();
        base.OnModelCreating(modelBuilder);
    }

    // A,,!Aô A A` Aô B` A, A AT Aô+A` A A,, Aô"D
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        base.OnConfiguring(optionsBuilder);

        // 1. Aô@Cä2CT@Dô5CÄâ GD$> CÄK C" @C =D$0C"<CRÂ 0 CÔ5 C" ?D >Dd5D ACR <C,,3D 0Dd8C, 4Ä•
        if (!EF.IsDesignTime && serviceProvider != null)
        {
            DatabaseTriggerInterceptor? interceptor = null;

            try
            {
                // Aô>CôKD$:C ¢ D >C CCT< D 0Ct@CTHC,,BDÂ =C ?D OCÄCDâ „5D ;C, ?D >C$0C"4CT@ Cä:C 7C ;D O S
                interceptor = serviceProvider.GetService(typeof(DatabaseTriggerInterceptor)) as
DatabaseTriggerInterceptor;
            }
            catch (InvalidOperationException)
            {
                // Aô>CôKD$:C #¢ D ;C, ?D >C$0C"4CT@ Cæ>D =CT2Cä9 (Root), D >Ct4C 5CÄ 66÷ R 2D CDT=D4N!D
                // BôBCâ 3C @C =D$8D CCTB, DtBCâ 66÷ VBô8CôBCT@Dd5CôBCâ@ D 0Ct@CTHC,,BD O C 5Cr >D,,8C >C¢ > Ro
                var scopeFactory = serviceProvider.GetService(typeof(IServiceScopeFactory)) as
IServiceScopeFactory;

                if (scopeFactory != null)
                {
                    // B >Ct4C 5CÄ 2D 5CÄ5Cô=D4N Cä1C`0D BDÂ 2C,,4C,,<CäAD$8D
                    using IServiceScope scope = scopeFactory.CreateScope();
                    interceptor =
scope.ServiceProvider.GetService(typeof(DatabaseTriggerInterceptor)) as DatabaseTriggerInterceptor;
                }
            }

            // ATAC`8 C,,=D$5D FCT?D$>D CD ?CTHCô> Cô0C"4CT= Cä4Cô8CÄ 8Cr ACô>D >C >C" r ?Cä4C¢;DäGC 5CÄ 5C4>
            if (interceptor != null)
            {
                optionsBuilder.AddInterceptors(interceptor);
            }
        }
    }
}

```

```

    }
}
</file>

```

```

<file path="Data/DbContext/ModelBuilderExtensions.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain.Interface;
using System.Reflection;
using Microsoft.EntityFrameworkCore.Metadata;
using Microsoft.EntityFrameworkCore.Metadata.Builders;

namespace Promatis.Net.Data;

public static class ModelBuilderExtensions
{
    public static void ApplyModuleConfigurations(this ModelBuilder modelBuilder, DbContext context)
    {
        // 1. B :C =C,,@D45CÂ BCä;DÄ:Câ AC >D :C, 4C =CÔKDR ?D >CT:D$0 Promatis.*.Data
        List<Assembly> dataAssemblies = AppDomain.CurrentDomain.GetAssemblies()
            .Where(a => {
                string? name = a.GetName().Name;
                return name != null &&
                    name.StartsWith("Promatis.") &&
                    name.EndsWith(".Data");
            }).ToList();

        // 2. Aô>C´CDtøCT< D$8CÔK D CD"=CäAD$5C'Â :CäBCä@D´5 Dô2CÔ> Cä1D=0C$;CT=D² 2 D$5C=CD"5CÂ :Cä=D$5C=AD$
        HashSet<Type> contextDbSetTypes = context.GetType()
            .GetProperties(BindingFlags.Public | BindingFlags.Instance)
            .Where(p => p.PropertyType.IsGenericType && p.PropertyType.GetGenericTypeDefinition()
                == typeof(DbSet<>))
            .Select(p => p.PropertyType.GetGenericArguments()[0])
            .ToHashSet();

        foreach (Assembly assembly in dataAssemblies)
        {
            // Aô@C,,<CT=Dô5CÂ :Cä=DD8C4CD ØDd8C, BCä;DÄ:Câ 4C´O D$5DR BC,,?Cä2, C= >D$>D KCR <C BCT@C,,øC´LCÔK (
            modelBuilder.ApplyConfigurationsFromAssembly(assembly, type =>
            {
                if (type is not { IsClass: true, IsAbstract: false, IsGenericTypeDefinition:
                    false })
                    return false;

                // AôøDT>CD8CÂ 8CÔBCT@DD5C"A C= >CÔDC,,3D4@C FC,,8 C, BC,,? D CD"=CäAD$8, C= >D$>D CDâ >CÔø CÔøD B
                Type? configInterface = type.GetInterfaces()
                    .FirstOrDefault(i => i.IsGenericType && i.GetGenericTypeDefinition() ==
                        typeof(IEntityTypeConfiguration<>));

                if (configInterface == null) return false;

                Type entityType = configInterface.GetGenericArguments()[0];

                // A= A,, "A,, 'AT!A= A' $A,, BÄ"B ¢ D ;C, =C AD$@C 8C$øCT<C O D CD"=CäAD$L C 1D BD øC=BCÔø,
                // Cô@Cä2CT@Dô5CÂÂ 5D BDÄ ;C, C CÔ5E ECäBDÄ >CD8CÔ 6C,,2Cä9 CÔøD ;CT4CÔ8C¢ 2 D$5C=CD"5CÂ F%6W
                if (entityType.IsAbstract)
                {
                    bool hasConcreteDerivedInDbSet = contextDbSetTypes.Any(t =>
                        t.IsSubclassOf(entityType));
                    if (!hasConcreteDerivedInDbSet) return false; // Aô@Cä?D4AC=øCT< C= >CÔDC,,3D4@C FC,,N C 1D
                }

                return type.GetConstructor(Type.EmptyTypes) != null;
            });
        }

        // 3. A B$ -A,, Aô B B A$ Aô AR´ B B B$ A$(A,,%» A B "B A= &A,, ø
        IEnumerable<Type> allAbstractEntities = dataAssemblies
            .SelectMany(a => a.GetTypes())
            .Where(t => t is { IsClass: true, IsAbstract: true } && !t.IsGenericTypeDefinition);

        foreach (Type abstractType in allAbstractEntities)
        {
            bool isUsedInCurrentContext = contextDbSetTypes.Any(t => t == abstractType ||
                t.IsSubclassOf(abstractType));

            if (!isUsedInCurrentContext)

```

```

        {
            modelBuilder.Ignore(abstractType);
        }
    }

    // 4. A B$ AÄ B$ Bt B A / AÔ B "B A" A At A´ B A$ AÔ B´% AD B A$,AT A, AÔ AT B A-
    foreach (IMutableEntityType entityType in modelBuilder.Model.GetEntityTypes())
    {
        Type? clrType = entityType.ClrType;
        if (clrType == null) continue;

        // A,,ICT< D 5C ;C,,7C FC,,N C,,=D$5D DCT9D 0 ITreeNode<>
        Type? treeInterface = clrType.GetInterfaces()
            .FirstOrDefault(i => i.IsGenericType && i.GetGenericTypeDefinition() ==
typeof(ITreeNode<>));

        if (treeInterface != null)
        {
            Type targetTreeType = treeInterface.GetGenericArguments()[0];

            // AÔ0D BD 0C,,2C 5CÂ BCä;DÄ:Câ =C 1C 7Cä2Cä< D4@Cä2C05 Cä1D=0C$;CT=C,,O D 2Cä9D BC" „GD$>C K
            if (clrType == targetTreeType)
            {
                EntityTypeBuilder builder = modelBuilder.Entity(clrType);

                // 1. AÔ0D BD >C":C AC$0Ct8 B >CD8D$5C´L-Aô>D$>CÄ>C-
                builder.HasOne("Parent")
                    .WithMany("Children")
                    .HasForeignKey("ParentId")
                    .OnDelete(DeleteBehavior.Restrict);

                // 2. A B$ -A,, AD A=!: B >Ct4C 5CÂ 8CÔ4CT:D 4C´O C$=CTHCÔ5C4> C=;DäGC &VçD-M
                // BÔBCâ CD :Cä@C,,B Cö>C,,AC¢ MC´5CÄ5CÔBCä2 Cö> ParentId Cö@C, @C 1CäBCR "Æöö·W 2 D 5D 2C
                builder.HasIndex("ParentId")
                    .HasDatabaseName($"IX_{clrType.Name}_ParentId");
            }
        }
    }
}
</file>

<file path="Data/Interceptors/ChangeEntryModel.cs">
namespace Promatis.Net.Data;

/// <summary>
/// AÄ>CD5C´L DD8C=AC FC,,8 C,,7CÄ5CÔ5CÔ8C´ „BD 0CÔACô>D BCÔKC´ >C JCT:D""Á ?D 5CDAD$0C$;DôND"0Dò ACÔ8CÄ>C¢ ACä
/// CD>CÄ5CÔ=Cä9 D CD"=CäAD$8 C" <Cä<CT=D" 5E 4Cä1C 2C´5CÔ8DòÄ <Cä4C,,DC,,:C FC,,8 C,,;C, CCD0C´5CÔ8Dòí
/// </summary>
internal class ChangeEntryModel
{
    /// <summary>
    /// A,,7CÄ5CÔ0CT<D´9 DÔ:Ct5CÄ?C´OD 4Cä<CT=CÔ>C4> Cä1D=5C=BC „AD4ICÔ>D BC,´í
    /// Aô@C,,2Cä4C,,BD 0 C¢ :Cä=C=CTBCÔ>CÄC D$8CÔC C$=D4BD 8 D$@C,,3C45D >C"í
    /// </summary>
    public object Entity { get; init; } = null!;

    /// <summary>
    /// B$5C=CD"8C´ AD$0D$CD 8Ct<CT=CT=C,,O D CD"=CäAD$8 C" AC,,AD$5CÄ5 (Added, Modified, Deleted, SoftDeleted)
    /// </summary>
    public EntityStateChangeEnum State { get; init; }

    /// <summary>
    /// A=;>C´;CT:Dd8Dò BCäGCTGCÔKDR 8Ct<CT=CT=C,,9 D 2Cä9D BC" „?Cä;CT9) D CD"=CäAD$8.
    /// At0Cô>C´=Dô5D$ADò 4CT;DÄBCä9 Ct=C GCT=C,,9 "B BC @Cä5 -> AÔ>C$>CR"â C´O CÔ>C$KDR AD4ICÔ>D BCT9 D >CD5
    /// </summary>
    public List<PropertyChangeInfo> Changes { get; init; } = [];

    /// <summary>
    /// A,,4CT=D$8DD8C=0D$>D „8CÄO C,,;C, ;Cä3C,,=) Cö>C´Lct>C$0D$5C´O, D >C$5D HC,,2D,,5C4> CD0CÔ=Cä5 CD5C"AD$2C
    /// </summary>
    public string? ChangedBy { get; init; }

    /// <summary>
    /// AD0D$0 C, 2D 5CÄO DD8C=AC FC,,8 C,,7CÄ5CÔ5CÔ8Dò 2 DD>D <C BCR UD2í
    /// </summary>

```



```

        public DateTime ChangedAt { get; init; }
    }
}
</file>

<file path="Data/Interceptors/DatabaseTriggerInterceptor.cs">
using System.Collections.Concurrent;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.ChangeTracking;
using Microsoft.EntityFrameworkCore.Diagnostics;
using Microsoft.Extensions.DependencyInjection;
using Promatis.Net.Domain.Interface;

namespace Promatis.Net.Data;

/// <summary>
/// A05D 5DT2C Bdt8C$ >C05D 0Dd8C' F$6öçFW†BÂ >C 5D ?CTGC,,2C ND"8C' @C 1CäBD2 $<Dô3C¤>C4> D44C ;CT=C,,0" (Soft
/// C0@CT4C$0D 8D$5C'LC0CDâ 2C ;C,,4C FC,,N C,,7CÄ5C05C08C' 8 D :C$>Ct=Câ5 D$@C,,3C45D =Câ5 D42CT4Cä<C'5C08CR AC,,
/// </summary>
/// <param name="scopeFactory">BD0C @C,,:C 4C'0 D >Ct4C =C,,0 C,,7Cä;C,,@Cä2C =C0KDR >C ;C AD$5C' 2C,,4C,,<CäAD$8
public class DatabaseTriggerInterceptor(IServiceScopeFactory scopeFactory) : SaveChangesInterceptor
{
    /// <summary>
    /// A0>D$>C¤>C 5Ct>C00D =Câ5 DT@C =C,,;C,,ICR 7C DC,,:D 8D >C$0C0=D'E C,,7CÄ5C05C08C' 2 D 0CÄ:C E D$5C¤CD"5C'
    /// A¤;DäG – D4=C,,:C ;DÄ=D'9 C,,4CT=D$8DD8C¤0D$>D MC¤7CT<C0;D0@C F$6öçFW†Bi
    /// </summary>
    private readonly ConcurrentDictionary<Guid, List<ChangeEntryModel>> _capturedChanges = new();

    /// <summary>
    /// A AC,,=DT@Cä=C0KC' ?CT@CTEC$0D$GC,,:, C$KC0>C'=Dô5CÄKC' Aâ DC :D$8Dt5D :Cä3Câ ACäED 0C05C08Dò 8Ct<CT=C
    /// AâBC$5Dt0CTB Ct0 C'>C48C¤C Soft Delete, DD8C¤AC FC,,N D =C,,<C¤0 C,,7CÄ5C05C08C' 8 Ct0C0CD : D$@C,,3C45D
    /// </summary>
    public override async ValueTask<InterceptionResult<int>> SavingChangesAsync(
        DbContextEventData eventData, InterceptionResult<int> result, CancellationToken ct =
        default)
    {
        if (eventData.Context == null) return result;

        // B >Ct4C 5CÄ ;Cä:C ;DÄ=D'9 C40D 0C0BC,,@Cä2C =C0> Cd8C$>C' 66÷ R 4C'0 C0@CT4CäBC$@C ICT=C,,0 C0@Cä1C'
        using IServiceScope scope = scopeFactory.CreateScope();
        var triggerService = scope.ServiceProvider.GetRequiredService<IDatabaseTriggerService>();

        string? userName = await GetUserNameInternalAsync(scope.ServiceProvider);

        // 1. A0@CT4C$0D 8D$5C'LC00Dò >C @C 1CäBC¤0 Soft Delete (AÄ0C4:Câ5 D44C ;CT=C,,5)
        // A00DT>CD8CÄ 2D 5 D CD"=CäAD$8 D 8C0BCT@DD5C"ACä< ISoftDeletable, D2 :CäBCä@D'E D BC BD4A "B44C ;C
        IEnumerable<EntityEntry<ISoftDeletable>> entries =
            eventData.Context.ChangeTracker.Entries<ISoftDeletable>().
                .Where(e => e.State == EntityState.Deleted);

        foreach (EntityEntry<ISoftDeletable> entry in entries)
        {
            // A0>CD<CT=Dô5CÄ DC,,7C,,GCTAC¤>CR CCD0C'5C08CR =C >C =Cä2C'5C08CR 4C =C0KDR 2 A 0
            entry.State = EntityState.Modified;
            entry.Entity.DeletedAt = DateTime.UtcNow;
            entry.Entity.DeletedBy = userName;
        }

        // A0@C,,=D44C,,BCT;DÄ=Câ 7C AD$0C$;Dô5CÄ Tb 6÷&R ?CT@CTADt8D$0D$L CD5D 5C$> C,,7CÄ5C05C08C' ?CäAC'5 C0>
        eventData.Context.ChangeTracker.DetectChanges();

        // 2. At0DT2C B C,,7CÄ5C05C08C' ,,ACäED 0C00CTB C00D$8C$=D'5 D$8C0K CD0C0=D'E object? CD;Dò BCTAD$>C".
        List<ChangeEntryModel> captured = CaptureChanges(eventData.Context, userName);

        // A0@C,,2Dô7D'2C 5CÄ :Cä;C'5C¤FC,,N C,,7CÄ5C05C08C' : C¤>C0:D 5D$=Cä<D2 MC¤7CT<C0;D0@D2 :Cä=D$5C¤AD$0 C
        _capturedChanges[eventData.Context.Id.InstanceId] = captured;

        // 3. A$0C'8CD0Dd8Dò „&Vf÷&U6 fr' GCT@CT7 Cd8C$>C' G&-vvW%6W'f-6]
        // At0C0CD :C 5D" 4Cä<CT=C0KCR ?D 0C$8C'0 C, 2C ;C,,4C BCä@D² ?CT@CT4 D$5CÄÄ :C : CD0C0=D'5 C0>C00CDCD
        foreach (ChangeEntryModel item in captured)
            await triggerService.ValidateAsync(item.Entity, item.State, item.Changes,
            eventData.Context);

        return result;
    }
}
/// <summary>

```

```

/// A AC,,=DT@Cä=CÖKC' ?CT@CTEC$0D$GC,,:, C$KCô>C'=Dô5CÄKC' Aä!A' D4ACô5D,,=Cä3Cä ACäED 0C05C08Dò 8Ct<CT=C
/// AäBC$5Dt0CTB Ct0 D42CT4Cä<C'5C08CR ?Cä4C08D GC,,:Cä2 C, BD 8C43CT@Cä2 Cö>D B-Cä1D 0C >D$:C, ,,DC 7C 6
/// </summary>D
public override async ValueTask<int> SavedChangesAsync(D
    SaveChangesCompletedEventData eventData, int result, CancellationToken ct = default)D
{D
    // A,,7C$;CT:C 5CÄ 7C EC$0Dt5C0=D'5 C,,7CÄ5C05C08Dò 8 D @C 7D2 0D$>CÄ0D =Cä CCD0C'0CT< C,,E C,,7 D ;Cä2C
    if (eventData.Context != null &&
        _capturedChanges.TryRemove(eventData.Context.ContextId.InstanceId, out List<ChangeEntryModel>?
entries))D
    {D
        // B >Ct4C 5CÄ ;Cä:C ;DÄ=D'9 C40D 0C0BC,,@Cä2C =C0> Cd8C$>C' 66÷ R 4C'0 DD0CtK Saved (C$KCô>C'=Dô5
        using IServiceScope scope = scopeFactory.CreateScope();D
        var triggerService =
scope.ServiceProvider.GetRequiredService<IDatabaseTriggerService>();D
D
        // A AC,,=DT@Cä=C0> D42CT4Cä<C'0CT< D 8D BCT<D2 >C 8Ct<CT=CT=C,,ODR ,,=C ?D 8CÄ5D Ä 4C'0 Cä1C0>C$;C
        foreach (ChangeEntryModel item in entries)D
            await triggerService.NotifyAsync(item.Entity, item.State, item.Changes,
item.ChangedBy, item.ChangedAt);D
        }D
        return result;D
    }D
}D
D
/// <summary>D
/// Aô5D 5DT2C BDt8C$ AC >Dò ?D 8 D >DT@C =CT=C,,8 C,,7CÄ5C05C08C'â C @C =D$8D CCTB CäGC,,AD$:D2 :DôHC 7C
/// C0@CT4CäBC$@C IC 0 D4BCTGC=8 C00CÄ0D$8 C0@C, ?C 4CT=C,,8 D$@C =Ct0C=FC,,9.D
/// </summary>D
public override Task SaveChangesFailedAsync(DbContextErrorEventData eventData,
CancellationToken ct = default)D
{D
    if (eventData.Context != null)D
        _capturedChanges.TryRemove(eventData.Context.ContextId.InstanceId, out _);D
D
    return base.SaveChangesFailedAsync(eventData, ct);D
}D
D
/// <summary>D
/// A$=D4BD 5C0=C,,9 CÄ5D$>CB 0C00C'8Ct0 D$@CT:CT@C 8Ct<CT=CT=C,,9 EF Core. D
/// A$KDt8D ;Dô5D" 4CT;DÄBD2 ,,AD$0D KCR0=Cä2D'5 Ct=C GCT=C,,0 C0>C'5C'' 8 DD>D <C,,@D45D" <Cä4CT;C, 8Ct<CT=
/// </summary>D
/// <param name="context">B$5C=CD"8C' :Cä=D$5C=AD" 1C 7D² 4C =C0KDRäÄ÷ & Óí
/// <param name="userName">A,,<Dò ?Cä;DÄ7Cä2C BCT;DòÄ ACä2CT@D,,8C$HCT3Cä >C05D 0Dd8DäâÄ÷ & Óí
/// <returns>B ?C,,ACä: D BD CC=BD4@C,,@Cä2C =C0KDR <Cä4CT;CT9 C,,7CÄ5C05C08C' Ç6VR 7&Vc0$6† ævTVÇG'"ÖöFVÄ"ö
private List<ChangeEntryModel> CaptureChanges(DbContext context, string? userName)D
{D
    // BD8C'LD$@D45CÄ BCä;DÄ:Cä AD4IC0>D BC, 2 D >D BCä0C08DòE AD>C 0C$;CT=C,,0, A,,7CÄ5C05C08Dò 8C'8 B44C
    List<EntityEntry> entries = context.ChangeTracker.Entries()D
        .Where(e => e.State is EntityState.Added or EntityState.Modified or EntityState.Deleted)D
        .Where(e => !e.Entity.GetType().Name.Contains("AuditLog"))D
        .ToList();D
D
    var result = new List<ChangeEntryModel>();D
D
    foreach (EntityEntry e in entries)D
    {D
        EntityStateChangeEnum state = MapState(e.State);D
D
        // B ?CTFC,,DC,,GC0KC' <C ?C08C03 D >D BCä0C08Dò 4C'0 "CÄ0C4:Cä CCD0C'5C0=D'E" Ct0C08D 5C•
        if (e.Entity is ISoftDeletable soft)D
        {D
            PropertyEntry prop = e.Property(nameof(ISoftDeletable.DeletedAt));D
            if (prop.IsModified && soft.DeletedAt != null) state =
EntityStateChangeEnum.SoftDeleted;D
        }D
D
        List<PropertyChangeInfo> changes = new();D
D
        // B FCT=C @C,,9 1: B CD"=CäAD$L CD>C 0C$;CT=C ä D 5 D$5C=CD"8CR 7C00Dt5C08Dò AC$>C"AD$2 D GC,,BC
        if (state == EntityStateChangeEnum.Added)D
        {D
            changes = e.PropertiesD
                .Select(p => new PropertyChangeInfoD
                {D
                    PropertyName = p.Metadata.Name,D
                    OriginalValue = null,D

```

```

        CurrentValue = p.CurrentValue
    }).ToList();
}
// B FCT=C @C,,9 2: B CD"=CäAD$ L C,,7CÄ5C05C00 C,,;C, <D03C> D44C ;CT=C â D´GC,,AC´OCT< D 0Ct=C,,FD2
else if (state is EntityStateChangeEnum.Modified or EntityStateChangeEnum.SoftDeleted)
{
    PropertyValues originalValues = e.OriginalValues;

    foreach (PropertyEntry p in e.Properties.Where(p => p.IsModified))
    {
        string propertyName = p.Metadata.Name;
        object? originalValue = originalValues[propertyName];
        object? currentValue = p.CurrentValue;

        // BD8C AC,,@D45CÄ 8Ct<CT=CT=C,,5 D$>C´LC> CTAC´8 Ct=C GCT=C,,0 CD5C"AD$2C,,BCT;DÄ=Câ @C 7C´
        if (!Equals(originalValue, currentValue))
        {
            changes.Add(new PropertyChangeInfo
            {
                PropertyName = propertyName,
                OriginalValue = originalValue,
                CurrentValue = currentValue
            });
        }
    }

    // AD0Cd5 CTAC´8 C>C´;CT:Dd8D0 BCäGCTGC0KDR 8Ct<CT=CT=C,,9 C0>C´5C´ ,,6† ævW2´ ?D4AD$0
    // C,,7-Ct0 D 0C0BC 9CÄ01C,,=CD8C03C &Æ |÷"Ä <D² B B A$ Aâ 4Cä1C 2C´OCT< D CD"=CäAD$ L C" AC08D
    // CTAC´8 DD0C @C,,:C 7C DC,,:D 8D >C$0C´0 D >D BCäOC08CR 0öF-f-VB -D$> C40D 0C0BC,,@D45D" R 4C
    if (state == EntityStateChangeEnum.Modified && changes.Count == 0)
    {
        // B >Ct4C 5CÄ DC,,:D$8C$=D4N Ct0C08D L C,,7CÄ5C05C08D0 4C´O CD>CÄ5C0=D´E D$@C,,3C45D >C"Ä
        // DtBCä1D² =CR ;Cä<C BDÄ 2C ;C,,4C FC,,N, C0> D >DT@C =C,,BDÄ 8CÄ?D4;DÄA CD;D0 T•
        changes.Add(new PropertyChangeInfo
        {
            PropertyName = "Id",
            OriginalValue = e.Property("Id").OriginalValue,
            CurrentValue = e.Property("Id").CurrentValue
        });
    }

    result.Add(new ChangeEntryModel
    {
        Entity = e.Entity,
        State = state,
        ChangedBy = userName,
        ChangedAt = DateTime.UtcNow,
        Changes = changes
    });
}

return result;
}

/// <summary>
/// AÄ0C0?C,,=C2 2C0CD$@CT=C08DR ACäAD$>D0=C,,9 EntityFramework State C$> C$=D4BD 5C0=C,,9 CD>CÄ5C0=D´9 C05D
/// </summary>
private EntityStateChangeEnum MapState(EntityState state) => state switch
{
    EntityState.Added => EntityStateChangeEnum.Added,
    EntityState.Deleted => EntityStateChangeEnum.Deleted,
    _ => EntityStateChangeEnum.Modified
};

/// <summary>
/// A 5Ct>C00D =Cä5 C0>C´CDt5C08CR 8CÄ5C08 D$5C=C'D"5C4> C0>C´LCt>C$0D$5C´O C,,7 C>C0BCT:D BC 0C$BCä@C,,7C
/// </summary>
/// <param name="provider">A´>C=0C´LC0KC´ ?D >C$0C"4CT@ D ;D46C ,,D´ 6öçF -æw"´äÄ÷ & Ó1
private async Task<string?> GetUserNameInternalAsync(IServiceProvider provider)
{
    try
    {
        var userProvider = provider.GetService<IUserProvider>();
        if (userProvider != null)
            return await userProvider.GetCurrentUserNameAsync();
    }
}

```

```

        }
        catch
        {
            // ignored
        }
    }
    return "System";
}
}
</file>

<file path="Data/Interceptors/IUserProvider.cs">
namespace Promatis.Net.Data;
{
    public interface IUserProvider
    {
        // AÄ5D$>CB <Cä6CTB C KD$L C AC,,=DT@Cä=CÔKCÂÂ 5D ;C, ?Cä;D4GCT=C,,5 C,,<CT=C, BD 5C CCTB Ct0Cô@CäAC
        Task<string?> GetCurrentUserAsync();
    }
}
</file>

<file path="Data/Promatis.Net.Data.csproj">
<Project Sdk="Microsoft.NET.Sdk">
{
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
    </PropertyGroup>
{
    <ItemGroup>
        <PackageReference Include="Microsoft.EntityFrameworkCore" Version="10.0.8" />
        <PackageReference Include="Microsoft.EntityFrameworkCore.Relational" Version="10.0.8" />
        <PackageReference Include="Npgsql.EntityFrameworkCore.PostgreSQL" Version="10.0.1" />
    </ItemGroup>
{
    <ItemGroup>
        <ProjectReference Include="..\Domain\Promatis.Net.Domain.csproj" />
    </ItemGroup>
}
    "Ä-FVÔw&÷W í
    "<AssemblyAttribute Include="System.Runtime.CompilerServices.InternalsVisibleTo">
    "Ä & ÖFW# ä &ö F-2äæWBä Ä-6 F-öäÖöFVÄâFW7G3Äö & ÖFW# ä Ä ÖÖ CÄO D$5D BCä2Cä3Cä ?D >CT:D$0 -->
    "</AssemblyAttribute>
    "Äö-FVÔw&÷W í
}
}
</Project>
</file>

<file path="Data/Triggers/Castom/AuditTrigger.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using System.Text.Json;
{
    namespace Promatis.Net.Data;
{
    public class AuditTrigger(
        IServiceScopeFactory scopeFactory,
        JsonSerializerOptions jsonOptions)
        : IAfterSaveTrigger<DomainObject>
    {
        public async Task HandleAsync(EntityChangedEventArgs<DomainObject> args)
        {
            // AäGC,,IC 5CÂ @C =D$0C"<-D$8Cò >D" 2Cä7CÄ>Cd=Cä9 CD8CÔ0CÄ8Dt5D :Cä9 Cö@Cä:D 8-Cä1Cä;CäGC=8 EF Core (
            Type entityType = args.Entity.GetType();
            if (entityType.Namespace == "Castle.Proxies" && entityType.BaseType != null)
            {
                entityType = entityType.BaseType;
            }
        }
    }
}
    // 1. BD8C´LD$@: C´>C48D CCT< D$>C´LC= D$5 D CD"=CäAD$8, C= >D$>D KCR ?Cä<CTGCT=D² " VF-M
    if (entityType.GetInterfaces().All(i => i.Name != nameof(IAudit)))
        return;
}
}

```

```

    }
    // 2. BD>D <C,,@D45CÂ 4C =CÔKCR 4C´O JSOND
    object changesToSerialize = args.State switch
    {
        EntityStateChangeEnum.Added => args.Changes.Select(c => new
        {
            c.PropertyName,
            NewValue = c.CurrentValue
        }),
        EntityStateChangeEnum.Modified or EntityStateChangeEnum.SoftDeleted =>
args.Changes.Select(c => new
        {
            c.PropertyName,
            OldValue = c.OriginalValue,
            NewValue = c.CurrentValue
        }),
        EntityStateChangeEnum.Deleted => new { Message = "Aâ1D¤5C¤B D44C ;CT= C¤>C´=CâAD$LDâ" ÔÍ
        _ => args.Changes
    };

    // 3. B >Ct4C 5CÂ 7C ?C,,ADÂ ;Câ3C
    var auditLog = new AuditLog
    {
        Id = Guid.NewGuid(),
        EntityName = entityType.Name, // A,,!Aô A A´ Aô : Aô8D,,5CÂ GC,,AD$>CR 8CÃO C¤;C AD 0 Câ@C4AD$@D4:D
        EntityId = args.Entity.Id,
        Action = args.State.ToString(),
        ChangedAt = args.ChangedAt,
        ChangedBy = args.ChangedBy,
        ChangesJson = JsonSerializer.Serialize(changesToSerialize, jsonOptions)
    };

    // 4. A,,!Aô A A´ Aô : A 5Ct>Cô0D =Câ5 D >DT@C =CT=C,,5 C" AB GCT@CT7 C$KCD5C´5CÔ=D´9, C40D 0CÔBC,,@Câ
    using IServiceScope scope = scopeFactory.CreateScope();

    // A,,ICT< DD0C @C,,:D2 1C 7Câ2Câ3Câ :Câ=D$5C¤AD$0 Dt5D 5Cr ?D >C$0C"4CT@ C´>C¤0C´LCÔ>C4> Scope
    var factory = scope.ServiceProvider.GetService<IDbContextFactory<ApplicationDbContext>>();

    if (factory != null)
    {
        await using ApplicationDbContext context = await factory.CreateDbContextAsync();
        context.Set<AuditLog>().Add(auditLog);
        await context.SaveChangesAsync();
    }
    else
    {
        Console.WriteLine("[AuditTrigger][Error] Aô5 D44C ;CâADÂ =C 9D$8 IDbContextFactory<ApplicationDbC
    }
}
</file>

<file path="Data/Triggers/Global/FluentValidationTrigger.cs">
using FluentValidation;
using FluentValidation.Results;
using Microsoft.Extensions.DependencyInjection;
using Promatis.Net.Domain.Interface;
namespace Promatis.Net.Data;
public class FluentValidationTrigger(IServiceScopeFactory scopeFactory) // A,,!Aô A A´ Aô : A,,=Cd5C¤BC,,@D45CÂ
    : IBeforeSaveTrigger<IDomainObjectHasKey<Guid>>
{
    public async Task HandleAsync(EntityCancelEventArgs<IDomainObjectHasKey<Guid>> args)
    {
        Type entityType = args.Entity.GetType();
        Type validatorType = typeof(IValidator<>).MakeGenericType(entityType);

        // A,,!Aô A A´ Aô : B >Ct4C 5CÂ 8Ct>C´8D >C$0CÔ=D4N, C 5Ct>Cô0D =D4N Câ1C´0D BDÂ 2C,,4C,,<CâAD$8 (Scope
        // BÔBCâ 3C @C =D$8D CCTB, DtBCâ •6W'f-6U &÷f-FW" 1D44CTB "Cd8C$KCÂ" 8 D4ACô5D,,=Câ @C 7D 5D,,8D" =C H
        using IServiceScope scope = scopeFactory.CreateScope();

```

```

// At0C0@C HC,,2C 5CÂ 2C ;C,,4C BCä@ C,,7 D 2CT6CT3CâÂ 3C @C =D$8D >C$0C0=Câ 6C,,2Cä3Câ 66÷ R0?D >C$0C"4C
if (scope.ServiceProvider.GetService(validatorType) is IValidator validator)ð
{ð
    var context = new ValidationContext<object>(args.Entity);ð
    ValidationResult result = await validator.ValidateAsync(context);ð
ð
    if (!result.IsValid)ð
    {ð
        args.Cancel = true;ð
        // BD>D <C,,@D45CÂ :D 0D 8C$>CRÂ GC,,AD$>CR ACä>C ICT=C,,5 Cä1 CäHC,,1C=0DR 4C´O UIð
        args.ErrorMessage = string.Join(" | ", result.Errors.Select(e => e.ErrorMessage));ð
    }ð
}ð
}
</file>

<file path="Data/Triggers/Global/ReferenceTreeParentTrigger.cs">
using Microsoft.EntityFrameworkCore;ð
using Microsoft.EntityFrameworkCore.Metadata;ð
using Promatis.Net.Domain.Interface;ð
using System.Reflection;ð
ð
namespace Promatis.Net.Data;ð
ð
/// <summary>ð
/// B4=C,,2CT@D 0C´LC0KC´ 8C0DD 0D BD CC=BD4@C0KC´ BD 8C43CT@ CD;Dò ?D >C$5D :C, FCT;CäAD$=CäAD$8 C,,5D 0D EC,,G
/// C, AC=2Cä7C0>C´ 7C IC,,BD² >D" FC,,:C´8Dt5D :C,,E Ct0C$8D 8CÄ>D BCT9 C05D 5CB ACäED 0C05C08CT< C" !B4 AB1
/// </summary>ð
public class ReferenceTreeParentTrigger : IBeforeSaveTrigger<IDomainObjectHasKey<Guid>>ð
{ð
    private static readonly MethodInfo DbContextSetMethod = typeof(DbContext)ð
        .GetMethods(BindingFlags.Public | BindingFlags.Instance)ð
        .First(m => m.Name == nameof(DbContext.Set) && m.IsGenericMethod &&
m.GetParameters().Length == 0);ð
ð
    private static readonly MethodInfo AsNoTrackingMethod =
typeof(EntityFrameworkQueryableExtensions)ð
        .GetMethods(BindingFlags.Public | BindingFlags.Static)ð
        .First(m => m.Name == nameof(EntityFrameworkQueryableExtensions.AsNoTracking) &&
m.IsGenericMethod);ð
ð
    private static readonly MethodInfo IgnoreQueryFiltersMethod =
typeof(EntityFrameworkQueryableExtensions)ð
        .GetMethods(BindingFlags.Public | BindingFlags.Static)ð
        .First(m => m.Name == nameof(EntityFrameworkQueryableExtensions.IgnoreQueryFilters) &&
m.IsGenericMethod);ð
ð
    public async Task HandleAsync(EntityCancelEventArgs<IDomainObjectHasKey<Guid>> args)ð
    {ð
        // A0 Aä!B$ AR AT(AT A,, : Bt5D 5Cr @CTDC´5C=AC,,N C0@Cä2CT@D05CÄÂ 5D BDÂ ;C, C Cä1D=5C=BC AC$>C"AD$2
        // ATAC´8 D 2Cä9D BC$0 C05D" r MD$>D" >C JCT:D" =CR 4CT@CT2CâÂ <D² ?D >D BCâ 2D´ECä4C,,<!ð
        PropertyInfo? parentIdProp = args.Entity.GetType().GetProperty("ParentId");ð
        if (parentIdProp == null) return;ð
ð
        // Bt8D$0CT< Ct=C GCT=C,,0 D 2Cä9D BC" 4C,,=C <C,,GCTAC=8ð
        Guid entityId = args.Entity.Id;ð
        Guid? parentId = (Guid?)parentIdProp.GetValue(args.Entity);ð
ð
        PropertyInfo? deletedAtProp = args.Entity.GetType().GetProperty("DeletedAt");ð
ð
        // 1. At0D"8D$0 CäB C0@D0<Cä3Câ AC <CäFC,,BC,,@Cä2C =C,,0ð
        if (parentId.HasValue && parentId == entityId)ð
        {ð
            args.Cancel = true;ð
            args.ErrorMessage = "Aä1D=5C=B C05 CÄ>Cd5D" 1D´BDÂ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5.";ð
            return;ð
        }ð
ð
        if (parentId.HasValue && parentId.Value != Guid.Empty)ð
        {ð
            // A,,ICT< D >CD8D$5C´O C" 6† ævUG& 6¶W" AD 5CD8 C´NC KDR >C JCT:D$>C"Â C C=ðD$>D KDR ACä2C00CD0CT
            object? localParent = args.Context.ChangeTracker.Entries()ð
                .FirstOrDefault(e => ((IDomainObjectHasKey<Guid>)e.Entity).Id ==
parentId.Value)?.Entity;ð
ð

```

```

        bool exists;
        bool isDeleted;

        if (localParent != null)
        {
            exists = true;
            var localDeletedAt = (DateTime?)deletedAtProp?.GetValue(localParent);
            isDeleted = localDeletedAt != null;
        }
        else
        {
            // A$0D,,0 Că@C,,3C,,=C ;DÄ=C 0 D 5DD;CT:D 8Dò 4C´O C$KDt8D ;CT=C,,0 Că>D =Dò 8 Query C" Tb 6÷&]
            IEntityType? entityTypeInModel =
args.Context.Model.FindEntityType(args.Entity.GetType());
            Type rootClrType = entityTypeInModel?.GetRootType()?.ClrType ??
args.Entity.GetType();
            object? rawSet =
DbContextSetMethod.MakeGenericMethod(rootClrType).Invoke(args.Context, null);
            object? noTrackingQuery =
AsNoTrackingMethod.MakeGenericMethod(rootClrType).Invoke(null, [rawSet]);
            object? ignoreFiltersQuery =
IgnoreQueryFiltersMethod.MakeGenericMethod(rootClrType).Invoke(null, [noTrackingQuery]);
            IQueryable<object> query = ((IQueryable)ignoreFiltersQuery!).Cast<object>();

            // A,,ICT< Că1D=5C=B Cò> CD8C00CÄ8Dt5D :Că<D2 AC$>C"AD$2D2 $-B" !B4 AM
            object? dbParent = await query.FirstOrDefaultAsync(x => EF.Property<Guid>(x, "Id")
== parentId.Value);

            exists = dbParent != null;
            var dbDeletedAt = dbParent != null ? (DateTime?)deletedAtProp?.GetValue(dbParent) :
null;

            isDeleted = dbDeletedAt != null;
        }

        // 2. A0@Că2CT@C=0 DD8Ct8Dt5D :Că3Că AD4ICTAD$2Că2C =C,,0 D >CD8D$5C´LD :Că3Că Cct;C
        if (!exists)
        {
            args.Cancel = true;
            args.ErrorMessage = $"B4:C 7C =C0KC' @Că4C,,BCT;DÄAC=8C' >C JCT:D" ,,"C¢ . &VçD-G0' =CR =C 9CD
            return;
        }

        // 3. A0@Că2CT@C=0 D BC BD4AC <D03C=C4> D44C ;CT=C,,0D
        if (isDeleted)
        {
            args.Cancel = true;
            args.ErrorMessage = "A05C´Lct0 C00Ct=C GC,,BDÂ @Că4C,,BCT;CT< D44C ;CT=C0KC' >C JCT:D"â#½
            return;
        }

        // 4. A4 B4 Aä A / At B" B$ Aä" Bd A= Aä (A -> B -> C -> A)
        Guid? currentCheckId = parentId;
        var visitedIds = new HashSet<Guid>();

        while (currentCheckId.HasValue && currentCheckId.Value != Guid.Empty)
        {
            if (currentCheckId == entityId)
            {
                args.Cancel = true;
                args.ErrorMessage = "Bd8C=;C,,GCTAC=0Dò 7C 2C,,AC,,<CăAD$L: C05C´Lct0 C05D 5Că5D BC,,BDÂ @Că4
                return;
            }

            if (!visitedIds.Add(currentCheckId.Value))
                break;

            // A,,ICT< D0;CT<CT=D" FCT?CăGC=8 D =C GC ;C 2 ChangeTracker
            object? nextLocalElement = args.Context.ChangeTracker.Entries()
                .FirstOrDefault(e => ((IDomainObjectHasKey<Guid>)e.Entity).Id ==
currentCheckId.Value)?.Entity;

            if (nextLocalElement != null)
            {
                currentCheckId = (Guid?)parentIdProp.GetValue(nextLocalElement);
            }
        }
    }
}

```

```

        }
        else
        {
            Guid? id = currentCheckId;

            IEntityType? loopEntityTypeInModel =
args.Context.Model.FindEntityType(args.Entity.GetType());
            Type loopRootClrType = loopEntityTypeInModel?.ClrType ??
args.Entity.GetType();
            object? loopRawSet =
DbContextSetMethod.MakeGenericMethod(loopRootClrType).Invoke(args.Context, null);
            object? loopNoTrackingQuery =
AsNoTrackingMethod.MakeGenericMethod(loopRootClrType).Invoke(null, [loopRawSet]);
            object? loopIgnoreFiltersQuery =
IgnoreQueryFiltersMethod.MakeGenericMethod(loopRootClrType).Invoke(null, [loopNoTrackingQuery]);
            IQueryable<object> loopQuery =
((IQueryable)loopIgnoreFiltersQuery!).Cast<object>();
            object? parentNodeObj = await loopQuery.FirstOrDefaultAsync(x =>
EF.Property<Guid>(x, "Id") == id.Value);
            currentCheckId = parentNodeObj != null ?
(Guid?)parentIdProp.GetValue(parentNodeObj) : null;
        }
    }
}
</file>

<file path="Data/TriggerService/DatabaseTriggerService.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using System.Collections.Concurrent;
namespace Promatis.Net.Data;
/// <summary>
/// B 5C ;C,,7C FC,,0 Dd5C0BD 0C`LC0>C4> CD8D ?CTBDt5D 0 D$@C,,3C45D >C"â #C0@C 2C`OCTB D 5C48D BD 0Dd8CT9 CD>CÄ
/// C, :Cä>D 4C,,=C,,@D45D" DC 7D² 2C ;C,,4C FC,,8 (CD> D >DT@C =CT=C,,0) C, CC$5CD>CÄ;CT=C,,0 (C0>D ;CR ACäED 0C05
/// </summary>
/// <param name="serviceProvider">A :D$CC ;DÄ=D´9 C¤>C0BCT:D B Ct0C$8D 8CÄ>D BCT9 (DI Container) D$5C¤CD"5C4>
public class DatabaseTriggerService(IServiceProvider serviceProvider) : IDatabaseTriggerService
{
    /// <summary>
    /// A4;Cä1C ;DÄ=Cä5 D >C KD$8CRÄ CC$5CD>CÄ;D0ND"5CR AC,,AD$5CÄC Cä1 D4AC05D,,=Cä< C¤>CÄ<C,,BCR AD4IC0>D BC,
    /// A,,AC0>C`LCtCCTBD 0 CD;D0 @CT0C¤BC,,2C0>C4> Cä1C0>C$;CT=C,,0 C,,=D$5D DCT9D 0 (MudBlazor) C,,;C, AC¤2Cä7C0
    /// </summary>
    public static event Action<EntityStateChangeEnum, object>? OnEntityCommitted;

    /// <summary>
    /// B 5CTAD$@ C0>CD?C,,Adt8C¤>C"Â 2D´?Cä;C00DäIC,,ED 0 AD D >DT@C =CT=C,,0 C,,7CÄ5C05C08C´ ,,SC 7C C ;C,,4C
    /// AD5C´5C40D" ?D 8C08CÄ0CTB C @C4CCÄ5C0BD² ACä1D´BC,,0 C, 0C¤BD40C`LC0KC´ 66÷ VB 6W'f-6U &÷f-FW"i
    /// </summary>
    private static readonly Dictionary<Type, List<Func<EntityCancelArgsBase, IServiceProvider,
Task>>> BeforeSubscribers = new();

    /// <summary>
    /// B 5CTAD$@ C0>CD?C,,Adt8C¤>C"Â 2D´?Cä;C00DäIC,,ED 0 A0 B AR CD ?CTHC0>C4> D >DT@C =CT=C,,0 C,,7CÄ5C05C08C
    /// </summary>
    private static readonly Dictionary<Type, List<Func<EntityChangedArgsBase, IServiceProvider,
Task>>> AfterSubscribers = new();

    /// <summary>
    /// A0>D$>C¤>C 5Ct>C00D =D´9 C¤MD, 8CT@C @DT8C, BC,,?Cä2. A,,AC¤;DäGC 5D" ?Cä2D$>D =Cä5 C,,AC0>C`Lct>C$0C08C
    /// C0@C, >C0@CT4CT;CT=C,,8 C 0Ct>C$KDR :C´0D ACä2 C, 8C0BCT@DD5C"ACä2 Cä1D 0C 0D$KC$0Ct<D´E D CD"=CäAD$5C
    /// </summary>
    private static readonly ConcurrentDictionary<Type, List<Type>> HierarchyCache = new();

    /// <summary>
    /// B 5C48D BD 8D CCTB CD>CÄ5C0=D´9 D$@C,,3C45D 4C´O C¤>C0:D 5D$=Cä3Cä BC,,?C AD4IC0>D BC,i
    /// ASKctKC$0CtBD 0 C 2D$>CÄ0D$8Dt5D :C, ?D 8 D :C =C,,@Cä2C =C,,8 D 1Cä@Cä: C$> C$@CT<D0 8C08Dd8C ;C,,7C FC
    /// </summary>
    /// <typeparam name="TEntity">B$8C0 4Cä<CT=C0>C´ AD4IC0>D BC,äÄ÷G- W & öi

```



```

/// <typeparam name="TTrigger">B$8Cò :C`0D AC ÔBD 8C43CT@C Â @CT0C`8CtCDäICT3Câ ;Cä3C,,D2 >C @C 1CäBC¤8.<
public void Register<TEntity, TTrigger>()
    where TEntity : class
    where TTrigger : class
{
    Type entityType = typeof(TEntity);
    Type triggerType = typeof(TTrigger);

    // Aô@Cä2CT@Dô5CÂÂ :C :C,,5 C,,=D$5D DCT9D K Cd8Ct=CT=Cô>C4> Dd8C¤;C @CT0C`8CtCCTB CD0CÔ=D`9 D$@C,,3C45
    bool isBefore = typeof(IBeforeSaveTrigger<TEntity>).IsAssignableFrom(triggerType);
    bool isAfter = typeof(IAfterSaveTrigger<TEntity>).IsAssignableFrom(triggerType);

    if (isBefore)
    {
        // BD>D <C,,@D45CÂ >D$;Cä6CT=CôCDâ ;Dô<C 4D2â Cä=D$5C`=CT@ 'sp' Cô5D 5CD0CTBD 0 C" <Cä<CT=D" 2D`7
        // DtBCâ ?D 5CD>D$2D 0D`0CTB D4BCTGC¤C CD>C`3Cä6C,,2D4IC,,E Cä1D¤5C¤BCä2 (Captive Dependency).
        AddSubscriber(BeforeSubscribers, entityType, async (argsBase, sp) =>
        {
            // B 0Ct@CTHC 5CÂ MC¤7CT<Cô;Dô@ D$@C,,3C45D 0 C,,7 C :D$CC ;DÄ=Cä3Câ 66÷ R BCT:D4ICT3Câ 7C ?D >
            if (sp.GetRequiredService<TTrigger>() is IBeforeSaveTrigger<TEntity> handler)
            {
                // Aä1Cä@C GC,,2C 5CÂ 1C 7Cä2D`5 C @C4CCÄ5CÔBD² 2 D BD >C4> D$8Cô8Ct8D >C$0CÔ=D`9 C¤>CÔBCT
                var typedArgs = new EntityCancelEventArgs<TEntity>(
                    (TEntity)argsBase.Entity,
                    argsBase.State,
                    argsBase.Changes,
                    argsBase.Context);

                await handler.HandleAsync(typedArgs);

                // A$>Ct2D 0D`0CT< D 5CtCC`LD$0D$K C$KCô>C`=CT=C,,0 D$@C,,3C45D 0 Cä1D 0D$=Cä 2 C 0Ct>C$KC`
                argsBase.Cancel = typedArgs.Cancel;
                argsBase.ErrorMessage = typedArgs.ErrorMessage;
                argsBase.Handled = typedArgs.Handled;
            }
        });
    }

    if (isAfter)
    {
        AddSubscriber(AfterSubscribers, entityType, async (argsBase, sp) =>
        {
            if (sp.GetRequiredService<TTrigger>() is IAfterSaveTrigger<TEntity> handler)
            {
                var typedArgs = new EntityChangedEventArgs<TEntity>(
                    (TEntity)argsBase.Entity,
                    argsBase.State,
                    argsBase.Changes,
                    argsBase.ChangedBy,
                    argsBase.ChangedAt);

                await handler.HandleAsync(typedArgs);
                argsBase.Handled = typedArgs.Handled;
            }
        });
    }

    // ATAC`8 C¤;C AD =CR @CT0C`8CtCCTB CÔ8 Cä4C,,= C¤>CÔBD 0C¤B — DôBCâ :D 8D$8Dt5D :C 0 CäHC,,1C¤0 C¤>CÔ
    if (!isBefore && !isAfter)
    {
        throw new InvalidOperationException(
            $"A¤;C AD ·G&-vvw%G— Räæ ÔWÔ =CR @CT0C`8CtCCTB IBeforeSaveTrigger C,,;C, " gFW%6 FUG&-vvw" 4C
        );
    }
}

/// <summary>
/// A$KCô>C`=Dô5D" AC¤2Cä7CÔCDâ 2C ;C,,4C FC,,N C,,7CÄ5CÔ0CT<Cä9 D CD`=CäAD$8 Cô5D 5CB 5E >D$?D 0C$:Cä9 C"
/// Aä?D 0D,,8C$0CTB C$ADâ FCT?CäGC¤C CÔ0D ;CT4Cä2C =C,,0 D CD`=CäAD$8. Aô@CT@D`2C 5D" >Cô5D 0Dd8Dâ ?D 8 CÔ
/// </summary>
public async Task ValidateAsync(object entity, EntityStateChangeEnum state,
    List<PropertyChangeInfo> changes, DbContext context)
{
    var args = new EntityCancelArgsBase(entity, state, changes, context);

    // Aä1DT>CD8CÂ 8CT@C @DT8Dâ BC,,?Cä2 (D 0CÄ>C` AD4ICô>D BC,Â 1C 7Cä2D`E C¤;C AD >C" 8 C,,=D$5D DCT9D >C
    foreach (Type type in GetTypesHierarchy(entity.GetType()))

```

```

        {
            if (BeforeSubscribers.TryGetValue(type, out List<Func<EntityCancelArgsBase,
IServiceProvider, Task>>? actions))
            {
                foreach (Func<EntityCancelArgsBase, IServiceProvider, Task> action in actions)
                {
                    // A05D 5CD0CT< C'>C0C'LC0KC' 6W'f-6U &÷f-FW"Â A C0D$>D KCÂ 1D'; D >Ct4C = CD0C0=D'9 D0
                    await action(args, serviceProvider);
                }
            }
        }

        // ATAC'8 D$@C,,3C45D 2D'AD$0C$8C² DC'0C2 6 æ6VÂ r =CT<CT4C'5C0=Câ ?D 5D KC$0CT< D$@C =Ct
        if (args.Cancel) throw new OperationCanceledException(args.ErrorMessage);
    }
}

}

}

/// <summary>
/// B 0D AD';C 5D" CC$5CD>CÄ;CT=C,,0 Cä1 D4AC05D,,=Cä< D >DT@C =CT=C,,8 C,,7CÄ5C05C08C' 2 C 0CtC CD0C0=D'E.D
/// A0>CD4CT@Cd8C$0CTB C00D :C 4C0KC' ?CT@CTEC$0D" ,,DC'0C2 † æFÆVB' 8 C0CC ;C,,:D45D" ACä1D'BC,,5 C" 3C'>C
/// </summary>
public async Task NotifyAsync(object entity, EntityStateChangeEnum state,
List<PropertyChangeInfo> changes, string? user, DateTime at)
{
    var args = new EntityChangedEventArgsBase(entity, state, changes, user, at);

    foreach (Type type in GetTypesHierarchy(entity.GetType()))
    {
        if (AfterSubscribers.TryGetValue(type, out List<Func<EntityChangedEventArgsBase,
IServiceProvider, Task>>? actions))
        {
            foreach (Func<EntityChangedEventArgsBase, IServiceProvider, Task> action in actions)
            {
                await action(args, serviceProvider);

                // ATAC'8 Cä4C,,= C,,7 D$@C,,3C45D >C" 2D'AD$0C$8C² † æFÆVB 0 G'VRÂ >C @C 1CäBC00 Dd5C0>Dt:C
                if (args.Handled) return;
            }
        }
    }

    // A0CC ;C,,:C FC,,0 C" AD$0D$8Dt5D :C,,9 D02CT=D" 4C'0 D 5C :D$8C$=Cä3Cä >C =Cä2C'5C08D0 :Cä<C0>C05C0BC
    OnEntityCommitted?.Invoke(state, entity);
}

/// <summary>
/// A$AC0>CÄ>C40D$5C'LC0KC' <CTBCä4 CD;D0 1CT7Cä?C AC0>C4> CD>C 0C$;CT=C,,0 C0>CD?C,,Adt8C0>C" 2 D ;Cä2C @C
/// </summary>
private void AddSubscriber<TArgs>(Dictionary<Type, List<Func<TArgs, IServiceProvider, Task>>>
dict, Type type, Func<TArgs, IServiceProvider, Task> action)
{
    if (!dict.TryGetValue(type, out List<Func<TArgs, IServiceProvider, Task>>? list))
    {
        list = new List<Func<TArgs, IServiceProvider, Task>>();
        dict[type] = list;
    }
    list.Add(action);
}

/// <summary>
/// A,,7C$;CT:C 5D" ?Cä;C0CDä 8CT@C @DT8Dä BC,,?Cä2 CD;D0 CC00Ct0C0=Cä3Cä >C JCT:D$0 (C$:C'NDt0D0 1C 7Cä2D'
/// B 5CtCC'LD$0D$K C0MD,,8D CDäBD 0 CD;D0 >C 5D ?CTGCT=C,,0 C$KD >C0>C' ?D >C,,7C$>CD8D$5C'LC0>D BC, ?Cä4 C
/// </summary>
private IEnumerable<Type> GetTypesHierarchy(Type type) =>
    HierarchyCache.GetOrAdd(type, t => {
        var types = new List<Type>();
        // B 5C0CD AC,,2C0> C0>CD=C,,<C 5CÄAD0 2C$5D E C0> CD@CT2D2 =C AC'5CD>C$0C08D0 :C'0D ACä2, C,,AC0;Dä
        for (Type? c = t; c != null && c != typeof(object); c = c.BaseType) types.Add(c);
        // AD>C 0C$;D05CÄ 2D 5 D 5C ;C,,7Cä2C =C0KCR 8C0BCT@DD5C"AD² 8 D41C,,@C 5CÄ 4D41C'8C00D$K0
        return types.Concat(t.GetInterfaces()).Distinct().ToList();
    });

/// <summary>
/// B @CäGC0KC' AC @CäA D BC BC,,GCTAC08DR @CT3C,,AD$@C FC,,9. D
/// A,,AC0>C'LCtCCTBD 0 C0@CT8CÄCD"5D BC$5C0=Cä 2 Unit-D$5D BC E CD;D0 8Ct>C'0Dd8C, ?D >C4>C0>C" BCTAD$>C"
/// </summary>
internal static void ClearInternalRegistrations()

```

```

        {
            BeforeSubscribers.Clear();
            AfterSubscribers.Clear();
            HierarchyCache.Clear();
        }
    }
</file>

```

```

<file path="Data/TriggerService/EntityStateChangeEnum.cs">
namespace Promatis.Net.Data;
{
    public enum EntityStateChangeEnum
    {
        Added,
        Modified,
        Deleted,
        SoftDeleted
    }
}
</file>

```

```

<file path="Data/TriggerService/IDatabaseTriggerService.cs">
using Microsoft.EntityFrameworkCore;
{
    namespace Promatis.Net.Data;
    {
        public interface IDatabaseTriggerService
        {
            // AA5D$>CB 4C´O C00D BD >C":C, AC$0Ct5C' $!D4IC0>D BDÂ0"D 8C43CT@"
            void Register<TEntity, TTrigger>()
                where TEntity : class
                where TTrigger : class;

            // AA5D$>CDK C$KC0>C´=CT=C,,0 (C$KctKC$0DäBD 0 C,,=D$5D FCT?D$>D >CÂ•
            Task ValidateAsync(object entity, EntityStateChangeEnum state, List<PropertyChangeInfo>
changes, DbContext context);
            Task NotifyAsync(object entity, EntityStateChangeEnum state, List<PropertyChangeInfo> changes,
string? user, DateTime at);
        }
    }
}
</file>

```

```

<file path="Data/TriggerService/PropertyChangeInfo.cs">
namespace Promatis.Net.Data;
{
    public class PropertyChangeInfo
    {
        public string PropertyName { get; set; } = string.Empty;
        public object? OriginalValue { get; set; }
        public object? CurrentValue { get; set; }
    }
}
</file>

```

```

<file path="Data/TriggerService/TriggerEvents/EntityCancelArgsBase.cs">
using Microsoft.EntityFrameworkCore;
{
    namespace Promatis.Net.Data;
    {
        public class EntityCancelArgsBase(
            object entity,
            EntityStateChangeEnum state,
            List<PropertyChangeInfo> changes,
            DbContext context) : EntityEventArgsBase(entity, state, changes)
        {
            public DbContext Context { get; } = context;
            public bool Cancel { get; set; }
            public string? ErrorMessage { get; set; }
        }
    }
}
</file>

```

```

<file path="Data/TriggerService/TriggerEvents/EntityCancelEventArgs.cs">
using Microsoft.EntityFrameworkCore;
{
    namespace Promatis.Net.Data;
    {
        public class EntityCancelEventArgs<T>(
            T entity,
            EntityStateChangeEnum state,

```

```

        List<PropertyChangeInfo> changes,
        DbContext context)
    : EntityCancelArgsBase(entity!, state, changes, context) where T : class
{
    public new T Entity => (T)base.Entity;
}
</file>

<file path="Data/TriggerService/TriggerEvents/EntityChangedEventArgs.cs">
namespace Promatis.Net.Data;

// A 0Ct>C$KC' :C'0D A CD;Dò 2C0CD$@CT=C05C4> C,,ACô>C'Lct>C$0C08Dò 2 D 5D 2C,,AC]
public class EntityChangedEventArgs(
    object entity,
    EntityStateChangeEnum state,
    List<PropertyChangeInfo> changes,
    string? changedBy,
    DateTime changedAt) : EntityEventArgsBase(entity, state, changes) // <-- A00D ;CT4D45CÂ 7CD5D L0
{
    public string? ChangedBy { get; } = changedBy;
    public DateTime ChangedAt { get; } = changedAt;
}
</file>

<file path="Data/TriggerService/TriggerEvents/EntityChangedEventArgs.cs">
namespace Promatis.Net.Data;

// A$0D, BC,,?C,,7C,,@Câ2C =C0KC' :C'0D A CD;Dò BD 8C43CT@Câ2D
public class EntityChangedEventArgs<T>(
    T entity, EntityStateChangeEnum state,
    List<PropertyChangeInfo> changes,
    string? changedBy,
    DateTime changedAt)
    : EntityChangedEventArgs(entity!, state, changes, changedBy, changedAt) where T : class
{
    public new T Entity => (T)base.Entity;
}
</file>

<file path="Data/TriggerService/TriggerEvents/EntityEventArgsBase.cs">
namespace Promatis.Net.Data;

public abstract class EntityEventArgsBase(
    object entity,
    EntityStateChangeEnum state,
    List<PropertyChangeInfo> changes) : EventArgs
{
    public object Entity { get; } = entity;
    public EntityStateChangeEnum State { get; } = state;
    public List<PropertyChangeInfo> Changes { get; } = changes;
    public bool Handled { get; set; }
}
</file>

<file path="Data/TriggerService/TriggerInterfaces/IAfterSaveTrigger.cs">
namespace Promatis.Net.Data;

// AD;Dò ?D 0C$8C² $ Aâ!A' " D >DT@C =CT=C,,O (D42CT4Câ<C'5C08DòôHC,,=C •
public interface IAfterSaveTrigger<T> where T : class
{
    Task HandleAsync(EntityChangedEventArgs<T> args);
}
</file>

<file path="Data/TriggerService/TriggerInterfaces/IBeforeSaveTrigger.cs">
namespace Promatis.Net.Data;

// AD;Dò ?D 0C$8C² $ Aâ" ACâED 0C05C08Dò ,,2C ;C,,4C FC,,0)D
public interface IBeforeSaveTrigger<T> where T : class
{
    Task HandleAsync(EntityCancelEventArgs<T> args);
}
</file>

<file path="Data/Validation/GlobalPolymorphicValidator.cs">
using FluentValidation;

```

```

using FluentValidation.Results;
using Microsoft.Extensions.DependencyInjection;
namespace Promatis.Net.Domain;
public class GlobalPolymorphicValidator<T> : AbstractValidator<T> where T : class
{
    private readonly IServiceProvider _serviceProvider;

    public GlobalPolymorphicValidator(IServiceProvider serviceProvider)
    {
        _serviceProvider = serviceProvider ?? throw new
ArgumentNullException(nameof(serviceProvider));
    }

    protected override bool PreValidate(ValidationContext<T> context, ValidationResult result)
    {
        if (context.InstanceToValidate == null) return true;

        // 1. A0>C'CDt0CT< D 5C ;DÄ=D'9 D$8C0 >C JCT:D$0 C" @C =D$0C"<CR „=C ?D 8CÄ5D Ä FW 'FÖVçEVæ-B 8C'8 A
        Type realType = context.InstanceToValidate.GetType();

        // ATAC'8 D$8C0 ACä2C00CD0CTB D B „B.CRâ MD$> C0>C05Dt=D'9 C0;C AD Ä 0 C05 C 0Ct>C$KC' 0C AD$@C :D$=
        // D$> C„AC00D$L CD>Dt5D =C„9 C$0C'8CD0D$>D =CR =D46C0>, DtBCä1D² =CR CC"BC, 2 C 5D :Cä=CTGC0CDâ @CT
        if (realType == typeof(T)) return true;

        // 2. B BD >C„< D$8C0 8C0BCT@DD5C"AC 4C'0 D$>Dt5Dt=Cä3Cä 2C ;C„4C BCä@C „=C ?D 8CÄ5D Ä •f Æ-F F÷#ÄF
        Type specificValidatorType = typeof(IValidator<>).MakeGenericType(realType);

        // 3. At0C0@C HC„2C 5CÄ 8Cr D' B C$0C'8CD0D$>D K CD;D0 MD$>C4> C0>C0:D 5D$=Cä3Cä BC„?C
        // B0BCä 2C 6C0>, D$0C0 :C : CD;D0 >CD=Cä3Cä BC„?C <Cä6CTB C KD$L Ct0D 5C48D BD 8D >C$0C0> C05D :Cä;
        List<IValidator> specificValidators =
_serviceProvider.GetServices(specificValidatorType).Cast<IValidator>().ToList();

        if (specificValidators.Any())
        {
            // B >Ct4C 5CÄ =CR04Cd5C05D 8C0 :Cä=D$5C0AD" 2C ;C„4C FC„8 FluentValidation CD;D0 ?CT@CT4C GC, 4C
            ValidationContext<object>? newContext =
ValidationContext<object>.CreateWithOptions(context.InstanceToValidate, options =>
            {
                // A„!A0 A A' A0 : A0C0>C08Dt=D'9 CÄ5D$>CB fçVVçEf Æ-F F-öâ 4C'0 C0@D0<Cä9 C0@Cä1D >D :C, A
                if (context.Selector != null)
                {
                    options.UseCustomSelector(context.Selector);
                }
            });

            // At0C0CD :C 5CÄ :C 6CDKC' =C 9CD5C0=D'9 D$>Dt5Dt=D'9 C$0C'8CD0D$>D =C AC'5CD=C„:C
            foreach (IValidator validator in specificValidators)
            {
                // BtBCä1D² 8Ct1CT6C BDÄ 7C FC„:C'8C$0C08D0Ä ?D >C$5D OCT<, DtBCä =C 9CD5C0=D'9 C$0C'8CD0D$>D
                if (validator.GetType().IsGenericType &&
validator.GetType().GetGenericTypeDefinition() == typeof(GlobalPolymorphicValidator<>))
                    continue;

                ValidationResult specificResult = validator.Validate(newContext);

                // A05D 5C0>D 8CÄ 2D 5 Cä1C00D CCd5C0=D'5 CäHC„1C08 C" >C IC„9 D 5CtCC'LD$0D" 2C ;C„4C FC„8 D
                foreach (ValidationFailure error in specificResult.Errors)
                {
                    result.Errors.Add(error);
                }
            }
        }

        return true;
    }
}
</file>

<file path="Domain.Interface/Enums/EnumExtensions.cs">
using System.ComponentModel;
using System.Reflection;
namespace Promatis.Net.Domain.Interface;

```

```

public static class EnumExtensions
{
    /// <summary>
    /// A$>Ct2D 0D"0CTB Ct=C GCT=C,,5 C BD 8C CD$0 [Description] CD;Dò ;Dä1Cä3Cä MC'5CÄ5C0BC ?CT@CTGC,,AC'5C08
    /// ATAC'8 C BD 8C CD" >D$AD4BD BC$CCTB, C$>Ct2D 0D"0CTB D BC =CD0D BC0>CR 8CÄ0 .ToString().
    /// </summary>
    public static string GetDescription(this Enum value)
    {
        FieldInfo? field = value.GetType().GetField(value.ToString());
        if (field == null) return value.ToString();

        var attribute = field.GetCustomAttribute<DescriptionAttribute>();
        return attribute != null ? attribute.Description : value.ToString();
    }
}
</file>

<file path="Domain.Interface/Interfaces/IAudit.cs">
namespace Promatis.Net.Domain.Interface;
{
    /// <summary>
    /// B CD'=CäAD$8, D 5C ;C,,7D4ND"8CR MD$>D" 8C0BCT@DD5C"A, C CCDCD" 7C ?C,,AD'2C BDÄADò 2 AuditLog
    /// </summary>
    public interface IAudit
    {
        DateTime CreatedAt { get; init; }
    }
}
</file>

<file path="Domain.Interface/Interfaces/IDomainObject.cs">
namespace Promatis.Net.Domain.Interface;
{
    public interface IDomainObject
    {
        DateTime CreatedAt { get; init; }
    }
}
</file>

<file path="Domain.Interface/Interfaces/IDomainObjectHasKey.cs">
namespace Promatis.Net.Domain.Interface;
{
    public interface IDomainObjectHasKey<TKey> : IDomainObject
    {
        TKey Id { get; set; }
    }
}
</file>

<file path="Domain.Interface/Interfaces/IEntityCloner.cs">
namespace Promatis.Net.Domain.Interface;
{
    /// <summary>
    /// A,,=D$5D DCT9D ?C'0D$DCä@CÄ5C0=Cä3Cä 4C$8Cd:C 8Ct>C'8D >C$0C0=Cä3Cä :C'>C08D >C$0C08Dò 4Cä<CT=C0KDR AD4I
    /// </summary>
    public interface IEntityCloner
    {
        T CloneEntity<T>(T entity) where T : class;
    }
}
</file>

<file path="Domain.Interface/Interfaces/ISoftDeletable.cs">
namespace Promatis.Net.Domain.Interface;
{
    /// <summary>
    /// A,,=D$5D DCT9D 4C'0 CÄ0C4:Cä3Cä CCD0C'5C08Dò >C JCT:D$>C-
    /// </summary>
    public interface ISoftDeletable
    {
        DateTime? DeletedAt { get; set; }
        string? DeletedBy { get; set; }
    }
}
</file>

<file path="Domain.Interface/Interfaces/ITreeNode.cs">
namespace Promatis.Net.Domain.Interface;
{

```

```

Ð
/// <summary>Ð
/// A4;Că1C ;DĂ=D´9 Cô;C BDD>D <CT=CÔKC´ :Că=D$@C :D" 4C´O C$ACTE C,,5D 0D EC,,GCTAC=8DR ,,4D 5C$>C$8CD=D´E) D C
/// </summary>Ð
/// <typeparam name="T">B$8Cò 4Că<CT=CÔ>C´ <Că4CT;C,ăÂ÷G- W & Óí
public interface ITreeNode<T> : IDomainObjectHasKey<Guid> where T : classÐ
{Ð
    /// <summary>Ð
    /// A,,4CT=D$8DD8C=0D$>D @Că4C,,BCT;DĂAC=8C4> D47C´0. B 0C$5Cò çVÆÂ 4C´O C= >D =CT2D´E Dô;CT<CT=D$>C"í
    /// A,,!Aô A A´ Aô : AD>C 0C$;CT= set CD;Dò 2Că7CĂ>Cd=CăAD$8 C,,7CĂ5C05C08Dò AC$0Ct5C´ ,,?CT@CT<CTICT=C,,O C
    /// </summary>Ð
    Guid? ParentId { get; set; }Ð
Ð
    /// <summary>Ð
    /// Aô0C$8C40Dd8Că=Cô>CR AC$>C"AD$2Că AD KC´:C, =C @Că4C,,BCT;DĂAC=8C´ >C JCT:D"í
    /// </summary>Ð
    T? Parent { get; set; }Ð
Ð
    /// <summary>Ð
    /// A= >C´;CT:Dd8Dò 4CăGCT@C08DR MC´5CĂ5C0BCă2 (Cô>CDGC,,=CT=CÔKDR CCT;Că2).Ð
    /// </summary>Ð
    ICollection<T> Children { get; }Ð
}
</file>

<file path="Domain.Interface/Promatis.Net.Domain.Interface.csproj">
<Project Sdk="Microsoft.NET.Sdk">Ð
Ð
    <PropertyGroup>Ð
        <TargetFramework>net10.0</TargetFramework>Ð
        <ImplicitUsings>enable</ImplicitUsings>Ð
        <Nullable>enable</Nullable>Ð
    </PropertyGroup>Ð
Ð
</Project>
</file>

<file path="Domain/AuditLog/AuditLog.cs">
namespace Promatis.Net.Domain;Ð
Ð
/// <summary>Ð
/// A CCD8D" AC,,AD$5CĂKÐ
/// </summary>Ð
public class AuditLog : DomainObjectÐ
{Ð
    /// <summary>Ð
    /// Aô0C,,<CT=Că2C =C,,5 D >C KD$8Dý
    /// </summary>Ð
    public string EntityName { get; init; } = string.Empty;Ð
Ð
    /// <summary>Ð
    /// A,,4CT=D$8DD8C=0D$>D ACă1D´BC,,0Ð
    /// </summary>Ð
    public Guid EntityId { get; init; }Ð
Ð
    /// <summary>Ð
    /// AD5C"AD$2C,,5Ð
    /// </summary>Ð
    public string Action { get; init; } = string.Empty;Ð
Ð
    /// <summary>Ð
    /// AD0D$0 C,,7CĂ5C05C08Dý
    /// </summary>Ð
    public DateTime ChangedAt { get; init; }Ð
Ð
    /// <summary>Ð
    /// A,,7CĂ5C05C0>Ð
    /// </summary>Ð
    public string? ChangedBy { get; init; }Ð
Ð
    /// <summary>Ð
    /// B BD >C=0 C,,7CĂ5C05C08C•
    /// </summary>Ð
    public string ChangesJson { get; init; } = string.Empty;Ð
}
</file>

```

```

<file path="Domain/DomainObject/DomainObject.cs">
namespace Promatis.Net.Domain;
{
    /// <summary>
    /// AÄACÔ>C$=Cä9 C;C AD „uT"B ?Cä CCÄ>C´GC =C„N)
    /// </summary>
    public abstract class DomainObject : DomainObjectBase<Guid>
    {
        protected DomainObject()
        {
            Id = Guid.NewGuid();
        }
    }
}
</file>

```

```

<file path="Domain/DomainObject/DomainObjectBase.cs">
using Promatis.Net.Domain.Interface;
{
namespace Promatis.Net.Domain;
{
    /// <summary>
    /// B4=C„2CT@D 0C´LCÔKC´ 1C 7Cä2D´9 C;C AD A CÔ>CD4CT@Cd:Cä9 C´NC >C4> D$8Cö0 C;DäGC
    /// </summary>
    /// <typeparam name="TKey">B$8Cö :C´NDt0</typeparam>
    public abstract class DomainObjectBase<TKey> : IDomainObjectHasKey<TKey>
    {
        public TKey Id { get; set; } = default!;
    }
    public DateTime CreatedAt { get; init; } = DateTime.UtcNow;
}
}
</file>

```

```

<file path="Domain/DomainObject/DomainObjectValidator.cs">
using FluentValidation;
{
namespace Promatis.Net.Domain;
{
    /// <summary>
    /// B4=C„2CT@D 0C´LCÔKC´ 2C ;C„4C BCä@ CD;Dò 2D 5DR :C´0D ACä2, CÔ0D ;CT4D45CÄKDR >D" FöÖ -äó&!V7M
    /// </summary>
    public abstract class DomainObjectValidator<T> : AbstractValidator<T> where T : DomainObject
    {
        protected DomainObjectValidator()
        {
            RuleFor(x => x.Id)
                .NotEmpty().WithMessage("A„4CT=D$8DD8Cä0D$>D >C JCT:D$0 CÔ5 CÄ>Cd5D" 1D´BDÄ ?D4AD$KCÄâ""½
        }
        RuleFor(x => x.CreatedAt)
            .Must(date => date <= DateTime.UtcNow.AddSeconds(1))
            .WithMessage("AD0D$0 D >Ct4C =C„0 CÔ5 CÄ>Cd5D" 1D´BDÄ 2 C CCDCD"5CÄâ""½
        }
    }
}
</file>

```

```

<file path="Domain/EntityCloner/JsonEntityCloner.cs">
using System.Text.Json;
using System.Text.Json.Serialization;
using System.Text.Json.Serialization.Metadata;
using Promatis.Net.Domain.Interface;
{
namespace Promatis.Net.Domain;
{
    public class JsonEntityCloner : IEntityCloner
    {
        private static readonly JsonSerializerOptions JsonOptions = new()
        {
            ReferenceHandler = ReferenceHandler.IgnoreCycles,
            TypeInfoResolver = new DefaultJsonTypeInfoResolver
            {
                Modifiers = { typeInfo =>
                {
                    foreach (var property in typeInfo.Properties)
                    {
                        // 1. A$KD 5Ct0CT< Dd8C;C„GCTACä8CR =C 2C„3C FC„>CÔ=D´5 D 2Cä9D BC$0 C„5D 0D EC„9D
                        if (property.Name is "Parent" or "Children")
                    }
                }
            }
        }
    }
}

```



```

        {
            property.ShouldSerialize = (_, _) => false;
            continue;
        }

        // 2. A$KD 5Ct0CT< D ;Cä6C0KCR AC$0Ct0C0=D´5 CD>CÄ5C0=D´5 Cä1D=5C=BD² 1D0:CT=CD0 C, :Cä;C
        Type propType = property.PropertyType;
        bool isDomainClass = typeof(Domain.DomainObject).IsAssignableFrom(propType);
        bool isDomainCollection = propType.IsGenericType &&

typeof(System.Collections.IEnumerable).IsAssignableFrom(propType) &&

typeof(Domain.DomainObject).IsAssignableFrom(propType.GetGenericArguments().FirstOrDefault());
        if (isDomainClass || isDomainCollection)
        {
            property.ShouldSerialize = (_, _) => false;
        }
    }
}

public T CloneEntity<T>(T entity) where T : class
{
    if (entity == null) throw new ArgumentNullException(nameof(entity));

    // B 5D 8C ;C,,7D45CÄ 8 CD5D 5D 8C ;C,,7D45CÄÄ AD$@Cä3Cä ACäED 0C00D0 @CT0C´LC0KC´ @C =D$0C"<-D$8C0 =C
    string json = JsonSerializer.Serialize(entity, entity.GetType(), JsonOptions);
    return (T)JsonSerializer.Deserialize(json, entity.GetType(), JsonOptions)!;
}
}
</file>

<file path="Domain/Promatis.Net.Domain.csproj">
<Project Sdk="Microsoft.NET.Sdk">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
    </PropertyGroup>
    <ItemGroup>
        <PackageReference Include="FluentValidation" Version="12.1.1" />
    </ItemGroup>
    <ItemGroup>
        <ProjectReference Include="..\Domain.Interface\Promatis.Net.Domain.Interface.csproj" />
    </ItemGroup>
</Project>
</file>

<file path="Domain/ReferenceBase/ReferenceBase.cs">
using Promatis.Net.Domain.Interface;
namespace Promatis.Net.Domain;
/// <summary>
/// A 0Ct>C$KC´ :C´0D A CD;D0 2D 5DR AC0@C 2CäGC08C= >C-
/// </summary>
public abstract class ReferenceBase : DomainObject, ISoftDeletable
{
    public string Name { get; set; } = string.Empty;

    public string? Description { get; set; }

    public string? Code { get; set; }

    public DateTime? DeletedAt { get; set; }

    public string? DeletedBy { get; set; }
}
</file>

```

```

<file path="Domain/ReferenceBase/ReferenceBaseValidator.cs">
using FluentValidation;
using FluentValidation.Results;
{
namespace Promatis.Net.Domain;
{
/// <summary>
/// B4=C,,2CT@D 0C'LC0KC' 2C ;C,,4C BCä@ CD;Dò 2D 5DR :C'0D ACä2, C00D ;CT4D45CÄKDR >D" &Vfw&Væ6T& 6]
/// </summary>
public abstract class ReferenceBaseValidator<T> : DomainObjectValidator<T> where T : ReferenceBase
{
/// <summary>
/// A$8D BD40C'LC0KC' ?C BD$5D = D 5C4CC'OD =Cä3Cä 2D'@C 6CT=C,,0 CD;Dò ?D >C$5D :C, Cä4C í
/// Aô> D4<Cä;Dt0C08Dä¢ AD$@Cä3Cä ;C BC,,=C,,FC 2 C$5D EC05CÄ @CT3C,,AD$@CRÄ FC,,DD K C, òí
/// </summary>
protected virtual string CodeRegexPattern => @"^[A-Z0-9_]*$";
{
/// <summary>
/// A$8D BD40C'LC0>CR ACä>C ICT=C,,5 Cäl CäHC,,1C¤5 C$0C'8CD0Dd8C, @CT3D4;Dô@C0>C4> C$KD 0Cd5C08Dò Cä4C í
/// </summary>
protected virtual string CodeValidationMessage => "A¤>CB <Cä6CTB D >CD5D 6C BDÄ BCä;DÄ:Cä ;C BC,,=C,,FD2 2
{
protected ReferenceBaseValidator()
{
RuleFor(x => x.Name)
.NotEmpty().WithMessage("A00C,,<CT=Cä2C =C,,5 CälDô7C BCT;DÄ=Cä 4C'0 Ct0C0>C'=CT=C,,0")
// Aô@Cä2CT@C¤0 C00 Cô@CälCT;D² 2 C00Dt0C'5 C, :Cä=Dd5D
.Must(name => name == null || name.Trim() == name)
.WithMessage("A00C,,<CT=Cä2C =C,,5 C05 CÄ>Cd5D" =C GC,,=C BDÄADò 8C'8 Ct0C¤0C0GC,,2C BDÄADò ?D >C 5C'
.MinimumLength(2).WithMessage("A00C,,<CT=Cä2C =C,,5 CD>C'6C0> D >CD5D 6C BDÄ <C,,=C,,<D4< 2 D 8CÄ2Cä;
.MaximumLength(250).WithMessage("Aô@CT2D'HCT=C <C :D 8CÄ0C'LC00Dò 4C'8C00 C00C,,<CT=Cä2C =C,,0 (25
{
RuleFor(x => x.Code)
.Matches(x => CodeRegexPattern)
.When(x => !string.IsNullOrEmpty(x.Code))
.WithMessage(x => CodeValidationMessage);
}
{
public Func<object, string, Task<IEnumerable<string>>> ValidateValue => async (model,
propertyName) =>
{
ValidationResult? result = await
ValidateAsync(ValidationContext<T>.CreateWithOptions((T)model, x =>
x.IncludeProperties(propertyName)));
return result.IsValid ? [] : result.Errors.Select(e => e.ErrorMessage);
};
}
}
</file>

<file path="Domain/ReferenceTreeBase/ReferenceTreeBase.cs">
using Promatis.Net.Domain.Interface;
{
namespace Promatis.Net.Domain;
{
/// <summary>
/// A 0Ct>C$KC' :C'0D A CD;Dò 2D 5DR ACô@C 2CäGC08C¤>C" ACä4CT@Cd0D"8DR 4CT@CT2DÄ0D
/// </summary>
/// <summary>
/// A 0Ct>C$KC' :C'0D A CD;Dò 2D 5DR 4D 5C$>C$8CD=D'E D ?D 0C$>Dt=C,,:Cä2 (MDM).D
/// A,,ACô>C'LTcCTB C00D$BCT@C0 5%E 4C'0 C05D 5CD0Dt8 D BD >C4> D$8C08Ct8D >C$0C0=D'E C00C$8C40Dd8Cä=C0KDR A
/// </summary>
/// <typeparam name="T">B$8Cò :Cä=C¤@CTBC0>C4> CD>CÄ5C0=Cä3Cä >C JCT:D$0 (C00D ;CT4C08C¤0).</typeparam>
public abstract class ReferenceTreeBase<T> : ReferenceBase, ITreeNode<T> where T : class
{
/// <summary>
/// A,,4CT=D$8DD8C¤0D$>D @Cä4C,,BCT;DÄAC¤>C4> D47C'0 C" !B4 ABí
/// </summary>
public Guid? ParentId { get; set; }
{
/// <summary>
/// A00C$8C40Dd8Cä=C0>CR AC$>C"AD$2Cä AD KC':C, =C @Cä4C,,BCT;DÄAC¤8C' >C JCT:D" 2 Cä?CT@C BC,,2C0>C' ?C <
/// A05D 5Cä?D 5CD5C'OCTBD 0 C¤0C¢ f-'GV Ä 2 C¤>C05Dt=D'E C¤;C AD 0DR 4C'0 C0>CD4CT@Cd:C, Æ $' Æ0 F-ærí
/// </summary>
public virtual T? Parent { get; set; }
{
/// <summary>

```

```

    /// A>C';CT:Dd8Dò 4CäGCT@CÔ8DR MC'5CÄ5CÔBCä2 (Cò>CDGC,,=CT=CÔKDR CCT;Cä2).D
    /// </summary>D
    public virtual ICollection<T> Children { get; set; } = new List<T>();D
}
</file>

<file path="Domain/ReferenceTreeBase/ReferenceTreeBaseValidator.cs">
using FluentValidation;D
using Promatis.Net.Domain.Interface;D
D
namespace Promatis.Net.Domain;D
D
/// <summary>D
/// B4=C,,2CT@D 0C'LCÔKC' 2C ;C,,4C BCä@ CD;Dò 2D 5DR :C'0D ACä2, CÔ0D ;CT4D45CÄKDR >D" &VfW&Væ6UG&VT& 6]
/// </summary>D
public abstract class ReferenceTreeBaseValidator<T> : ReferenceBaseValidator<T>D
    where T : ReferenceBase, ITreeNode<T>D
{D
    protected ReferenceTreeBaseValidator() : base()D
    {D
        // Aô@Cä2CT@C=D CÔ0 D 0CÄ>Dd8D$8D >C$0CÔ8CR ,,>C JCT:D" =CR <Cä6CTB D4:C 7D'2C BDÂ =C ACT1Dò :C : CÔ0
        RuleFor(x => x.ParentId)D
            .Must((model, parentId) => parentId != model.Id)D
            .WithMessage("Aä1D=D5C=D CÔ5 CÄ>Cd5D" 1D'BDÂ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5");D
    D
        // A 0Ct>C$0Dò ?D >C$5D :C DCä@CÄ0D$0 GUIDD
        RuleFor(x => x.ParentId)D
            .NotEqual(Guid.Empty)D
            .When(x => x.ParentId.HasValue)D
            .WithMessage("B >CD8D$5C'LD :C,,9 C,,4CT=D$8DD8C=D0D$>D =CR <Cä6CTB C KD$L CòCD BD'< GUID");D
    }D
}
</file>

<file path="Promatis.Net.UI/_Imports.razor">
@using Microsoft.AspNetCore.Components.FormsD
@using Microsoft.AspNetCore.Components.RoutingD
@using Microsoft.AspNetCore.Components.WebD
@using MudBlazorD
@using Promatis.Net.UI.Components.WorkspacesD
@using Promatis.Net.Domain.InterfaceD
@using Promatis.Net.UI.Components.Dialogs
</file>

<file path="Promatis.Net.UI/Components/ActionContext/CrudWorkspaceContext.cs">
using MudBlazor;D
using Promatis.Net.Service;D
using Promatis.Net.UI.Controls;D
D
namespace Promatis.Net.UI.Components;D
D
/// <summary>D
/// Aä1Cä1D"5CÔ=D'9 C 0Ct>C$KC' :Cä=D$5C=DAD" 4C'0 D BC =CD0D BCÔKDR 5%TBÔMC=D@C =Cä2 Cò;C BDD>D <D²í
/// Aô>C'=CäAD$LDä 0C$BCä<C BC,,7C,,@D45D" DCä=Cä2D4N Ct0C4@D47C=D C D0CÔ=D'E CÔ0 C 0Ct5 IBaseService.D
/// </summary>D
/// <typeparam name="TEntity">B$8Cò 4Cä<CT=CÔ>C' AD4ICÔ>D BC, ,,4Cä;Cd5Cò 1D'BDÂ :C'0D ACä< D :Cä=D BD CC=D BCä
/// <typeparam name="TKey">B$8Cò ?CT@C$8Dt=Cä3Cä :C'NDt0 D CD"=CäAD$8 (Guid, int, string C, B.CBâ'ää÷G— W &
public abstract class CrudWorkspaceContext<TEntity, TKey> : WorkspaceActionContext,
    IHasSelectedData<TEntity>D
    where TEntity : class, new()D
    where TKey : notnullD
{D
    private readonly IBaseService<TEntity, TKey> _baseService;D
    private TEntity? _selectedData;D
    private bool _isLoading;D
    D
    public UiDataBroker<TEntity, GridState<TEntity>, GridData<TEntity>> Broker { get; protected
set; }D
    public IUiOzuCache<TEntity> OzuCache { get; }D
    D
    public TEntity? SelectedDataD
    {D
        get => _selectedData;D
        setD
        {D
            if (_selectedData != value)D

```

```

        {
            _selectedData = value;
            OnContextUpdated?.Invoke();
            NotifyStateChanged();
        }
    }

    public Action? OnContextUpdated { get; set; }
    public bool IsLoading => _isLoading;

    protected CrudWorkspaceContext(IBaseService<TEntity, TKey> baseService, bool isInMemoryMode)
    {
        _baseService = baseService ?? throw new ArgumentNullException(nameof(baseService));
        OzuCache = new UiOzuCache<TEntity>();
        Broker = new UiDataBroker<TEntity, GridState<TEntity>, GridData<TEntity>>();

        // A,,=C,,FC,,0C`8Ct8D CCT< D0;CT<CT=D$K D4?D 0C$;CT=C,,0 Dt5D 5Cr 2C,,@D$CC ;DÄ=D`9 CÄ5D$>CM
        InitializeToolbarControls();

        if (isInMemoryMode)
        {
            Broker.ConfigureInMemoryMode(OzuCache, EvaluateGridStateInMemory);
        }
        else
        {
            Broker.ConfigureServerMode(FetchGridDataFromServerAsync);
        }
    }

    /// <summary>
    /// A00C0>C`=CT=C,,5 D$CC`1C @C :C0>C0:C <C, ?Cä CCÄ>C`GC =C,,N. A00D ;CT4C08C=8 CÄ>C4CD" ?Cä4C 2C,,BDÂ 8C`
    /// </summary>
    protected virtual void InitializeToolbarControls()
    {
        AddControl(new CreateEntityButton<TEntity>());
        AddControl(new EditEntityButton<TEntity>());
        AddControl(new DeleteEntityButton<TEntity>());
        AddControl(new ToolbarDivider());
    }

    public async Task LoadInitialDataAsync()
    {
        if (!Broker.IsInMemoryMode) return;

        try
        {
            _isLoading = true;
            NotifyStateChanged();

            List<TEntity> serverItems = await _baseService.GetAllAsync();
            OzuCache.InMemoryItems = serverItems ?? new List<TEntity>();
        }
        finally
        {
            _isLoading = false;
            NotifyStateChanged();
        }
    }

    protected GridData<TEntity> EvaluateGridStateInMemory(GridState<TEntity> state, List<TEntity> inMemoryList)
    {
        return new GridData<TEntity>
        {
            Items = inMemoryList.Skip(state.Page * state.PageSize).Take(state.PageSize).ToList(),
            TotalItems = inMemoryList.Count
        };
    }

    private async Task<GridData<TEntity>> FetchGridDataFromServerAsync(GridState<TEntity> state, Cancellation token ct)
    {
        // B$5C05D L Ct4CTADÂ @C 1CäBC 5D" GCTAD$=C 0 D 5D 2CT@C00D0 ?C 3C,,=C FC,,0 C 5Cr CD$5Dt5C$ ?C <D0BC•
        PagedResult<TEntity> pagedResult = await _baseService.GetPagedAsync(state.Page, state.PageSize, ct);
    }

```

```

    return new GridData<TEntity>()
    {
        Items = pagedResult.Items,
        TotalItems = pagedResult.TotalCount
    };
}

</file>

<file path="Promatis.Net.UI/Components/ActionContext/IWorkspaceActionContext.cs">
namespace Promatis.Net.UI.Components;

/// <summary>
/// AT4C,,=D'9 C=C0BD 0C=B C=C0BCT:D BC @C 1CäGCT9 Cä1C'0D BC,Ä CC0@C 2C'0DäIC,,9 C45Cä<CTBD 8CT9 C00D$8 Ct>
/// D >D BC 2Cä< CD>D BD4?C0KDR T'04CT9D BC$8C' 8 D$5C=C'D'8CÄ 2D'4CT;CT=C,,5CÄ 4C =C0KDR =C MC@C =CR1
/// </summary>
public interface IWorkspaceActionContext
{
    // --- B Ää B4 B A$ AT A,,/ A ÄÄ Ää AT B$ ÄÄ A, A A0+ÄÄ ---
    /// <summary>
    /// A=C';CT:Dd8D0 8C0BCT@C :D$8C$=D'E D0;CT<CT=D$>C" CC0@C 2C'5C08D0 ,, :C0>C0>C$Ä DC,,;DÄBD >C"Ä ?CT@CT:C'
    /// </summary>
    IEnumerable<IUIControl> Controls { get; }

    event Action? OnContextStateChanged;
    void NotifyStateChanged();

    // --- A0 B ÄÄ B$ B2 !B$ A' At Bd A, A4 Ää AT"B A, R Ää ---
    int PaperElevation { get; }
    string PaperClass { get; }
    string WorkspaceHeight { get; }
    string TopZoneHeight { get; }
    string BottomZoneHeight { get; }
    string LeftZoneWidth { get; }
    string RightZoneWidth { get; }

    bool IsTopZoneCollapsed { get; set; }
    bool IsBottomZoneCollapsed { get; set; }
    bool IsLeftZoneCollapsed { get; set; }
    bool IsRightZoneCollapsed { get; set; }
}

</file>

<file path="Promatis.Net.UI/Components/ActionContext/WorkspaceActionContext.cs">
namespace Promatis.Net.UI.Components;

/// <summary>
/// A 0Ct>C$KC' :Cä=D$5C=AD" ;Dä1Cä9 D 0C >Dt5C' >C ;C AD$8 (C$:C'0CD:C,' 2 D 8D BCT<CR1
/// Ää1D'8C' 7C00CÄ5C00D$5C'L CD;D0 <C05CÄ>D ECT<, D ?D 0C$>Dt=C,,:Cä2, CD0D,,1Cä@CD>C" 8 C4@C DC,,:Cä2.0
/// </summary>
public class WorkspaceActionContext : IWorkspaceActionContext
{
    protected readonly List<IUIControl> _controls = new();

    public IEnumerable<IUIControl> Controls => _controls;

    public event Action? OnContextStateChanged;

    public void NotifyStateChanged() => OnContextStateChanged?.Invoke();

    // --- AD BD A'"A0 B0 A !B$ Ää A A4 Ää AT"B A, B "A,, AT ---
    public virtual int PaperElevation => 1;
    public virtual string PaperClass => "pa-4 d-flex flex-column flex-grow-1 w-100 h-100";
    public virtual string WorkspaceHeight => "100%";
    public virtual string TopZoneHeight => "auto";
    public virtual string BottomZoneHeight => "auto";
    public virtual string LeftZoneWidth => "250px";
    public virtual string RightZoneWidth => "300px";

    public bool IsTopZoneCollapsed { get; set; } = false;
    public bool IsBottomZoneCollapsed { get; set; } = false;
    public bool IsLeftZoneCollapsed { get; set; } = false;
    public bool IsRightZoneCollapsed { get; set; } = false;
}

```

```

Ð
// --- AÄ B$ AD+ B4 B A$ AT A,,/ Aô Aô A´,Bâ A´/ Aô B$ ÄÄ Aä ---Ð
protected void AddControl(IUIControl control)Ð
{Ð
    _controls.Add(control);Ð
    NotifyStateChanged();Ð
}Ð
Ð
protected void RemoveControl(string controlId)Ð
{Ð
    _controls.RemoveAll(c => c.Id == controlId);Ð
    NotifyStateChanged();Ð
}Ð
}
</file>

<file path="Promatis.Net.UI/Components/Dialogs/Buttons/CancelButton.cs">
using MudBlazor;Ð
using Promatis.Net.UI.Components.ElementRenderBase;Ð
Ð
namespace Promatis.Net.UI.Components.Dialogs.Buttons;Ð
Ð
/// <summary>Ð
/// A 2D$>CÔ>CÄ=C 0 C>CÄ@CÔ4C >D$<CT=D² 8 Ct@C@D´BC,,0 C´NC >C4> CD8C ;Cä3Cä2Cä3Cä >C=C í
/// </summary>Ð
public class CancelDialogButton : BaseUiControlÐ
{Ð
    private readonly Action _cancelAction;Ð
    private readonly string _id = "dialog_action_cancel_" + Guid.NewGuid().ToString("N");Ð
    Ð
    public override string Id => _id;Ð
    public override Type ComponentType => typeof(ButtonRenderBase);Ð
    public override string Title => "AäBCÄ5CÔ0";Ð
    Ð
    public CancelDialogButton(Action cancelAction)Ð
    {Ð
        _cancelAction = cancelAction ?? throw new ArgumentNullException(nameof(cancelAction));Ð
        Ð
        ComponentParameters.Add("Color", Color.Default);Ð
        ComponentParameters.Add("Variant", Variant.Text);Ð
        Ð
        IsEnabled = true;Ð
    }Ð
    Ð
    protected override Task HandleTriggerAsync(object? targetData)Ð
    {Ð
        // AÄ3CÔ>C$5CÔ=Cä 7C :D KC$@CT< Cä:CÔ> Dt5D 5Cr MC=HCT= C@0D :C AC 4C,,0C´>C40Ð
        _cancelAction.Invoke();Ð
        return Task.CompletedTask;Ð
    }Ð
}
</file>

<file path="Promatis.Net.UI/Components/Dialogs/Buttons/SubmitDialogButton.cs">
using MudBlazor;Ð
using Promatis.Net.UI.Components.ElementRenderBase;Ð
Ð
namespace Promatis.Net.UI.Components.Dialogs.Buttons;Ð
Ð
/// <summary>Ð
/// A 2D$>CÔ>CÄ=C 0 C>CÄ@CÔ4C 7C ?D4AC@0 C$@C´8CD0Dd8C, 8 CÔ>CDBC$5D 6CD5CÔ8Dð 8Ct<CT=CT=C,,9 C´NC >C4> CD8C
/// </summary>Ð
public class SubmitDialogButton : BaseUiControlÐ
{Ð
    private readonly Func<Task> _submitAction;Ð
    private readonly string _id = "dialog_action_submit_" + Guid.NewGuid().ToString("N");Ð
    Ð
    public override string Id => _id;Ð
    public override Type ComponentType => typeof(ButtonRenderBase);Ð
    public override string Title => "B >DT@C =C,,BDÄ#½
    Ð
    public SubmitDialogButton(Func<Task> submitAction)Ð
    {Ð
        _submitAction = submitAction ?? throw new ArgumentNullException(nameof(submitAction));Ð
        Ð
        ComponentParameters.Add("Color", Color.Primary);Ð

```

```

        ComponentParameters.Add("Variant", Variant.Filled);
    }
    IsEnabled = true;
}

protected override async Task HandleTriggerAsync(object? targetData)
{
    // At0C0CD :C 5CÂ BD 8C43CT@ C$0C`8CD0Dd8C, 8 Ct0C@D`BC,,0 C00 D4@Cä2C05 UI-Cä:C00
    await _submitAction.Invoke();
}
}
</file>

<file path="Promatis.Net.UI/Components/Dialogs/DynamicDialog/DialogActionContextBase.cs">
using FluentValidation;
using FluentValidation.Results;
namespace Promatis.Net.UI.Components.Dialogs.DynamicDialog;
/// <summary>
/// A 1D BD 0C=BC0KC' 1C 7Cä2D`9 C=C AD :Cä=D$5C=AD$0 D4?D 0C$;CT=C,,0 CD8C ;Cä3Cä2D`< Cä:C0>CÄ1
/// A,,=C=0C0AD4;C,,@D45D" AD$@Cä3Cä BC,,?C,,7C,,@Cä2C =C0CDä <Cä4CT;DÄÄ :Cä;C`5C=FC,,8 D0;CT<CT=D$>C" CC0@C 2C`5C0
/// </summary>
/// <typeparam name="TModel">B$8C0 @CT4C :D$8D CCT<Cä9 CD>CÄ5C0=Cä9 D CD"=CäAD$8.</typeparam>
public abstract class DialogActionContextBase<TModel> : IDialogActionContext<TModel>
    where TModel : class, new()
{
    private readonly List<IUIControl> _actions = [];
    private readonly List<DialogTabConfig<TModel>> _tabs = [];
    private readonly IValidator? _validator;

    /// <summary>
    /// B BD >C4> D$8C08Ct8D >C$0C0=C O D AD`;C=0 C00 D 5CD0C=BC,,@D45CÄCDä 4Cä<CT=C0CDä <Cä4CT;DÄ 4C =C0KDR1
    /// A,,7CÄ5C05C08D0 ?C,,HD4BD O C00C0@D0<D4N C" MD$>D" >C JCT:D" ?Cä AD KC`:CR 2 Cä?CT@C BC,,2C0>C' ?C <D0BC
    /// </summary>
    public TModel Model { get; }

    /// <summary>
    /// A0;CäAC=8C' AC08D >C 4C,,=C <C,,GCTAC=8DR :C0>C0>C04CT9D BC$8C' ,,Cä<C =CB' ?C =CT;C, 4CT9D BC$8C' 4C
    /// </summary>
    public IReadOnlyCollection<IUIControl> Actions => _actions.AsReadOnly();

    /// <summary>
    /// B BD >C4> D$8C08Ct8D >C$0C0=C O C=C`;CT:Dd8D0 220:Cä=DD8C4CD 0Dd8C' 2C=C 4Cä: CD;D0 >D$@C,,ACä2C=8 M
    /// </summary>
    public IReadOnlyCollection<DialogTabConfig<TModel>> Tabs => _tabs.AsReadOnly();

    protected DialogActionContextBase(TModel model, IValidator? validator)
    {
        Model = model ?? throw new ArgumentNullException(nameof(model));
        _validator = validator;
    }

    /// <summary>
    /// AD>C 0C$;CT=C,,5 C=C=Cä?C=8-CD5C"AD$2C,,0 C00 C00C05C`L CD8C ;Cä3C 1
    /// </summary>
    public void AddAction(IUIControl? action)
    {
        if (action == null) return;
        _actions.Add(action);
    }

    /// <summary>
    /// AD>C 0C$;CT=C,,5 C#-C=C0DC,,3D4@C FC,,8 C$:C`0CD:C, 2 Cä1D"CDä :Cä;C`5C=FC,,N DD>D <D²1
    /// </summary>
    public void AddTab(DialogTabConfig<TModel>? tabConfig)
    {
        if (tabConfig == null) return;
        _tabs.Add(tabConfig);
    }

    /// <summary>
    /// A00D$8C$=C O D$>Dt5Dt=C O C$0C`8CD0Dd8D0 :Cä=C@CTBC0>C4> D 2Cä9D BC$0 C,,=C0CD$0 D 2Cä7C$@C BCä< D 5
    /// ASKtKC$0CTBD O C,,=CD8C$8CDCC ;DÄ=Cä :C 6CDKcÄ 8C0?D4BCä< MudBlazor C0@C, ?CäBCT@CR DCä:D4AC 8C`8 C,,
    /// </summary>
    public async Task<IEnumerable<string>> ValidateValueAsync(string propertyName)

```

```

    {
        if (_validator == null || string.IsNullOrWhiteSpace(propertyName))
            return [];
    }

    // B >Ct4C 5CÂ :Cä=D$5C=AD" fÇVVçEf Æ-F F-öâÂ 8Ct>C´8D CDò ?D >C$5D :D2 !B$ Aä Aâ 4C´O Cä4CÔ>C4> D 2C
    // A ;C 3Cä4C @Dò MD$>CÄC GlobalPolymorphicValidator CÔ0 C$ECä4CR CC$8CD8D" GCTAD$=D´9 TModel, C =CR
    ValidationContext<TModel>? context = ValidationContext<TModel>.CreateWithOptions(Model,
options =>
    options.IncludeProperties(propertyName));
    ValidationResult result = await _validator.ValidateAsync(context);
    if (result.IsValid)
        return [];
    // A,,7C$;CT:C 5CÂ ACä>C ICT=C,,O Cä1 CäHC,,1C=0DR 1DÔ:CT=CD0, CäBCÔ>D OD"8CTADò 8D :C´NDt8D$5C´LCÔ> Cç
    return result.Errors
        .Where(e => e.PropertyName == propertyName)
        .Select(e => e.ErrorMessage);
}
}
</file>

```

```

<file path="Promatis.Net.UI/Components/Dialogs/DynamicDialog/DialogTabConfig.cs">
using Microsoft.AspNetCore.Components;
namespace Promatis.Net.UI.Components.Dialogs.DynamicDialog;
/// <summary>
/// Bt8D BC O C#-C=CO0DC,,3D4@C FC,,O C$:C´0CD:C, 4C,,0C´>C4>C$>C4> Cä:CÔ0.
/// A,,=C=0C0AD4;C,,@D45D" <CTBC 4C =CÔKCR 7C 3Cä;Cä2C=0 C, AD$@Cä3Cä BC,,?C,,7C,,@Cä2C =CÔKC´ DD 0C4<CT=D" @C 7CÄ
/// </summary>
/// <typeparam name="TModel">B$8Cò @CT4C :D$8D CCT<Cä9 CD>CÄ5CÔ=Cä9 D CD"=CäAD$8.</typeparam>
public class DialogTabConfig<TModel> where TModel : class
{
    /// <summary>
    /// B$5C=AD$>C$KC´ 7C 3Cä;Cä2Cä: C$:C´0CD:C, =C ?C =CT;C, xVEF '21
    /// </summary>
    public string Title { get; }
    /// <summary>
    /// B 8D BCT<CÔ0Dò 8C=CO:C xVD&Æ |÷" 4C´O Ct0C4>C´>C$:C 2C=C 4C=8 (Cä?Dd8Cä=C ;DÄ=Cä´i
    /// </summary>
    public string? Icon { get; }
    /// <summary>
    /// B BD >C4> D$8C08Ct8D >C$0CÔ=C O D 0Ct<CTBC=0 CÔ>C´5C´ 2C$>CD0, CÔ@C,,=C,,<C ND"0Dò 6C,,2D4N CD>CÄ5CÔ=D4N
    /// </summary>
    public RenderFragment<TModel> Content { get; }
    public DialogTabConfig(string title, RenderFragment<TModel> content, string? icon = null)
    {
        if (string.IsNullOrWhiteSpace(title))
            throw new ArgumentException("At0C4>C´>C$>Cç 2C=C 4C=8 CÔ5 CÄ>Cd5D" 1D´BDÄ ?D4AD$KCÄâ"Ä æ ÖVöb±F-
        Title = title;
        Content = content ?? throw new ArgumentNullException(nameof(content));
        Icon = icon;
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Components/Dialogs/DynamicDialog/DynamicEditDialog.razor">
@typeparam TModel where TModel : class
<MudDialog>
    <TitleContent>
        <MudText Typo="Typo.h6">
            @Title
        </MudText>
    </TitleContent>
    <DialogContent>
        <MudForm @ref="_form"
            Model="@Context.Model"
            ValidationDelay="0">

```



```

        @* A$0C´8CD0Dd8Dò 8CÔ?D4BCä2 C,,4CTB C,,7Cä;C,,@Cä2C =CÔ> Dt5D 5Cr f Æ-F Fuf ÇVT 7-æ2 ¢
    }
    @* A»>CÔBCT9CÔ5D 6CTAD$:Cä 4CT@Cd8D" 2D´ACäBD2 >C»=C ?Cä =C HCT<D2 ?D >C$5D 5CÔ=Cä<D2 :C =Cä=D2
    <div class="d-flex flex-column pt-2" style="min-height: 340px; height: 60vh; max-
height: 60vh; min-width: 0; overflow: hidden;">
    }
        @if (Context.Tabs.Count == 1){
        {
            @* B &AT A A,, A ¢ D 5C4> Cä4CÔ0 C$:C´0CD:C r 2D´2Cä4C,,< CÔ>C´0 D ?C´>D,,=D´< D ?C,,AC»>
            <div class="flex-grow-1 pa-4" style="height: 100%; overflow-y: auto; padding-
right: 4px;">
                @Context.Tabs.First().Content(Context.Model)
            </div>
        }
        else if (Context.Tabs.Count > 1){
        {
            @* B &AT A A,, A ¢ D4;DÄBC,Ô2C»;C 4C»8 – D 5CÔ4CT@C,,< MudTabs, CÔ5D 5CD0C$0Dò <Cä4CT;DÄ
            <MudTabs Elevation="0"
                Rounded="true"
                Class="d-flex flex-column flex-grow-1 h-100"
                @bind-ActivePanelIndex="_activeTabIndex">
                @foreach (DialogTabConfig<TModel> tab in Context.Tabs)
                {
                    <MudTabPanel Text="@tab.Title" Icon="@tab.Icon" PanelClass="pa-4 flex-
grow-1">
                        <div style="max-height: 48vh; overflow-y: auto; padding-right:
4px;">
                            @tab.Content(Context.Model)
                        </div>
                    </MudTabPanel>
                }
            </MudTabs>
        }
    }
    </div>
    </MudForm>
    </DialogContent>
    <DialogActions>
        @* Aô>C´8CÄ>D DCÔKCR :CÔ>Cô:C, 8Cr :Cä=D$5C»AD$0 D 5CÔ4CT@DôBD 0 Dt5D 5Cr 2C H D BC 1C,,;DÄ=D´9 Dynam
        @foreach (IUIControl action in Context.Actions)
        {
            @if (!action.IsVisible) continue;
            <DynamicComponent Type="@action.ComponentType"
Parameters="@action.ComponentParameters" />
        }
    </DialogActions>
    </MudDialog>
    </file>

<file path="Promatis.Net.UI/Components/Dialogs/DynamicDialog/DynamicEditDialog.razor.cs">
using Microsoft.AspNetCore.Components;
using MudBlazor;
}
namespace Promatis.Net.UI.Components.Dialogs.DynamicDialog;
}
public partial class DynamicEditDialog<TModel> : ComponentBase where TModel : class
{
    protected MudForm _form = null!;
    protected int _activeTabIndex;

    [CascadingParameter]
    protected IMudDialogInstance MudDialog { get; set; } = null!;

    [Parameter]
    public required IDialogActionContext<TModel> Context { get; set; }

    [Parameter]
    public string Title { get; set; } = "B 5CD0C»BC,,@Cä2C =C,,5 Ct0CÔ8D 8";

    protected override void OnInitialized()
    {
        base.OnInitialized();
        if (Context == null)
            throw new ArgumentNullException(nameof(Context), "A,,=D$5D DCT9D "F- Æöt 7F-öä6öçFW‡CÄDÖöFVÃâ 4Cä
CD8C ;Cä3C â""½

```

```

    }
}
}
/// <summary>
/// AöCC ;C,,GCÖKC' <CTBCä4 CD;Dò :CÔ>Cô:C, 7V&Ö-DF- Æöt'WGFöâí
/// AtöCöCD :C 5D" ?Cä;CÖCDä 2C ;C,,4C FC,,N DD>D <D² 8 CtöC@D'2C 5D" 4C,,öC'>C2 A D4ACö5D,,=D'< D 5CtCC'LD$
/// </summary>
public async Task HandleSubmitFormAsync()
{
    if (_form == null) return;

    // Aö@C,,=D44C,,BCT;DÄ=Cä AC@öCö8D CCT< C$ADä DCä@CÄC MudForm
    await _form.ValidateAsync();

    if (_form.IsValid)
    {
        // A,,!Aö A A' Aö : Aö5D 5CDöCT< CÄ>CD5C'L C" ö²,'Â GD$>C K D44Cä2C'5D$2Cä@C,,BDÄ AC,,3Cö0D$CD C CÄ
        // ADöCö=D'5 Cö>-Cö@CT6Cö5CÄC Cä1Cö>C$;CT=D² 2 Aä B2 ?Cä AD KC':CR
        MudDialog.Close(DialogResult.Ok(Context.Model));
    }
}
}
/// <summary>
/// AöCC ;C,,GCÖKC' <CTBCä4 CD;Dò :CÔ>Cô:C, 6 æ6VÄF- Æöt'WGFöâí
/// AtöC@D'2C 5D" 4C,,öC'>C2 2 D 5Cd8CÄ5 CäBCÄ5CÖK C 5Cr ACäED öCö5Cö8Döí
/// </summary>
public void HandleCancelForm()
{
    MudDialog.Cancel();
}
}
</file>

```

```

<file path="Promatis.Net.UI/Components/Dialogs/DynamicDialog/IDialogActionContext.cs">
namespace Promatis.Net.UI.Components.Dialogs.DynamicDialog;
{
    /// <summary>
    /// B BD >C4> D$8Cö8Ct8D >C$öCö=D'9 C,,=D$5D DCT9D :Cä=D$5C=AD$ö D4?D öC$;CT=C,,ö CD8C ;Cä3Cä2D'< Cä:Cö>CÄí
    /// </summary>
    /// <typeparam name="TModel">B$8Cö @CT4C :D$8D CCT<Cä9 CD>CÄ5Cö=Cä9 D CD'=CäAD$8.</typeparam>
    public interface IDialogActionContext<TModel> where TModel : class
    {
        /// <summary>
        /// B BD >C4> D$8Cö8Ct8D >C$öCö=C ö D AD';C@ö Cöö D 5CDöC=BC,,@D45CÄCDä 4Cä<CT=CöCDä <Cä4CT;DÄ 4C =CöKDRí
        /// </summary>
        TModel Model { get; }

        /// <summary>
        /// Aö;CäAC=8C' ACö8D >C 4C,,=C <C,,GCTAC=8DR :Cö>Cö>Cöö4CT9D BC$8C' ,, :Cä<C =CB'Â @CT=CD5D 8CÄKDR 2Cö8CtC
        /// </summary>
        IReadOnlyCollection<IUIControl> Actions { get; }

        /// <summary>
        /// B BD >C4> D$8Cö8Ct8D >C$öCö=C ö C@>C';CT:Dd8Dò :Cä=DD8C4CD öDd8C' 2C@;C 4Cä: CD;Dò >D$@C,,ACä2C=8 MudT
        /// </summary>
        IReadOnlyCollection<DialogTabConfig<TModel>> Tabs { get; }

        /// <summary>
        /// AD>C öC$;CT=C,,5 C= Cä?C=8-CD5C"AD$2C,,ö Cöö CööCö5C'L CD8C ;Cä3C í
        /// </summary>
        void AddAction(IUIControl action);

        /// <summary>
        /// AD>C öC$;CT=C,,5 C#-C@>CöDC,,3D4@C FC,,8 C$:C'öCD:C, 2 Cä1D"CDä :Cä;C'5C=FC,,N DD>D <D²í
        /// </summary>
        void AddTab(DialogTabConfig<TModel> tabConfig);

        /// <summary>
        /// AööD$8C$=C ö D$>Dt5Dt=C ö C$öC'8CDöDd8Dò :Cä=C@CTBCö>C4> D 2Cä9D BC$ö C,,=CöCD$ö D 2Cä7C$@C BCä< D 5
        /// </summary>
        Task<IEnumerable<string>> ValidateValueAsync(string propertyName);
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Components/ElementRenderBase/ButtonRenderBase.razor">
@inherits RenderBase

```

```

<MudTooltip Text="@((Control.Tooltip ?? Control.Title)" Arrow="true" Placement="Placement.Bottom">
    <MudButton Variant="@Variant"
        StartIcon="@((Control.IsRunning ? null : Control.Icon)"
        Disabled="@(!Control.IsEnabled || Control.IsRunning || !
Control.IsEnabledForData(CurrentSelectedData))"
        Color="@Color"
        OnClick="@HandleClickAsync">
    }
        @if (Control.IsRunning)
        {
            <MudProgressCircular Class="ms-n1 me-2" Size="Size.Small" Indeterminate="true"
Color="Color.Inherit" />
        }
    }
    @Control.Title
</MudButton>
</MudTooltip>
</file>

```

```

<file path="Promatis.Net.UI/Components/ElementRenderBase/ButtonRenderBase.razor.cs">
using Microsoft.AspNetCore.Components;
using MudBlazor;
}
namespace Promatis.Net.UI.Components.ElementRenderBase;
}
public partial class ButtonRenderBase : RenderBase
{
    [Parameter] public Color Color { get; set; } = Color.Inherit;
    [Parameter] public Variant Variant { get; set; } = Variant.Text;
}
    protected async Task HandleClickAsync()
    {
        await Control.TriggerAsync(CurrentSelectedData);
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Components/ElementRenderBase/DateRangeRenderBase.razor">
@inherits RenderBase
}
<MudDateRangePicker Label="@Control.Title"
    DateRange="@GetDateRange()"
    DateRangeChanged="@HandleDateRangeChangedAsync"
    Disabled="@(!Control.IsEnabled || Control.IsRunning)"
    Margin="Margin.Dense"
    DateFormat="dd.MM.yyyy"
    Variant="Variant.Outlined"
    Style="max-width: 260px;" />
</file>

```

```

<file path="Promatis.Net.UI/Components/ElementRenderBase/DateRangeRenderBase.razor.cs">
using MudBlazor;
}
namespace Promatis.Net.UI.Components.ElementRenderBase;
}
public partial class DateRangeRenderBase : RenderBase
{
    /// <summary>
    /// A 5Ct>C00D =Câ 8Ct2C'5C00CTB Că1D05C0B DateRange C,,7 CĂ>CD5C'8 C0>C0BD >C'0 Dt5D 5Cr 8C0BCT@DD5C"A-D
    /// </summary>
    protected DateRange? GetDateRange()
    {
        if (Control is IHasValue valueProvider)
        {
            return valueProvider.Value as DateRange;
        }
        return null;
    }
}
    /// <summary>
    /// At0C08D KC$0CTB C0>C$>CR 7C00Dt5C08CR 2D'1D 0C0=Că3Că 4C,,0C00Ct>C00 CD0D" 2 CĂ>CD5C'L C, 7C ?D4AC00CT
    /// </summary>
    protected async Task HandleDateRangeChangedAsync(DateRange? newRange)
    {
        if (Control is IHasValue valueProvider)
        {

```

```

        valueProvider.Value = newRange;
        await Control.TriggerAsync(CurrentSelectedData);
    }
}
</file>

<file path="Promatis.Net.UI/Components/ElementRenderBase/DividerRenderBase.razor">
@inherits RenderBase
{
    <MudDivider Vertical="true" FlexItem="true" Class="my-2mx-1" />
</file>

<file path="Promatis.Net.UI/Components/ElementRenderBase/DividerRenderBase.razor.cs">
namespace Promatis.Net.UI.Components.ElementRenderBase;
{
    public partial class DividerRenderBase : RenderBase
    {
    }
</file>

<file path="Promatis.Net.UI/Components/ElementRenderBase/RenderBase.cs">
using Microsoft.AspNetCore.Components;
{
    namespace Promatis.Net.UI.Components.ElementRenderBase;
    {
        /// <summary>
        /// A 1D BD 0C=BC0KC' 1C 7Cä2D'9 C=CA?Cä=CT=D" 4C'0 C$ACTE Razor-D 5C04CT@CT@Cä2 D0;CT<CT=D$>C" CC0@C 2C'5C0
        /// </summary>
        public abstract class RenderBase : ComponentBase, IDisposable
        {
            /// <summary>
            /// A 8Ct=CTA-CÄ>CD5C'L D0;CT<CT=D$0 D4?D 0C$;CT=C,,0, C05D 5CD0C$0CT<C 0 C,,7 DynamicComponent.D
            /// </summary>
            [Parameter]
            public IUIControl Control { get; set; } = null!;

            /// <summary>
            /// A=0D :C 4C0KC' :Cä=D$5C=AD" ECä;D BC @C 1CäGCT9 Cä1C'0D BC,1
            /// </summary>
            [CascadingParameter]
            protected IWorkspaceActionContext? ActionContext { get; set; }

            /// <summary>
            /// A 5Ct>C00D =Cä 8Ct2C'5C=0CTB D$5C=CD"8CR 2D'4CT;CT=C0KCR 4C =C0KCR 8Cr :Cä=D$5C=AD$0 DD>D <D'1
            /// </summary>
            protected object? CurrentSelectedData => GetSelectedDataFromContext();

            protected override void OnInitialized()
            {
                base.OnInitialized();

                // B 5C48D BD 8D CCT< D 5C :D$8C$=Cä5 Cä1C0>C$;CT=C,,5 C,,=D$5D DCT9D 0 C0@C, ACÄ5C05 D >D BCä0C08D0 1C
                Control.OnStateChanged += HandleStateChanged;
            }

            private void HandleStateChanged() => InvokeAsync(StateHasChanged);

            private object? GetSelectedDataFromContext()
            {
                if (ActionContext == null) return null;

                Type? interfaceType = ActionContext.GetType().GetInterface("IHasSelectedData`1");
                return interfaceType?.GetProperty("SelectedData")?.GetValue(ActionContext);
            }

            public virtual void Dispose() => Control.OnStateChanged -= HandleStateChanged;
        }
    }
</file>

<file path="Promatis.Net.UI/Components/ElementRenderBase/SelectRenderBase.razor">
@inherits RenderBase
<MudSelect T="string"
    Value="@GetControlValue()"
    ValueChanged="@HandleValueChangedAsync"
    Disabled="@(Control.IsEnabled || Control.IsRunning)"

```



```

        For="@((() => referenceModel.Name))" />
    }

    <MudTextField @bind-Value="@referenceModel.Description"
        Label="Aä?C,,AC =C,,5 / Aô@C,,<CTGC =C,,5"
        Variant="Variant.Outlined"
        Margin="Margin.Dense"
        Lines="3"
        For="@((() => referenceModel.Description))" />

    </div>
    </Content>
    </DialogTab>
    </MudTabs>
    </Tabs>
</DynamicEditDialog>
</file>

<file path="Promatis.Net.UI/Components/References/Dialogs/ReferenceEditDialog.razor.cs">
using FluentValidation;
using Microsoft.AspNetCore.Components;
using Microsoft.Extensions.DependencyInjection;
using MudBlazor;
using Promatis.Net.Service;
{
namespace Promatis.Net.UI.Components.References.Dialogs;
{
public partial class ReferenceEditDialog<TEntity> : ComponentBase
    where TEntity : class, new()
    {
        [Inject] protected ISnackbar Snackbar { get; set; } = null!;
        [Inject] protected IServiceProvider ServiceProvider { get; set; } = null!;

        [Parameter] public required TEntity Model { get; set; }
        [Parameter] public bool IsNew { get; set; } // <-- Aô5D 5CD0CT< Dt8D BD´9 bool Cô0D 0CÄ5D$@ C$<CTAD$> C´0

        [Parameter] public string Title { get; set; } = "B 5CD0C=BC,,@Cä2C =C,,5 Ct0C08D 8";

        protected ReferenceEditDialogContext<TEntity> Context { get; set; } = null!;

        protected override void OnInitialized()
        {
            base.OnInitialized();

            // A,,7C$;CT:C 5CÄ 8Cr D´ AD$@Cä3Cä BC,,?C,,7C,,@Cä2C =CÔKCR ACT@C$8D K C MC=5CÔ4C 4C´O CÔ0D,,5C´ AD4ICÔ>
            var referenceService =
                (IReferenceService<TEntity>)ServiceProvider.GetRequiredService(typeof(IReferenceService<TEntity>));
            var validator = (IValidator?)ServiceProvider.GetService(typeof(IValidator<TEntity>));

            // A,,=C,,FC,,0C´8Ct8D CCT< C= >CÔBCT:D B CD8C ;Cä3C GC,,AD$KCÄ8 D AD´;C=0CÄ8D
            Context = new ReferenceEditDialogContext<TEntity>(Model, referenceService, IsNew,
                validator, Snackbar);
        }
    }
</file>

<file path="Promatis.Net.UI/Components/References/Dialogs/ReferenceEditDialogContext.cs">
using FluentValidation;
using MudBlazor;
using Promatis.Net.Service;
using Promatis.Net.UI.Components.Dialogs.Buttons;
{
namespace Promatis.Net.UI.Components.References.Dialogs;
{
public class ReferenceEditDialogContext<TEntity> : DialogActionContextBase<TEntity>
    where TEntity : class, new()
    {
        private readonly IReferenceService<TEntity> _referenceService;
        private readonly bool _isNew;

        // A$=CT4D OCT< D 5D 2C,,A C, DC´0C2 @CT6C,,<C ?D OCÄ> C" 22 :Cä=D$5C=AD" 4C,,0C´>C40, C 5Cr CDt0D BC,,0 Dia
        public ReferenceEditDialogContext(
            TEntity model,
            IReferenceService<TEntity> referenceService,
            bool isNew,
            IValidator? validator,
            ISnackbar snackbar)
            : base(model, validator, onSaveAction: null, snackbar) // Aô5D 5CD0CT< null C" öâ6 fT 7F-öí
    }
}

```

```

        {
            _referenceService = referenceService ?? throw new
ArgumentNullException(nameof(referenceService));
            _isNew = isNew;
        }

        AddAction(new SubmitDialogButton(this));
        AddAction(new CancelDialogButton(this));
    }

    /// <summary>
    /// A05D 5Că?D 5CD5C`OCT< C 0Ct>C$CDă ;Că3C,,:D2 ACăED 0C05C08D0ı
    /// B$5C05D L Ct4CTADÂ 2D`?Că;C00CTBD 0 D BD >C4> CD5D$5D <C,,=C,,@Că2C =C0KC' 8CĂ5C0>C$0C0=D'9 CĂ5D$>CB 1C
    /// </summary>
    public override async Task ExecuteSubmitAsync()
    {
        bool isValid = await ValidateFormAsync();
        if (!isValid) return;

        // A$:C`NDt0CT< D ?C,,=C05D
        NotifyStateChanged();

        try
        {
            // A$KctKC$0CT< Dt5D BC0KCR <CTBCă4D² 1D0:CT=CB ACT@C$8D 0 C00C0@D0<D4N0
            if (_isNew)
                await _referenceService.AddAsync(Model);
            else
                await _referenceService.UpdateAsync(Model);

            CloseSuccess();
        }
        catch (Exception ex)
        {
            // Aă1D 0C >D$:C >D,,8C >C-
            NotifyStateChanged();
            throw; // A 0Ct>C$KC' :C`0D A C05D 5DT2C BC,,B C, ?Că:C 6CTB Snackbar
        }
    }
}
</file>

<file path="Promatis.Net.UI/Components/References/ReferenceGridPage.razor">
@typeparam TEntity where TEntity : class, IDomainObjectHasKey<Guid>, new()
@using Promatis.Net.Domain
{
    <CascadingValue Value="Context" IsFixed="true">
        <WorkspacePage>
            <!-- 1. A$ B %A0/B0 Aă A $ C$BCă<C BC,,GCTAC=8C' ?Că;C,,<Că@DD=D'9 D$CC'1C @ D 5%TB0>C05D 0Dd8D0<C,
            <TopContent>
                <UiToolbar Controls="@Context.Controls" />
            </TopContent>

            <!-- 2. Bd A0"B A',A0 B0 Aă A $ "C 1C'8Dt=C 0 D 5D$:C AC0@C 2CăGC08C=0 -->
            <BodyContent>
                <GridPage TEntity="TEntity"
                    @ref="_grid"
                    ServerDataProvider="@Context.Broker.FetchDataAsync">
                    <GridColumns>
                        <PropertyColumn T="TEntity" TProperty="string" Property="x => ((ReferenceBase)
(object)x).Code" Title="A=CB" †V FW%7G-ÆS0'v-GFf$ S f²" óı
                        <PropertyColumn T="TEntity" TProperty="string" Property="x => ((ReferenceBase)
(object)x).Name" Title="A00C,,<CT=Că2C =C,,5" HeaderStyle="width: auto;" />
                        <PropertyColumn T="TEntity" TProperty="string" Property="x => ((ReferenceBase)
(object)x).Description" Title="Aă?C,,AC =C,,5" HeaderStyle="width: auto;" />
                    </GridColumns>
                </GridPage>
            </BodyContent>
        </WorkspacePage>
    </CascadingValue>
</file>

<file path="Promatis.Net.UI/Components/References/ReferenceGridPage.razor.cs">
using Microsoft.AspNetCore.Components;
using MudBlazor;

```

```

using Promatis.Net.Domain.Interface;
using Promatis.Net.Service;
using Promatis.Net.UI.Components.Workspaces;

namespace Promatis.Net.UI.Components.References;

public partial class ReferenceGridPage<TEntity> : ComponentBase
    where TEntity : class, IDomainObjectHasKey<Guid>, new()
{
    protected GridPage<TEntity>? _grid;

    [Inject] protected IReferenceService<TEntity> ReferenceService { get; set; } = null!;

    [Inject] protected IDialogService DialogService { get; set; } = null!;

    protected ReferenceWorkspaceContext<TEntity> Context { get; set; } = null!;

    protected override void OnInitialized()
    {
        base.OnInitialized();
        Context = new ReferenceWorkspaceContext<TEntity>(ReferenceService, DialogService,
onStateChangedNotifier: RefreshGrid);
    }

    protected override async Task OnAfterRenderAsync(bool firstRender)
    {
        await base.OnAfterRenderAsync(firstRender);

        if (firstRender)
        {
            await Context.LoadInitialDataAsync();
            RefreshGrid();
        }
    }

    protected void RefreshGrid()
    {
        if (_grid == null) return;

        // A, !A A A' A : B BD >C4> Dt5D 5Cr -çföT 7-æ2Â GD$>C K C05D 5D 8D >C$:C BC 1C'8DdK C$AD$0C'0 C"
        // A B AR BCâ3CâÂ :C : Blazor Server Ct0Câ>C0GC,,B D 5C04CT@C,,BDÂ :C 4D >D$:D KC$0DâICT3CâAD0 4C,,0C
        InvokeAsync(async () =>
        {
            if (_grid != null)
            {
                await _grid.ReloadServerData();
            }
        });
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Components/References/ReferenceWorkspaceContext.cs">
using MudBlazor;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Promatis.Net.Service;
using Promatis.Net.UI.Components.References.Dialogs;
using Promatis.Net.UI.Controls;

namespace Promatis.Net.UI.Components.References;

public class ReferenceWorkspaceContext<TEntity> : CrudWorkspaceContext<TEntity, Guid>
    where TEntity : class, IDomainObjectHasKey<Guid>, new()
{
    private readonly IReferenceService<TEntity> _referenceService;
    private readonly IDialogService _dialogService;

    // A$=CT4D OCT< IDialogService C" :Câ=D BD CCâBCâ@ D4=C,,2CT@D 0C'LC0>C4> Câ>C0BCT:D BC
    public ReferenceWorkspaceContext(IReferenceService<TEntity> referenceService, IDialogService
dialogService, Action onStateChangedNotifier)
        : base(referenceService, isInMemoryMode: true)
    {
        _referenceService = referenceService ?? throw new
ArgumentNullException(nameof(referenceService));
    }
}

```



```

        _dialogService = dialogService ?? throw new ArgumentNullException(nameof(dialogService));
    }

    Broker = new UiDataBroker<TEntity, GridState<TEntity>,
GridData<TEntity>>(onStateChangedNotifier);
    Broker.ConfigureInMemoryMode(OzuCache, EvaluateGridStateInMemory);
}

    /// <summary>
    /// A05D 5Cä?D 5CD5C´OCT< C,,=C,,FC,,0C´8Ct0Dd8Dâ :CÔ>Cô>CçÂ GD$>C K D 2Dô7C BDÂ 8DR A CÔ0D$8C$=Cä9 D ;D46C
    /// </summary>
    protected override void InitializeToolBarControls()
    {
        // 1. A0@C,,2Dô7D´2C 5CÄ >D$:D KD$8CR ?D4AD$>C4> CD8C ;Cä3C ACä7CD0C08Dy
        AddControl(new CreateEntityButton<TEntity>().OnExecute(async () =>
            await OpenEditDialogAsync(new TEntity(), isNew: true)));

        // 2. A0@C,,2Dô7D´2C 5CÄ >D$:D KD$8CR 4C,,0C´>C40 D :C´>C08D >C$0C08CT< C$KC @C =CÔ>C´ 2 C4@C,,4CR AD$@
        AddControl(new EditEntityButton<TEntity>().OnExecute(async (selectedRow) =>
        {
            var cloner = new JsonEntityCloner();
            TEntity clone = cloner.CloneEntity(selectedRow);
            await OpenEditDialogAsync(clone, isNew: false);
        })));

        // 3. A0@C,,2Dô7D´2C 5CÄ 2D´7Cä2 MessageBox C, :Cä<C =CDC D44C ;CT=C,,0 C" !B4 AM
        AddControl(new DeleteEntityButton<TEntity>().OnExecute(async (selectedRow) =>
        {
            bool? confirm = await _dialogService.ShowMessageBoxAsync("B44C ;CT=C,,5", $"A$K D42CT@CT=D²Â GD$>
noText: "AäBCÄ5C00");
            if (confirm == true)
            {
                await _referenceService.DeleteAsync(selectedRow.Id);
            }
        })));

        AddControl(new ToolbarDivider());
    }

    /// <summary>
    /// B4=C,,2CT@D 0C´LC0KC´ <CTBCä4 C$Kct>C$0 CÄ>CD0C´LCÔ>C4> Cä:C00 CD;Dò ;Dä1Cä3Cä ACô@C 2CäGC08C=00
    /// </summary>
    private async Task OpenEditDialogAsync(TEntity model, bool isNew)
    {
        var parameters = new DialogParameters<ReferenceEditDialog<TEntity>>
        {
            { x => x.Model, model },
            { x => x.IsNew, isNew }
        };

        var options = new DialogOptions { CloseButton = true, MaxWidth = MaxWidth.Small, FullWidth
= true };

        // A,,!Aô A A´ Aô : B41D 0C0 -çfö¶T 7-æ2â D OCÄ>C´ 2D´7Cä2 ShowAsync CÔ0D$8C$=Cä 1CT7Cä?C ACT= C" &A
        IDialogReference dialog = await _dialogService.ShowAsync<ReferenceEditDialog<TEntity>>(
            isNew ? "B >Ct4C =C,,5 CÔ>C$>C´ 7C ?C,,AC," ¢ $ CT4C :D$8D >C$0C08CR 7C ?C,,AC,"í
            parameters,
            options);

        await dialog.Result;
    }
}
</file>

<file path="Promatis.Net.UI/Components/UIControls/BaseUiControl.cs">
namespace Promatis.Net.UI.Components;
{
    /// <summary>
    /// A 0Ct>C$0Dò 0C AD$@C :Dd8Dò 4C´O C$ACTE Dô;CT<CT=D$>C" CCô@C 2C´5C08Dò ?C´0D$DCä@CÄK (C= Cä?Cä:, Dt5C=1Cä
    /// B 5C ;C,,7D45D" 8C0DD 0D BD CC=BD4@C0CDâ @D4BC,,=D2 8 CÔ@CT4CäAD$0C$;Dô5D" <CäAD" : CD8C00CÄ8Dt5D :Cä<D2 @C
    /// </summary>
    public abstract class BaseUiControl : IUiControl
    {
        private bool _isVisible = true;
        private bool _isEnabled = true;
        private bool _isRunning;
    }
}

```

```

    public abstract string Id { get; }
    public abstract Type ComponentType { get; }
}
    public virtual string? Title { get; init; }
    public virtual string? Icon { get; init; }
    public virtual string? Tooltip { get; init; }
}
    public Dictionary<string, object> ComponentParameters { get; } = new();
}
    public virtual UIControlAlignment Alignment { get; init; } = UIControlAlignment.Left;
}
    public bool IsVisible
    {
        get => _isVisible;
        set { if (_isVisible != value) { _isVisible = value; NotifyStateChanged(); } }
    }
}
    public bool IsEnabled
    {
        get => _isEnabled;
        set { if (_isEnabled != value) { _isEnabled = value; NotifyStateChanged(); } }
    }
}
    public bool IsRunning => _isRunning;
}
    public event Action? OnStateChanged;
}
    protected BaseUiControl()
    {
        ComponentParameters.Add("Control", this);
    }
}
    protected void NotifyStateChanged() => OnStateChanged?.Invoke();
}
    public virtual bool IsEnabledForData(object? targetData) => true;
}
    public async Task TriggerAsync(object? targetData)
    {
        if (!_isEnabled || _isRunning || !IsEnabledForData(targetData)) return;

        try
        {
            _isRunning = true;
            NotifyStateChanged();
            await HandleTriggerAsync(targetData);
        }
        finally
        {
            _isRunning = false;
            NotifyStateChanged();
        }
    }
}
    protected abstract Task HandleTriggerAsync(object? targetData);
}

```

</file>

<file path="Promatis.Net.UI/Components/UIControls/IHasOptions.cs">

namespace Promatis.Net.UI.Components;

/// <summary>

/// A<C>BD 0C=B CD;Dò MC´5CÄ5C0BCä2 D4?D 0C\$;CT=C,,0, Cä1C´0CD0DäIC,,E D ?C,,AC=>CÄ 4CäAD\$CCô=D´E Cä?Dd8C´ 2D´1

/// </summary>

public interface IHasOptions : IUiControl

{

IEnumerable<string> Options { get; }

}

</file>

<file path="Promatis.Net.UI/Components/UIControls/IHasValue.cs">

namespace Promatis.Net.UI.Components;

/// <summary>

/// A<C>BD 0C=B CD;Dò MC´5CÄ5C0BCä2 D4?D 0C\$;CT=C,,0, C=>D\$>D KCR ED 0C00D" 8 C,,7CÄ5C00DäB D 0C0BC 9CÄ07C00Dt

/// </summary>

public interface IHasValue : IUiControl

```

{
    object? Value { get; set; }
}
</file>

<file path="Promatis.Net.UI/Components/UIControls/IUiControl.cs">
namespace Promatis.Net.UI.Components;
{
    /// <summary>
    /// B4=C,,2CT@D 0C'LC0KC' :Cä=D$@C :D" MC'5CÄ5C0BC CC0@C 2C'5C08D0 „:C0>C0:C Ä GCT:C >C=A, C$KC00CD0DäIC,,9 D
    /// </summary>
    public interface IUiControl
    {
        string Id { get; }
        Type ComponentType { get; }
        Dictionary<string, object> ComponentParameters { get; }

        string? Title { get; }
        string? Icon { get; }
        string? Tooltip { get; }

        bool IsVisible { get; set; }
        bool IsEnabled { get; set; }
        bool IsRunning { get; }

        UIControlAlignment Alignment { get; }

        bool IsEnabledForData(object? targetData);
        Task TriggerAsync(object? targetData);

        event Action? OnStateChanged;
    }
}
</file>

<file path="Promatis.Net.UI/Components/UIControls/UiControlAlignment.cs">
namespace Promatis.Net.UI.Components;
{
    public enum UIControlAlignment
    {
        Left,
        Right
    }
}
</file>

<file path="Promatis.Net.UI/Components/Workspaces/GridPage.razor">
@typeparam TEntity where TEntity : class
{
    <MudDataGrid @ref="_mudGrid"
        ServerData="@ServerDataProvider"
        T="TEntity"
        SelectedItem="@SelectedRow"
        SelectedItemChanged="@OnSelectedRowChanged"
        RowStyleFunc="@FormatSelectedRowStyle"
        RowsPerPage="@RowsPerPage"
        SelectOnRowClick="true"
        Hover="true"
        Dense="true"
        Bordered="true"
        Elevation="0"
        FixedHeader="true"
        Height="calc(100vh - 280px)"
        Class="flex-grow-1 w-100 cursor-pointer">
        <Columns>
            @GridColumns
        </Columns>
        @* AS:C'NDt0CT< C00C48C00D$>D BCä;DÄ:Cä ?Cä BD 5C >C$0C08Dä :Cä=CTGC0>C' DCä@CÄK *@
        <PagerContent>
            @if (WithPagination)
            {
                <MudDataGridPager T="TEntity" InfoFormat="{first_item}-{last_item} C,,7 {all_items}"
                RowsPerPageString="B BD >C£¢" óí
            }
        </PagerContent>
    </MudDataGrid>
}
</file>

```

```

<file path="Promatis.Net.UI/Components/Workspaces/GridPage.razor.cs">
using Microsoft.AspNetCore.Components;
using MudBlazor;
@
namespace Promatis.Net.UI.Components.Workspaces;
@
public partial class GridPage<TEntity> : ComponentBase where TEntity : class
{
    private MudDataGrid<TEntity>? _mudGrid;

    [CascadingParameter]
    protected IWorkspaceActionContext? ActionContext { get; set; }

    /// <summary>
    /// BD>C0>C$>CR ACT@C$5D =Cä5 D >C KD$8CR 7C 3D Cct:C, 4C =C0KDR „<C ?C„BD 0 C00 MudBlazor ServerData).
    /// </summary>
    [Parameter]
    public Func<GridState<TEntity>, CancellationTokens, Task<GridData<TEntity>>>? ServerDataProvider { get; set; }

    [Parameter]
    public RenderFragment? GridColumns { get; set; }

    [Parameter]
    public bool WithPagination { get; set; } = false;

    [Parameter]
    public int RowsPerPage { get; set; } = 10000;

    protected TEntity? SelectedRow { get; set; }

    /// <summary>
    /// AöCC ;C„GCÖKC' <CTBCä4, Cö>Ct2Cä;DöND"8C' AD$@C =C„FC <-C$;C 4CT;DÄFC < Cö@C„=D44C„BCT;DÄ=Cä >C =Cä2C
    /// </summary>
    public Task ReloadServerData()
    {
        return _mudGrid != null ? _mudGrid.ReloadServerData() : Task.CompletedTask;
    }

    protected string FormatSelectedRowStyle(TEntity item, int rowNumber)
        => item == SelectedRow ? "background-color: var(--mud-palette-action-disabled-background);
font-weight: 500;" : string.Empty;

    protected void OnSelectedRowChanged(TEntity? newSelection)
    {
        if (newSelection == null && SelectedRow != null) return;

        SelectedRow = newSelection;

        if (ActionContext is IHasSelectedData<TEntity> bindableContext)
        {
            bindableContext.SelectedData = newSelection;

            // A„!Aö A A' Aö : At0C$>D 0Dt8C$0CT< D42CT4Cä<C'5C08CR >C >C =Cä2C'5C08C, :Cä=D$5C=AD$0 DT>C'A
            // BÖBCâ @C 7C$>CD8D" 2Cä 2D 5CÄ5C08 D 5C04CT@C„=C2 BC 1C'8DdK C, 7C ?D4AC¢ :Cä<C =CB :C0>C0>C¢04
            // C0>C' =CäAD$LDä ;C„:C$8CD8D CDò :Cä=DD;C„:D" &VæFW%G&VTF-fd'V-ÆFW"
            InvokeAsync(() =>
            {
                bindableContext.OnContextUpdated?.Invoke();
            });
        }
    }
}
</file>

<file path="Promatis.Net.UI/Components/Workspaces/TreePage.razor">
@typeparam TEntity where TEntity : class
@
@
<MudTreeView T="TEntity">
    Items="@TreeItems"
    SelectedValue="@SelectedNode"
    SelectedValueChanged="@OnSelectedNodeChanged"
    Hover="true"
    Dense="true"

```

```

        SelectionMode="SelectionMode.SingleSelection"@
        Height="calc(100vh - 240px)"@
        Class="flex-grow-1 w-100 h-100 overflow-y-auto">@
<ItemTemplate>@
    <MudTreeViewItem T="TEntity"@
        Value="@context.Value"@
        Icon="@GetNodeIcon(context.Value)"@
        Text="@GetNodeText(context.Value)"@
        Items="@GetChildTreeItemData(context.Value)" />@
</ItemTemplate>@
</MudTreeView>
</file>

<file path="Promatis.Net.UI/Components/Workspaces/TreePage.razor.cs">
using Microsoft.AspNetCore.Components;@
using MudBlazor;@
@
namespace Promatis.Net.UI.Components.Workspaces;@
@
public partial class TreePage<TEntity> : ComponentBase where TEntity : class@
{
    [CascadingParameter]@
    protected IWorkspaceActionContext? ActionContext { get; set; }@
@
    [Parameter] public IEnumerable<TEntity>? RootItems { get; set; }@
    [Parameter] public Func<TEntity, IEnumerable<TEntity>>? ChildSelector { get; set; }@
    [Parameter] public Func<TEntity, string>? TextSelector { get; set; }@
    [Parameter] public Func<TEntity, string>? IconSelector { get; set; }@
@
    // A,ACô>C´LctCCT< Dt8D BD´9 TEntity, D$0C¢ :C : Că3D 0C08Dt5C08CR 6Æ 72 CCd5 CD>CôCD :C 5D" çVÆÂ =C CD
    protected List<TreeItemData<TEntity>> TreeItems { get; set; } = new();@
    protected TEntity? SelectedNode { get; set; }@
@
    protected override void OnInitialized()@
    {
        base.OnInitialized();@
@
        if (ActionContext is IHasSelectedData<TEntity> bindableContext)@
        {
            SelectedNode = bindableContext.SelectedData;@
        }@
@
    }@
@
    protected override void OnParametersSet()@
    {
        base.OnParametersSet();@
@
        if (RootItems != null)@
        {
            TreeItems = RootItems@
                .Where(item => item != null)@
                .Select(item => new TreeItemData<TEntity> { Value = item })@
                .ToList();@
        }@
        else@
        {
            TreeItems = new();@
        }@
@
    }@
@
    protected void OnSelectedNodeChanged(TEntity? newSelection)@
    {
        SelectedNode = newSelection;@
@
        if (ActionContext is IHasSelectedData<TEntity> bindableContext)@
        {
            bindableContext.SelectedData = newSelection;@
            bindableContext.OnContextUpdated?.Invoke();@
        }@
@
    }@
@
    // AD>C 0C$;Dô5CÂ ?D >C$5D :D2 =C çVÆÂÂ BC : C¢0C¢ @C =D$0C" <-Că1D¢5C¢B C" 6öçFW†Băf çVR <Că6CTB CăBD CD
    protected string GetNodeText(TEntity? item) =>@
        item == null ? string.Empty : (TextSelector?.Invoke(item) ?? item.ToString()) ??
string.Empty);@
@
}

```

```

        protected string GetNodeIcon(TEntity? item) =>
        {
            item == null ? Icons.Material.Filled.Folder : (IconSelector?.Invoke(item) ??
Icons.Material.Filled.Folder);
        }

        protected List<TreeItemData<TEntity>> GetChildTreeItemData(TEntity? item)
        {
            if (item == null || ChildSelector == null) return new();

            IEnumerable<TEntity>? children = ChildSelector.Invoke(item);
            if (children == null) return new();

            return children
                .Where(child => child != null)
                .Select(child => new TreeItemData<TEntity> { Value = child })
                .ToList();
        }
    }
}
</file>

<file path="Promatis.Net.UI/Components/Workspaces/UiToolbar.razor">
<div class="d-flex align-center justify-space-between gap-4 pa-0 w-100" style="min-height: 48px;
width: 100%;">
    @if (Controls != null)
    {
        <div class="d-flex align-center gap-4">
            @foreach (IUiControl control in Controls.Where(c => c.Alignment ==
UiControlAlignment.Left))
            {
                @if (!control.IsVisible) continue;
                <DynamicComponent Type="@control.ComponentType"
Parameters="@control.ComponentParameters" />
            }
        </div>

        @* A0@C 2C 0 C4@D4?C00: D 0Ct4CT;C,,BCT;DÂ 8 C=Cä?C=8 C$KC4@D47C=8 *@
        <div class="d-flex align-center gap-4">
            @foreach (IUiControl control in Controls.Where(c => c.Alignment ==
UiControlAlignment.Right))
            {
                @if (!control.IsVisible) continue;
                <DynamicComponent Type="@control.ComponentType"
Parameters="@control.ComponentParameters" />
            }
        </div>
    }
</div>
</file>

<file path="Promatis.Net.UI/Components/Workspaces/UiToolbar.razor.cs">
using System.Reflection;
using Microsoft.AspNetCore.Components;
namespace Promatis.Net.UI.Components.Workspaces;
public partial class UiToolbar : ComponentBase, IDisposable
{
    /// <summary>
    /// B ?C,,ACä: C0>C'8CÄ>D DC0KDR :Cä=D$@Cä;Cä2 CD;D0 >D$@C,,ACä2C=8 C00 C00C05C'8.D
    /// </summary>
    [Parameter]
    public IEnumerable<IUiControl>? Controls { get; set; }

    /// <summary>
    /// A00D :C 4C0KC' :Cä=D$5C=AD" ECä;D BC 4C'0 CäBD ;CT6C,,2C =C,,0 D <CT=D² 2D'4CT;CT=C,,0 CD0C0=D'E.D
    /// </summary>
    [CascadingParameter]
    protected IWorkspaceActionContext? ActionContext { get; set; }

    /// <summary>
    /// A,,7C$;CT:C 5D" BCT:D4IC,,9 C$KCD5C'5C0=D'9 Cä1D=5C=B CD;D0 ?CT@CT4C GC, 2 C 8Ct=CTA-C'>C48C=C C= >C0BD
    /// </summary>
    protected object? CurrentSelectedData => GetSelectedDataFromContext();

    protected override void OnInitialized()

```

```

    {
        base.OnInitialized();
        SubscribeToSelectionChanged(true);
        if (Controls != null)
        {
            foreach (IUIControl control in Controls)
            {
                control.OnStateChanged += HandleStateChanged;
            }
        }
        private void HandleStateChanged() => InvokeAsync(StateHasChanged);
        private object? GetSelectedDataFromContext()
        {
            if (ActionContext == null) return null;
            Type? interfaceType = ActionContext.GetType().GetInterface("IHasSelectedData`1");
            if (interfaceType != null)
            {
                return interfaceType.GetProperty("SelectedData")?.GetValue(ActionContext);
            }
            return null;
        }
        private void SubscribeToSelectionChanged(bool subscribe)
        {
            if (ActionContext == null) return;
            Type? interfaceType = ActionContext.GetType().GetInterface("IHasSelectedData`1");
            if (interfaceType != null)
            {
                PropertyInfo? eventInfo = interfaceType.GetProperty("OnContextUpdated");
                if (eventInfo != null)
                {
                    var currentDelegate = (Action?)eventInfo.GetValue(ActionContext);
                    if (subscribe)
                        currentDelegate += HandleStateChanged;
                    else
                        currentDelegate -= HandleStateChanged;
                    eventInfo.SetValue(ActionContext, currentDelegate);
                }
            }
        }
        public void Dispose()
        {
            SubscribeToSelectionChanged(false);
            if (Controls != null)
            {
                foreach (IUIControl control in Controls)
                {
                    control.OnStateChanged -= HandleStateChanged;
                }
            }
        }
    }
</file>

<file path="Promatis.Net.UI/Components/Workspaces/WorkspacePage.razor">
<MudPaper Elevation="@GetPaperElevation()"
    Class="@GetPaperClass()"
    Style="@($"width: 100%; height: {GetWorkspaceHeight()}; overflow: hidden;")">
    @* A,,!A A A' A : min-height Ct0CÄ5C05C00 C00 height *@
    <!-- 1. A$ B %A0/B0 Aä A ,, A Aä A0/AT"B / B$ A',A A A0 A, A A,, 'A,, TOPCONTENT) -->
    @if (TopContent != null && !IsTopCollapsed())
    {
        <div class="workspace-zone-top w-100 mb-2"
            style="@($"height: {GetTopHeight()}; min-height: {GetTopHeight()}; max-height: {GetTopHeight()}; flex-shrink: 0;")">

```

```

        @TopContent
    </div>
}
}
<!-- B AT A' !A' A' " AT A / At A + A AD + A A / At A ) -->
<div class="d-flex flex-row align-stretch flex-grow-1 gap-4 w-100" style="min-height: 0; height: 100%; overflow: hidden;">
    <!-- 2. A' A$ B A A " A A A &A" / / AD B A$ ) -->
    @if (LeftContent != null && !IsLeftCollapsed())
    {
        @* A"!A A A' A : AD>C 0C$;CT= D :D >C'; overflow-y: auto CD;D 7C IC,BD² >D" ?CT@CT?Cä;C05C08D
        <div class="workspace-zone-left d-flex flex-column align-stretch justify-space-between"
            style="@($width: {GetLeftWidth()}; min-width: {GetLeftWidth()}; max-width: {GetLeftWidth()}; overflow-y: auto;)">
            @LeftContent
        </div>
    }
    <!-- 3. Bd A"B A',A B A A " A A f B AB ò A A B %AT A ' 00í
    @* A"!A A A' A : B41D 0C0 ,Ó A 4Cä1C 2C'5C0K C;C AD K d-flex flex-column CD;D :C AC=0CD=Cä9 C0
    <div class="workspace-zone-body flex-grow-1 h-100" style="min-width: 0; min-height: 0;">
        @if (BodyContent != null)
        {
            @BodyContent
        }
    </div>
    <!-- 4. A A A / At A (A, B AT B$ B !A$ A"!B$ ) -->
    @if (RightContent != null && !IsRightCollapsed())
    {
        @* A"!A A A' A : AD>C 0C$;CT= D :D >C'; overflow-y: auto CD;D 7C IC,BD² >D" ?CT@CT?Cä;C05C08D
        <div class="workspace-zone-right d-flex flex-column align-stretch justify-space-between"
            style="@($width: {GetRightWidth()}; min-width: {GetRightWidth()}; max-width: {GetRightWidth()}; overflow-y: auto;)">
            @RightContent
        </div>
    }
}
</div>
}
<!-- 5. A Ad B/ At A (B "A "B4!-A B ' 00í
@if (BottomContent != null && !IsBottomCollapsed())
{
    <div class="workspace-zone-bottom w-100 mt-2"
        style="@($height: {GetBottomHeight()}; min-height: {GetBottomHeight()}; max-height: {GetBottomHeight()}; flex-shrink: 0;)">
        @BottomContent
    </div>
}
}
</MudPaper>
</file>

```

```

<file path="Promatis.Net.UI/Components/Workspaces/WorkspacePage.razor.cs">
using Microsoft.AspNetCore.Components;
namespace Promatis.Net.UI.Components.Workspaces;
public partial class WorkspacePage : ComponentBase, IDisposable
{
    /// <summary>
    /// A 0D :C 4C0KC' 8C0BCT@DD5C"A C>C0BCT:D BC BCT:D4ICT9 D 0C >Dt5C' >C ;C AD$8.D
    /// </summary>
    [CascadingParameter]
    protected IWorkspaceActionContext? ActionContext { get; set; }

    [Parameter] public RenderFragment? BodyContent { get; set; }
    [Parameter] public RenderFragment? TopContent { get; set; }
    [Parameter] public RenderFragment? BottomContent { get; set; }
    [Parameter] public RenderFragment? LeftContent { get; set; }
    [Parameter] public RenderFragment? RightContent { get; set; }

    protected override void OnInitialized()
    {
        base.OnInitialized();
    }
}

```



```

    // A0>CD?C,,AC=0 C00 D 5C :D$8C$=Cä5 Cä1C0>C$;CT=C,,5 C00D 0CÄ5D$@Cä2 C45Cä<CTBD 8C•
    if (ActionContext != null)
    {
        ActionContext.OnContextStateChanged += HandleContextStateChanged;
    }
}

private void HandleContextStateChanged()
{
    InvokeAsync(StateHasChanged);
}

public void Dispose()
{
    if (ActionContext != null)
    {
        ActionContext.OnContextStateChanged -= HandleContextStateChanged;
    }
}

// --- AÄ B$ AD+ B B 'AT"A A,, B4 A',A0 A4 B "A,, B0 Aä A' Ad A, 00Ý
protected int GetPaperElevation() => ActionContext?.PaperElevation ?? 1;
protected string GetPaperClass() => ActionContext?.PaperClass ?? "pa-4 d-flex flex-column flex-grow-1 w-100";

// --- AÄ B$ AD+ B B 'AT"A AT AÄ B$ A,, A= B A !A %Aä B "A 00Ý
protected string GetWorkspaceHeight() => ActionContext?.WorkspaceHeight ?? "100%";
protected string GetTopHeight() => ActionContext?.TopZoneHeight ?? "auto";
protected string GetBottomHeight() => ActionContext?.BottomZoneHeight ?? "auto";
protected string GetLeftWidth() => ActionContext?.LeftZoneWidth ?? "250px";
protected string GetRightWidth() => ActionContext?.RightZoneWidth ?? "300px";

// ATAC'8 C= >C0BCT=D" AR ?CT@CT4C =, Ct>C00 B AT A0#B$ C 2D$>CÄ0D$8Dt5D :C,â
// ATAC'8 C= >C0BCT=D" 5D BDÄÄ 5E ACäAD$>D0=C,,5 C 5D 5D$AD0 8Cr :Cä=D$5C=AD$0 (C0> D4<Cä;Dt0C08Dä f Ç6R 0
protected bool IsTopCollapsed() => TopContent == null || (ActionContext?.IsTopZoneCollapsed ??
false);
protected bool IsBottomCollapsed() => BottomContent == null ||
(ActionContext?.IsBottomZoneCollapsed ?? false);
protected bool IsLeftCollapsed() => LeftContent == null ||
(ActionContext?.IsLeftZoneCollapsed ?? false);
protected bool IsRightCollapsed() => RightContent == null ||
(ActionContext?.IsRightZoneCollapsed ?? false);
}
</file>

<file path="Promatis.Net.UI/Controls/CreateEntityButton.cs">
using MudBlazor;
using Promatis.Net.UI.Components;
using Promatis.Net.UI.Components.ElementRenderBase;
namespace Promatis.Net.UI.Controls;
/// <summary>
/// B4=C,,2CT@D 0C'LC0KC' 8C0DD 0D BD CC=BD4@C0KC' MC'5CÄ5C0B C= Cä?C=8 "B >Ct4C BDÄ" 4C'0 D CD"=CäAD$5C' ;Dä1
/// </summary>
public class CreateEntityButton<TEntity> : BaseUiControl where TEntity : class, new()
{
    private Func<Task> _command; // A00D, 8D ?Cä;C08D$5C'L CD8C ;Cä3C

    public override string Id => $"crud_create_{typeof(TEntity).Name.ToLower()}";
    public override Type ComponentType => typeof(ButtonRenderBase);
    public override string Title => "B >Ct4C BDÄ#%
    public override string Icon => Icons.Material.Filled.Add;
    public override string Tooltip => $"B >Ct4C BDÄ =Cä2D4N Ct0C08D L";

    public CreateEntityButton() { IsEnabled = true; }

    // AÄ5D$>CB 4C'0 C$=CT4D 5C08D0 @CT0C'LC0>C4> CD5C"AD$2C,,0 C,,7 C= >C0BCT:D BC
    public CreateEntityButton<TEntity> OnExecute(Func<Task> command)
    {
        _command = command;
        return this;
    }

    public override bool IsEnabledForData(object? targetData) => true;

```

```

    protected override async Task HandleTriggerAsync(object? targetData)
    {
        if (_command != null) await _command.Invoke();
    }
}
</file>

<file path="Promatis.Net.UI/Controls/DeleteEntityButton.cs">
using MudBlazor;
using Promatis.Net.UI.Components;
using Promatis.Net.UI.Components.ElementRenderBase;

namespace Promatis.Net.UI.Controls;

/// <summary>
/// B4=C,,2CT@D 0C'LC0KC' 8C0DD 0D BD CC=BD4@C0KC' MC'5CÄ5C0B C=Cä?C=8 "B44C ;C,,BDÄ" 4C'0 C$KC @C =C0>C' AD4I
/// </summary>
public class DeleteEntityButton<TEntity> : BaseUiControl where TEntity : class
{
    private Func<TEntity, Task>? _command;

    public override string Id => $"crud_delete_{typeof(TEntity).Name.ToLower()}";
    public override Type ComponentType => typeof(ButtonRenderBase);
    public override string Title => "B44C ;C,,BDÄ#½
    public override string Icon => Icons.Material.Filled.Delete;
    public override string Tooltip => "B44C ;C,,BDÄ 2D'1D 0C0=D4N Ct0C08D L";

    public DeleteEntityButton<TEntity> OnExecute(Func<TEntity, Task> command)
    {
        _command = command;
        return this;
    }

    public override bool IsEnabledForData(object? targetData)
    {
        if (targetData is not TEntity typedEntity) return false;
        if (typedEntity is Domain.Interface.ISoftDeletable softDeletable)
        {
            return softDeletable.DeletedAt == null;
        }
        return true;
    }

    protected override async Task HandleTriggerAsync(object? targetData)
    {
        if (_command != null && targetData is TEntity typedEntity)
        {
            await _command.Invoke(typedEntity);
        }
    }
}
</file>

<file path="Promatis.Net.UI/Controls/EditEntityButton.cs">
using MudBlazor;
using Promatis.Net.UI.Components;
using Promatis.Net.UI.Components.ElementRenderBase;

namespace Promatis.Net.UI.Controls;

/// <summary>
/// B4=C,,2CT@D 0C'LC0KC' 8C0DD 0D BD CC=BD4@C0KC' MC'5CÄ5C0B C=Cä?C=8 "B 5CD0C=BC,,@Cä2C BDÄ" 4C'0 C$KC @C =C
/// </summary>
public class EditEntityButton<TEntity> : BaseUiControl where TEntity : class
{
    private Func<TEntity, Task>? _command;

    public override string Id => $"crud_edit_{typeof(TEntity).Name.ToLower()}";
    public override Type ComponentType => typeof(ButtonRenderBase);
    public override string Title => "B 5CD0C=BC,,@Cä2C BDÄ#½
    public override string Icon => Icons.Material.Filled.Edit;
    public override string Tooltip => "B 5CD0C=BC,,@Cä2C BDÄ 2D'1D 0C0=D4N Ct0C08D L";

    public EditEntityButton<TEntity> OnExecute(Func<TEntity, Task> command)
    {

```

```

        _command = command;
        return this;
    }
}

public override bool IsEnabledForData(object? targetData) => targetData is TEntity;

protected override async Task HandleTriggerAsync(object? targetData)
{
    if (_command != null && targetData is TEntity typedEntity)
    {
        await _command.Invoke(typedEntity);
    }
}
}
</file>

<file path="Promatis.Net.UI/Controls/ToolbarDivider.cs">
using Promatis.Net.UI.Components;
using Promatis.Net.UI.Components.ElementRenderBase;
namespace Promatis.Net.UI.Controls;
/// <summary>
/// B 8D BCT<CÖKC' MC'5CÄ5CÖB C$5D BC,,:C ;DÄ=Cä3Cä @C 7CD5C'8D$5C'O, D 0Ct<CTIC 5CÄKC' <CT6CDC DÖ;CT<CT=D$0CÄ
/// </summary>
public class ToolbarDivider : BaseUiControl
{
    private readonly string _id = "core_toolbar_divider_" + Guid.NewGuid().ToString("N");

    public override string Id => _id;

    /// <summary>
    /// Aö@C,,2Dö7C□0 Cφ =C HCT<D2 7C DC,,:D 8D >C$0CÖ=Cä<D2 1C 7Cä2Cä<D2 @CT=CD5D 5D C DividerRenderBase.D
    /// </summary>
    public override Type ComponentType => typeof(DividerRenderBase);

    protected override Task HandleTriggerAsync(object? targetData)
    {
        // AD5C□>D 0D$8C$=D'9 DÖ;CT<CT=D" ?C =CT;C, 8CÖAD$@D4<CT=D$>C" =CR 2D' ?Cä;CÖ0CTB C'>C48C□C Cö> D$@C,,3
        return Task.CompletedTask;
    }
}
</file>

<file path="Promatis.Net.UI/Data/Cache/FlatOzuMutationStrategy.cs">
using Promatis.Net.Data;
namespace Promatis.Net.UI;
public class FlatOzuMutationStrategy<TEntity> : IOzuMutationStrategy<TEntity> where TEntity : class
{
    public void ApplyDelta(List<TEntity> inMemoryItems, EntityStateChangeEnum state, TEntity
entity, Func<TEntity, object?> idSelector)
    {
        object? entityId = idSelector(entity);
        if (entityId == null) return;

        switch (state)
        {
            case EntityStateChangeEnum.Added:
                if (!inMemoryItems.Any(x => Equals(idSelector(x), entityId)))
                {
                    inMemoryItems.Add(entity);
                }
                break;

            case EntityStateChangeEnum.Modified:
                int index = inMemoryItems.FindIndex(x => Equals(idSelector(x), entityId));
                if (index >= 0) inMemoryItems[index] = entity;
                break;

            case EntityStateChangeEnum.Deleted:
            case EntityStateChangeEnum.SoftDeleted:
                TEntity? itemToRemove = inMemoryItems.FirstOrDefault(x => Equals(idSelector(x),
entityId));
                if (itemToRemove != null) inMemoryItems.Remove(itemToRemove);
        }
    }
}

```

```

        break;
    }
}
</file>

<file path="Promatis.Net.UI/Data/Cache/HierarchicalOzuMutationStrategy.cs">
using Promatis.Net.Data;
{
namespace Promatis.Net.UI;
{
public class HierarchicalOzuMutationStrategy<TEntity> : IOzuMutationStrategy<TEntity> where
TEntity : class
{
    private readonly Func<TEntity, object?> _parentIdSelector;
    private readonly Func<TEntity, List<TEntity>> _childrenSelector;

    public HierarchicalOzuMutationStrategy(Func<TEntity, object?> parentIdSelector, Func<TEntity,
List<TEntity>> childrenSelector)
    {
        _parentIdSelector = parentIdSelector ?? throw new
ArgumentNullException(nameof(parentIdSelector));
        _childrenSelector = childrenSelector ?? throw new
ArgumentNullException(nameof(childrenSelector));
    }

    public void ApplyDelta(List<TEntity> inMemoryItems, EntityStateChangeEnum state, TEntity
entity, Func<TEntity, object?> idSelector)
    {
        object? entityId = idSelector(entity);
        if (entityId == null) return;

        // ATAC`8 Că1D=5C= B C= D =CT2Că9 (ParentId == null), Că1D 0C 0D$KC$0CT< CT3Că =C 2CT@DT=CT< D4@Că2C0
        object? parentId = _parentIdSelector(entity);
        if (parentId == null)
        {
            ProcessNodeInList(inMemoryItems, state, entity, entityId, idSelector);
            return;
        }

        // ATAC`8 D2 >C JCT:D$0 CTAD$ L D >CD8D$5C`L, C,,ICT< D0BCă3Că @Că4C,,BCT;D0 ?Că 2D 5CĂC CD5D 5C$C D 5C
        TEntity? parentNode = FindNodeRecursive(inMemoryItems, parentId, idSelector);
        if (parentNode != null)
        {
            List<TEntity> siblings = _childrenSelector(parentNode) ?? new List<TEntity>();
            ProcessNodeInList(siblings, state, entity, entityId, idSelector);
        }
    }

    private void ProcessNodeInList(List<TEntity> list, EntityStateChangeEnum state, TEntity entity,
object entityId, Func<TEntity, object?> idSelector)
    {
        switch (state)
        {
            case EntityStateChangeEnum.Added:
                if (!list.Any(x => Equals(idSelector(x), entityId))) list.Add(entity);
                break;

            case EntityStateChangeEnum.Modified:
                int idx = list.FindIndex(x => Equals(idSelector(x), entityId));
                if (idx >= 0) list[idx] = entity;
                break;

            case EntityStateChangeEnum.Deleted:
            case EntityStateChangeEnum.SoftDeleted:
                TEntity? toRemove = list.FirstOrDefault(x => Equals(idSelector(x), entityId));
                if (toRemove != null) list.Remove(toRemove);
                break;
        }
    }

    private TEntity? FindNodeRecursive(List<TEntity> nodes, object targetId, Func<TEntity, object?>
idSelector)
    {
        foreach (TEntity node in nodes)
        {

```

```

        if (Equals(idSelector(node), targetId)) return node;
    }
    List<TEntity> children = _childrenSelector(node);
    if (children != null && children.Any())
    {
        TEntity? found = FindNodeRecursive(children, targetId, idSelector);
        if (found != null) return found;
    }
    }
    return null;
}
}
</file>

<file path="Promatis.Net.UI/Data/Cache/IOzuMutationStrategy.cs">
using Promatis.Net.Data;
{
namespace Promatis.Net.UI;
{
public interface IOzuMutationStrategy<TEntity> where TEntity : class
{
    void ApplyDelta(List<TEntity> inMemoryItems, EntityStateChangeEnum state, TEntity entity,
Func<TEntity, object?> idSelector);
}
</file>

<file path="Promatis.Net.UI/Data/Cache/IUiOzuCache.cs">
using Promatis.Net.Data;
{
namespace Promatis.Net.UI;
{
/// <summary>
/// A C0BD 0C B C,,7Cä;C,,@Cä2C =C0>C4> C'>C0C'LC0>C4> Aä B20ED 0C08C'8D"0 CD0C0=D'E C0>C0:D 5D$=Cä9 D0:D 0C0
/// A,,AC0>C'LCtCCTB D 8D BCT<C0KC' VçF-G*7F FT6† ævTVçV0 4C'0 C0@C,,<CT=CT=C,,0 D$>Dt5Dt=D'E CD5C'LD" 8Ct<CT=CT
/// </summary>
/// <typeparam name="TEntity">B$8C0 4Cä<CT=C0>C' AD4IC0>D BC,äÄ÷G- W & Óí
public interface IUiOzuCache<TEntity> where TEntity : class
{
    /// <summary>
    /// A0@D0<Cä9 CD>D BD4? Cç BCT:D4ICT<D2 <C AD 8C$C CD0C0=D'E, DT@C =D0ICT<D4AD0 2 Cä?CT@C BC,,2C0>C' ?C <D
    /// </summary>
    List<TEntity> InMemoryItems { get; set; }
}
{
    /// <summary>
    /// A 2D$>C0>Cä=D'9 Aä B204C$8Cd>Cç <D4BC FC,,9. BT8D CD 3C,,GCTAC08 D$>Dt=Cä >C =Cä2C'OCTB C0>C';CT:Dd8Dä
    /// C0@C,,=C,,<C 0 D 8D BCT<C0KC' AD$5C"B D$@C =Ct0C0FC,,8 B #A .D
    /// </summary>
    void ApplyOzuDelta(EntityStateChangeEnum state, TEntity entity);
}
</file>

<file path="Promatis.Net.UI/Data/Cache/UiOzuCache.cs">
using Promatis.Net.Data;
using Promatis.Net.Domain.Interface;
{
namespace Promatis.Net.UI;
{
/// <summary>
/// B 5C ;C,,7C FC,,0 C,,7Cä;C,,@Cä2C =C0>C4> C0MD,,0 C" >C05D 0D$8C$=Cä9 C00CÄ0D$8 CD;D0 :Cä=C0@CTBC0>C4> C,,=D BC
/// </summary>
public class UiOzuCache<TEntity> : IUiOzuCache<TEntity> where TEntity : class
{
    private IOzuMutationStrategy<TEntity> _mutationStrategy = new
FlatOzuMutationStrategy<TEntity>();
{
    public List<TEntity> InMemoryItems { get; set; } = new();
}
{
    public UiOzuCache() { }
}
{
    public UiOzuCache(List<TEntity> initialItems)
{
    InMemoryItems = initialItems ?? throw new ArgumentNullException(nameof(initialItems));
}
}
{
    // AÄ5D$>CB 4C'0 CD8C00CÄ8Dt5D :Cä9 D <CT=D² AD$@C BCT3C,,8 (C$KctKC$0CTBD 0 C0>C0BCT:D BCä< CD5D 5C$0)
    public void SetMutationStrategy(IOzuMutationStrategy<TEntity> strategy)

```

```

        {
            _mutationStrategy = strategy ?? throw new ArgumentNullException(nameof(strategy));
        }

        public void ApplyOzuDelta(EntityStateChangeEnum state, TEntity entity)
        {
            // AD5C'5C48D CCT< D 0C >D$C C$KC @C =C0>C' AD$@C BCT3C,,8 CÄCD$0Dd8C•
            _mutationStrategy.ApplyDelta(InMemoryItems, state, entity, GetEntityId);
        }

        private object? GetEntityId(TEntity item)
        {
            Type? hasKeyInterface = item.GetType()
                .GetInterfaces()
                .FirstOrDefault(i => i.IsGenericType && i.GetGenericTypeDefinition() ==
typeof(IDomainObjectHasKey<>));

            if (hasKeyInterface != null)
            {
                return hasKeyInterface.GetProperty("Id")?.GetValue(item);
            }

            return item.GetType().GetProperty("Id")?.GetValue(item);
        }
    }
}
</file>

<file path="Promatis.Net.UI/Data/IUiDataBroker.cs">
using Promatis.Net.Data;
namespace Promatis.Net.UI;

/// <summary>
/// A>C0BD 0C=B C @Cä:CT@C 4C =C0KDRÄ CC0@C 2C'0DäICT3Cä 2D'1Cä@Cä< C,,AD$>Dt=C,,:C ...6W'fW"ö0¥R' 8 D BD 0D$5
/// A05 D >CD5D 6C,,B CD5DD>C'BC0KDR @CT6C,,<Cä2 - C0@Cä3D 0CÄ<C,,AD" >C 0Ct0C0 0C$=Cä AC>C0DC,,3D4@C,,@Cä2C BDÄ
/// </summary>
public interface IUiDataBroker<TEntity, TQueryState, TResultData> where TEntity : class
{
    /// <summary>
    /// A0@C,,7C00C#Ä CC=0CtKC$0DäIC,,9, DtBCä BCT:D4IC,,9 C,,=D BC =D 1D >C=5D 0 C00 DD>D <CR @C 1CäBC 5D" 2 D
    /// </summary>
    bool IsInMemoryMode { get; }

    /// <summary>
    /// Ad5D BC=0D0 :Cä=DD8C4CD 0Dd8D0 @C 1CäBD² =C ?D OCÄCDä A B #A / gRPC API (Server Mode).
    /// </summary>
    void ConfigureServerMode(Func<TQueryState, CancellationToken, Task<TResultData>>
serverDataProvider);

    /// <summary>
    /// Ad5D BC=0D0 :Cä=DD8C4CD 0Dd8D0 @C 1CäBD² GCT@CT7 C,,7Cä;C,,@Cä2C =C0>CR ;Cä:C ;DÄ=Cä5 Aä B20ED 0C08C'8D
    /// </summary>
    void ConfigureInMemoryMode(IUiOzuCache<TEntity> ozuCache, Func<TQueryState, List<TEntity>,
TResultData> inMemoryEvaluator);

    /// <summary>
    /// B4=C,,2CT@D 0C'LC00D0 BCäGC=0 C AC,,=DT@Cä=C0>C4> Ct0C0@CäAC 4C =C0KDR 2C,,7D40C'8Ct0D$>D 0CÄ8 C,,=D$5D
    /// ATAC'8 C08 Cä4C,,= D 5Cd8CÄ =CR 1D'; D :Cä=DD8C4CD 8D >C$0C0 ?D >C4@C <CÄ8D BCä<, C$Kct>C$5D" -çf Æ-D+
    /// </summary>
    Task<TResultData> FetchDataAsync(TQueryState state, CancellationToken cancellationToken =
default);

    /// <summary>
    /// B$>Dt:C 2DT>CD0 CD;D0 ?D >C @CäAC 3C'>C 0C'LC0>C4> D 8C4=C ;C öäVçF-G"6ö0Ö-GFVB 2 C'>C=0C'LC0KC' :
    /// </summary>
    void HandleDatabaseCommit(EntityStateChangeEnum state, object entity);
}
</file>

<file path="Promatis.Net.UI/Data/UiDataBroker.cs">
using Promatis.Net.Data;
namespace Promatis.Net.UI;

/// <summary>
/// A0;C BDD>D <CT=C0KC' 1D >C=5D 4C =C0KDRä &CT=D$@C ;C,,7Cä2C =C0> D4?D 0C$;D05D" ?CäAD$0C$:Cä9 CD0C0=D'E C

```

```

/// Aô>C´=CäAD$LDâ 0C AD$@C 3C,,@D45D" T´ >D" :Cä=Cª@CTBCÔKDR 8CÔBCT@DD5C"ACä2 CD>CÄ5CÔ=D´E D ;D46C 1DÔ:CT=CD
/// </summary>ð
/// <typeparam name="TEntity">B$8Cò 4Cä<CT=CÔ>C´ AD4ICÔ>D BCfÂ÷G– W & Ó1
public class UiDataBroker<TEntity, TQueryState, TResultData> : IUiDataBroker<TEntity, TQueryState,
TResultData>ð
    where TEntity : classð
{ð
    private Func<TQueryState, CancellationToken, Task<TResultData>>? _serverDataProvider;ð
    private Func<TQueryState, List<TEntity>, TResultData>? _inMemoryEvaluator;ð
    private IUiOzuCache<TEntity>? _ozuCache;ð
    private readonly Action? _onDataChangedNotifier; // B AD´;Cª0 CÔ0 NotifyStateChanged Cª>CÔBCT:D BC
ð
    private bool _isInitialized;ð
    private bool _isInMemoryMode;ð
ð
    public bool IsInMemoryMode => _isInMemoryMode;ð
ð
    // A$=CT4D OCT< notifier C" :Cä=D BD CCªBCä@ CD;Dò @CTÔCªBC,,2CÔ>C4> CÔ8CÔ:C T•
    public UiDataBroker(Action? onDataChangedNotifier = null)ð
    {ð
        _onDataChangedNotifier = onDataChangedNotifier;ð
ð
        // A4 Aä A BÄ B´ Aô B BT A ": Aª0Cd4D´9 C @Cä:CT@ C" AC,,AD$5CÄ5 CÔ0D$8C$=Cä =C GC,,=C 5D" AC´CD,,0D$
        DatabaseTriggerService.OnEntityCommitted += HandleGlobalEntityCommitted;ð
    }ð
ð
    private void HandleGlobalEntityCommitted(EntityStateChangeEnum state, object entity)ð
    {ð
        // A$KctKC$0CT< CÔ0D, 2CÔCD$@CT=CÔ8C´ <CTBCä4 Cä1D 0C >D$:C, <D4BC FC,,9D
        HandleDatabaseCommit(state, entity);ð
    }ð
ð
    public void ConfigureServerMode(Func<TQueryState, CancellationToken, Task<TResultData>>
serverDataProvider)ð
    {ð
        _serverDataProvider = serverDataProvider ?? throw new
ArgumentNullException(nameof(serverDataProvider));ð
        _inMemoryEvaluator = null;ð
        _ozuCache = null;ð
        _isInMemoryMode = false;ð
        _isInitialized = true;ð
    }ð
ð
    public void ConfigureInMemoryMode(IUiOzuCache<TEntity> ozuCache, Func<TQueryState,
List<TEntity>, TResultData> inMemoryEvaluator)ð
    {ð
        _ozuCache = ozuCache ?? throw new ArgumentNullException(nameof(ozuCache));ð
        _inMemoryEvaluator = inMemoryEvaluator ?? throw new
ArgumentNullException(nameof(inMemoryEvaluator));ð
        _serverDataProvider = null;ð
        _isInMemoryMode = true;ð
        _isInitialized = true;ð
    }ð
ð
    public async Task<TResultData> FetchDataAsync(TQueryState state, CancellationToken
cancellation token = default)ð
    {ð
        if (!_isInitialized)ð
        {ð
            throw new InvalidOperationException(ð
                $"A @Cä:CT@ CDÔCÔ=D´E CD;Dò w·G– Vöb...DVçF–G´´äæ ÖWòr =CR 1D´; D :Cä=DD8C4CD 8D >C$0CÒâ " ½
                $"A$K Cä1Dô7C =D² 2D´7C$0D$L ConfigureServerMode C,,;C, 6öæf–ww&T–äÖVö÷'"ÖöFR 2 Cª>CÔAD$@D4:D$
            )ð
        }ð
        if (_serverDataProvider != null)ð
        {ð
            return await _serverDataProvider(state, cancellation token);ð
        }ð
        if (_isInMemoryMode && _ozuCache != null && _inMemoryEvaluator != null)ð
        {ð
            return _inMemoryEvaluator(state, _ozuCache.InMemoryItems);ð
        }ð
        throw new InvalidOperationException($"Aª@C,,BC,,GCTACª8C´ AC >C´ 2CÔCD$@CT=CÔ5C´ :Cä=DD8C4CD 0Dd8C, 1D >
    )ð
}ð

```

```

Ð
public void HandleDatabaseCommit(EntityStateChangeEnum state, object entity)Ð
{Ð
    // Aô@Cä2CT@Dô5CÃ¢ >D$=CäAC,,BD O C´8 Cô@C,,;CTBCT2D,,8C´ 8Cr !B4 AB >C JCT:D" : D$8CôC CDôCÔ=D´E D$5Cp
    if (entity is not TEntity typedEntity) return;Ð
Ð
    // AÄ AD BD Bd B A$ AÔ : ATAC´8 CÄK C" @CT6C,,<CR -âÖVÖ÷´''Â BCäGCTGCÔ> Cä1CÔ>C$;Dô5CÄ ;Cä:C ;DÄ=D´9
    if (_isInMemoryMode && _ozuCache != null)Ð
    {Ð
        _ozuCache.ApplyOzuDelta(state, typedEntity);Ð
    }Ð
Ð
    // A" A A,,% B Ad AÄ BR ,, Server-Side, C, -âÖVÖ÷´''"-
    // ADôCT< D 5C :D$8C$=D´9 C,,<CôCC´LD =C CD >C$5CÔL UI-Cp>CÔBCT:D BC Â GD$>C K MudDataGrid Cö;C 2CÔ>
    _onDataChangedNotifier?.Invoke();Ð
}Ð
Ð
public void Dispose()Ð
{Ð
    // Ap A,, "A,, 'AT!Ap A$ Ad Aä¢ D$?C,,AD´2C 5CÄADò >D" AD$ôD$8Dt5D :Cä3Cä MC$5CÔBC AC´CCd1D² BD 8C43CT@
    // Cö>C´=CäAD$LDä 8D :C´NDtôDò CD$5Dt:C, >Cô5D ôD$8C$=Cä9 CöôCÄôD$8 C" &Æ |÷-
    DatabaseTriggerService.OnEntityCommitted -= HandleGlobalEntityCommitted;Ð
}Ð
}
</file>

<file path="Promatis.Net.UI/Extensions/MudColorExtensions.cs">
using MudBlazor;Ð
Ð
namespace Promatis.Net.UI;Ð
Ð
public static class MudColorExtensionsÐ
{Ð
    public static Color GetActionColor(this string? action)Ð
    {Ð
        if (string.IsNullOrEmpty(action)) return Color.Default;Ð
Ð
        return action.ToLower().Trim() switchÐ
        {Ð
            "create" or "insert" or "added" or "CD>C ôC$;CT=C,,5" or "D >Ct4C =C,,5" => Color.Success,Ð
            "update" or "edit" or "modified" or "C,,7CÄ5CÔ5CÔ8CR" ÷" $@CT4C :D$8D >C$ôCÔ8CR" ôâ 6ôÆ÷"âv &æ-ærí
            "delete" or "remove" or "deleted" or "softdeleted" or "D44C ;CT=C,,5" => Color.Error,Ð
            _ => Color.DefaultÐ
        };Ð
    }Ð
}
</file>

<file path="Promatis.Net.UI/Interfaces/IHasSelectedData.cs">
namespace Promatis.Net.UI.Components;Ð
Ð
/// <summary>Ð
/// B4=C,,2CT@D ôC´LCôKC´ 8CÔBCT@DD5C"A-CÄôD :CT@ CD;Dò 2C,,7D4ôC´8CtôD$>D >C"Â :CäBCä@D´< CÔ5Cä1DT>CD8CÄ> Ð
/// D 8CÔED >CÔ8Ct8D >C$ôD$L DD>CpCD AD$@Cä:C,ôCct;C 8 Cö>C´CDtôD$L C,,<CôCC´LD K Cô5D 5D 8D >C$:C,i
/// </summary>Ð
public interface IHasSelectedData<TEntity> where TEntity : classÐ
{Ð
    TEntity? SelectedData { get; set; }Ð
    Action? OnContextUpdated { get; set; }Ð
}
</file>

<file path="Promatis.Net.UI/IUiModule.cs">
namespace Promatis.Net.UI;Ð
Ð
/// <summary>Ð
/// Ap>CÔBD ôCpB CD;Dò 4C,,=C <C,,GCTACp8DR <Cä4D4;CT9 Cö>C´Lct>C$ôD$5C´LD :Cä3Cä 8CÔBCT@DD5C"AC í
/// </summary>Ð
public interface IUiModuleÐ
{Ð
    /// <summary>Ð
    /// AäBCä1D ôCdôCT<Cä5 C,,<Dò <Cä4D4;Dò 2 D 8D BCT<CR ,,8D ?Cä;DÄ7D45D$ADò 4C´O D :C =CT@C ´í
    /// </summary>Ð
    string Name { get; }Ð
Ð
    /// <summary>Ð

```



```

    /// A$>Ct2D 0D"0CTB D ?C,,ACä: D0;CT<CT=D$>C" =C 2C,,3C FC,,8 CD;D0 1Cä:Cä2Cä3Cä <CT=Dâ xVDG& vW"í
    /// Bt5D$2CT@D$KC' ?C @C <CTBD >D$2CTGC 5D" 7C "0CD >C$=CT2D4N C4@D4?C08D >C$:D2 MC'5CÄ5C0BCä2.D
    /// </summary>D
    /// <returns>A>>C';CT:Dd8D0 :Cä@D$5Cd5C"¢ ,, C 7C$0C08CRÄ !D KC':C Ä C>>C0:C Ä C 7C$0C08CT D CC0?D²"Ä÷&W
    IEnumerable<(string Title, string Href, string Icon, string? Group)> GetMenuItems();D
}
</file>

<file path="Promatis.Net.UI/Pages/AuditLogs/AuditLogPage.razor">
@page "/audit-logs"D
@using Promatis.Net.DomainD
D
<CascadingValue Value="Context">D
    <WorkspacePage>D
D
        <!-- 1. A$ B %A0/B0 Ää A ¢ C$BCä<C BC,,GCTAC=8C' ?Cä;C,,<Cä@DD=D'9 D$CC'1C @ (AD8C00CÄ8Dt5D :C,,5 C>>C
        <TopContent>D
            <UiToolbar Controls="@Context.Controls" />D
        </TopContent>D
D
        <!-- 2. Bd A0"B A',A0 B0 Ää A ¢ "C 1C'8Dt=C 0 D 5D$:C Ä @C 1CäBC ND"0D0 2 DD>C0>C$>CÄ ACT@C$5D =Cä<
        <BodyContent>D
            <GridPage TEntity="AuditLog"D
                @ref="_grid"D
                ServerDataProvider="@Context.Broker.FetchDataAsync"D
                WithPagination="true"D
                RowsPerPage="20">D
                <GridColumn>D
                    <PropertyColumn T="AuditLog" TProperty="DateTime" Property="x => x.ChangedAt"
Title="AD0D$0 C, 2D 5CÄ0" Format="dd.MM.yyyy HH:mm:ss" HeaderStyle="width: 160px;" />D
                    <PropertyColumn T="AuditLog" TProperty="string" Property="x => x.ChangedBy"
Title="A0>C'Lct>C$0D$5C'L" HeaderStyle="width: 150px;" />D
                    <PropertyColumn T="AuditLog" TProperty="string" Property="x => x.EntityName"
Title="B$8C0 AD4IC0>D BC," †V FW%7G-ÆS0'v-GFf¢ S f²" óí
                    <PropertyColumn T="AuditLog" TProperty="string" Property="x => x.Action"
Title="Aä?CT@C FC,,0" HeaderStyle="width: 120px;" />D
                    <PropertyColumn T="AuditLog" TProperty="Guid" Property="x => x.EntityId"
Title="ID B CD"=CäAD$8" HeaderStyle="width: 300px;" />D
                    <PropertyColumn T="AuditLog" TProperty="string" Property="x => x.ChangesJson"
Title="A,,7CÄ5C05C08D0 „¥40â'" †V FW%7G-ÆS0'v-GFf¢ WF0²" 6Æ 730'FW‡B×G'Væ6 FR" óí
                </GridColumn>D
            </GridPage>D
        </BodyContent>D
D
    </WorkspacePage>D
</CascadingValue>
</file>

<file path="Promatis.Net.UI/Pages/AuditLogs/AuditLogPage.razor.cs">
using Microsoft.AspNetCore.Components;D
using Promatis.Net.Domain;D
using Promatis.Net.Service;D
using Promatis.Net.UI.Components.Workspaces;D
D
namespace Promatis.Net.UI.Pages.AuditLogs;D
D
public partial class AuditLogPage : ComponentBaseD
{D
    private GridPage<AuditLog>? _grid;D
    private bool _isLoading;D
D
    [Inject]D
    protected IAuditLogService AuditLogService { get; set; } = null!;D
D
    protected AuditLogWorkspaceContext Context { get; set; } = null!;D
D
    protected override void OnInitialized()D
    {D
        base.OnInitialized();D
D
        // ÄÄ A0 A$ A0 Ää ACä7CD0CT< C0CD BCä9 C>>C0BCT:D B. B BD 0C08Dd0 D 5C04CT@C,,BD 0 C 5Cr 7C 4CT@Cd5C¢
        Context = new AuditLogWorkspaceContext(AuditLogService, onFilterChanged: RefreshGrid);D
    }D
D
    /// <summary>D
    /// A=0C0>C08Dt5D :C 0 D$>Dt:C 7C ?D4AC=0 D$0Cd5C'KDR 7C ?D >D >C"â !D 0C 0D$KC$0CTB A0 B AR BCä3CäÄ

```

```

/// C#0C# ?Cä;DÄ7Cä2C BCT;DÄ CCd5 D42C,,4CT; C4>D$>C$CDä AD$@C =C,,FD2 =C MC#@C =CRi
/// </summary>#
protected override async Task OnAfterRenderAsync(bool firstRender)#
{#
    await base.OnAfterRenderAsync(firstRender);#
#
    // A$KCö>C´=Dö5CÄ 7C 3D CCT:D2 BCä;DÄ:Cä >CD8Cö @C 7 Cö@C, ?CT@C$>CÄ @CT=CD5D 5 Dö:D 0Cö0#
    if (firstRender)#
    {#
        // B ?Cä:Cä9Cö> D :C GC,,2C 5CÄ CCö8C#0C´LCöKCR 8CÄ5Cö0 D CD´=CäAD$5C´ 8Cr !B4 AB ÷7Fw&U5 Í
        List<string> availableEntities = await AuditLogService.GetAvailableEntityNamesAsync();#
#
        // Aö5D 5CD0CT< CD0Cö=D´5 C" :Cä=D$5C#AD" 4C´O CD8Cö0CÄ8Dt5D :Cä9 C45Cö5D 0Dd8C, ACT;CT:D$0 C, ?C
        Context.InitializeFilters(availableEntities);#
#
        _isLoading = true;#
        StateHasChanged(); // Aö5D 5D 8D >C$KC$0CT< D$CC´1C @ C, 7C ?D4AC#0CT< Cö5D 2D4N Ct0C4@D47C#C C´>
    }#
#
protected void RefreshGrid()#
{#
    // At0Cö@CTIC 5CÄ ?D 8CöCCD8D$5C´LCö>CR >C =Cä2C´5Cö8CR 3D 8CD0 CäB D$@C,,3C45D >C" DC,,;DÄBD >C"Í
    // Cö>C#0 DöBC, DC,,;DÄBD K CTICR ?CT@C$8Dt=Cä =CR =C AD$@Cä5CöK C" öä gFW%&VæFW$ 7-æ=
    if (_isLoading && _grid != null)#
    {#
        _ = _grid.ReloadServerData();#
    }#
#
}
</file>

<file path="Promatis.Net.UI/Pages/AuditLogs/AuditLogWorkspaceContext.cs">
using MudBlazor;#
using Promatis.Net.Domain;#
using Promatis.Net.Service;#
using Promatis.Net.UI.Components;#
using Promatis.Net.UI.Pages.AuditLogs.Toolbar;#
#
namespace Promatis.Net.UI.Pages.AuditLogs;#
#
public class AuditLogWorkspaceContext : WorkspaceActionContext, IHasSelectedData<AuditLog>#
{#
    private readonly IAuditLogService _auditLogService;#
    private readonly Action _onFilterChanged;#
#
    public UiDataBroker<AuditLog, GridState<AuditLog>, GridData<AuditLog>> Broker { get; }#
#
    public AuditLog? SelectedData#
    {#
        get;#
        set;#
        {#
            if (field != value)#
            {#
                field = value;#
                OnContextUpdated?.Invoke();#
                NotifyStateChanged();#
            }#
        }#
    }#
#
    public Action? OnContextUpdated { get; set; }#
#
    public override string TopZoneHeight => "48px";#
#
    public AuditLogWorkspaceContext(IAuditLogService auditLogService, Action onFilterChanged)#
    {#
        _auditLogService = auditLogService ?? throw new
ArgumentNullException(nameof(auditLogService));#
        _onFilterChanged = onFilterChanged;#
#
        Broker = new UiDataBroker<AuditLog, GridState<AuditLog>,
GridData<AuditLog>>(NotifyStateChanged);#
        Broker.ConfigureServerMode(FetchAuditDataFromServerAsync);#
#
    }
}

```

```

        // At0CÄ5C00CT< new ToolbarDivider() C00 C0>C$KC' AC05Dd8C ;C,,7C,,@Cä2C =C0KC' ?D 0C$KC' @C 7CD5C'8D$5
        AddControl(new AuditToolbarDivider());ð
        AddControl(new AuditExportButton());ð
    }ð
ð
    public void InitializeFilters(List<string> entityNames)ð
    {ð
        _controls.Insert(0, new AuditEntitySelect(entityNames, _onFilterChanged));ð
        _controls.Insert(1, new AuditActionSelect(_onFilterChanged));ð
        _controls.Insert(2, new AuditPeriodPicker(_onFilterChanged));ð
        NotifyStateChanged();ð
    }ð
ð
    /// <summary>ð
    /// B 5D 2CT@C00D0 AD$@C BCT3C,,0. AÄ0C08D" ACäAD$>D0=C,,5 MudBlazor C" AD$@Cä3Cä ?Cä7C,,FC,,>C0=D'9 C0>C0AD$
    /// </summary>ð
    private async Task<GridData<AuditLog>> FetchAuditDataFromServerAsync(GridState<AuditLog> state,
        CancellationToken ct)ð
    {ð
        IUiControl? entityControl = Controls.FirstOrDefault(c => c.Id == "audit_filter_entity");ð
        IUiControl? actionControl = Controls.FirstOrDefault(c => c.Id == "audit_filter_action");ð
        IUiControl? periodControl = Controls.FirstOrDefault(c => c.Id == "audit_filter_period");ð
ð
        string? selectedEntity = entityControl is IHasValue ev ? ev.Value as string : null;ð
        DateRange? selectedPeriod = periodControl is IHasValue pv ? pv.Value as DateRange : null;ð
ð
        // A0>C'CDt0CT< D 8D BCT<C0>CR 8CÄ0 Cä?CT@C FC,,8 (create/update/delete C,,;C, çVÆÄ•
        string? selectedAction = actionControl is AuditActionSelect av ?
        av.GetSelectedActionValue() : null;ð
ð
        if (selectedEntity == "A$ACR AD4IC0>D BC,"' 00 !D$@Cä:C 07C 3C'CD,,:C 8Cr 2C HCT3Cä :Cä4C
            selectedEntity = null;ð
ð
        DateTime fromDate = selectedPeriod?.Start ?? DateTime.Today.AddDays(-7);ð
        DateTime toDate = selectedPeriod?.End ?? DateTime.Today;ð
ð
        fromDate = DateTime.SpecifyKind(fromDate.Date, DateTimeKind.Utc);ð
        toDate = DateTime.SpecifyKind(toDate.Date.AddDays(1).AddTicks(-1), DateTimeKind.Utc);ð
ð
        var searchRequest = new AuditLogSearchRequest(ð
            FromDate: fromDate,ð
            ToDate: toDate,ð
            EntityName: selectedEntity,ð
            Action: selectedAction, ð
            PageIndex: state.Page,ð
            PageSize: state.PageSizeð
        );ð
ð
        PagedResult<AuditLog> pagedResult = await _auditLogService.SearchLogsAsync(searchRequest,
        ct);ð
ð
        return new GridData<AuditLog>ð
        {ð
            Items = pagedResult.Items,ð
            TotalItems = pagedResult.TotalCountð
        };ð
    }ð
}
</file>

```

```

<file path="Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditActionSelect.cs">
using Promatis.Net.UI.Components;ð
using Promatis.Net.UI.Components.ElementRenderBase;ð
ð
namespace Promatis.Net.UI.Pages.AuditLogs.Toolbar;ð
ð
public class AuditActionSelect : BaseUiControl, IHasValue, IHasOptionsð
{ð
    private readonly Action _onChanged;ð
    private readonly Dictionary<string, string> _actionMapping = new()ð
    {ð
        { "A$ACR >C05D 0Dd8C,"Ä "" 0í
        { "B >Ct4C =C,,5", "Added" },ð
        { "A,,7CÄ5C05C08CR"Ä $0öF-f-VB" 0í
        { "B44C ;CT=C,,5", "Deleted" },ð
        { "AÄ0C4:Cä5 D44C ;CT=C,,5", "SoftDeleted"}ð
    }
}

```

```

};
}

public override string Id => "audit_filter_action";
public override Type ComponentType => typeof(SelectRenderBase);
public override string Title => "Audit Filter Action";

public object? Value { get; set; }
public IEnumerable<string> Options => _actionMapping.Keys;

public AuditActionSelect(Action onChanged)
{
    _onChanged = onChanged;
    Value = "Audit Filter Action";
}

// Audit Filter Action
public string? GetSelectedActionValue()
{
    if (Value is string selectedText && _actionMapping.TryGetValue(selectedText, out string? sysValue))
    {
        return string.IsNullOrEmpty(sysValue) ? null : sysValue;
    }
    return null;
}

protected override Task HandleTriggerAsync(object? targetData)
{
    _onChanged.Invoke();
    return Task.CompletedTask;
}
}
</file>

```

```

<file path="Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditEntitySelect.cs">
using Promatis.Net.UI.Components;
using Promatis.Net.UI.Components.ElementRenderBase;
namespace Promatis.Net.UI.Pages.AuditLogs.Toolbar;
public class AuditEntitySelect : BaseUiControl, IHasValue, IHasOptions
{
    private readonly Action _onChanged;

    public override string Id => "audit_filter_entity";
    public override Type ComponentType => typeof(SelectRenderBase);
    public override string Title => "Audit Filter Entity";

    public object? Value { get; set; }
    public IEnumerable<string> Options { get; set; }

    public AuditEntitySelect(List<string> availableEntities, Action onChanged)
    {
        _onChanged = onChanged;
        Value = "Audit Filter Entity";

        // Audit Filter Entity
        var list = new List<string> { "Audit Filter Entity", "Audit Filter Entity" };
        list.AddRange(availableEntities);
        Options = list;
    }

    protected override Task HandleTriggerAsync(object? targetData)
    {
        _onChanged.Invoke();
        return Task.CompletedTask;
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditExportButton.cs">
using MudBlazor;
using Promatis.Net.UI.Components;
using Promatis.Net.UI.Components.ElementRenderBase;
namespace Promatis.Net.UI.Pages.AuditLogs.Toolbar;

```

```

ð
public class AuditExportButton : BaseUiControlð
{ð
    public override string Id => "audit_action_export";ð
    public override Type ComponentType => typeof(ButtonRenderBase); // B 5C04CT@CT@ D04D 0ð
    public override string Title => "A$KC4@D47C,,BDÄ#½
    public override string Icon => Icons.Material.Filled.Download;ð
    public override string Tooltip => "A$KC4@D47C,,BDÄ >D$DC,,;DÄBD >C$0C0=D´5 C´>C48 C" W†6VÂ#½
ð
    public AuditExportButton()ð
    {ð
        Alignment = UiControlAlignment.Right; // B 4C$8C40CT< C¤=Cä?C¤C D0:D ?Cä@D$0 C$?D 0C$>ð
    }ð
ð
    protected override async Task HandleTriggerAsync(object? targetData)ð
    {ð
        // A,,<C,,BC,,@D45CÂ 4Cä;C4CDâ 2D´3D CCt:D2 r :C0>C0:C 0C$BCä<C BC,,GCTAC¤8 Ct0C ;Cä:C,,@D45D$AD0 8 C0>C¤
        await Task.Delay(2000);ð
    }ð
}
</file>

<file path="Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditPeriodPicker.cs">
using MudBlazor;ð
using Promatis.Net.UI.Components;ð
using Promatis.Net.UI.Components.ElementRenderBase;ð
ð
namespace Promatis.Net.UI.Pages.AuditLogs.Toolbar;ð
ð
public class AuditPeriodPicker : BaseUiControl, IHasValueð
{ð
    private readonly Action _onChanged;ð
ð
    public override string Id => "audit_filter_period";ð
    public override Type ComponentType => typeof(DateRangeRenderBase); // B 5C04CT@CT@ D04D 0ð
    public override string Title => "A05D 8Cä4 C´>C4>C"#½
ð
    public object? Value { get; set; }ð
ð
    public AuditPeriodPicker(Action onChanged)ð
    {ð
        _onChanged = onChanged;ð
        Value = new DateRange(DateTime.Today.AddDays(-7), DateTime.Today);ð
    }ð
ð
    protected override Task HandleTriggerAsync(object? targetData)ð
    {ð
        _onChanged.Invoke(); // A05D 8Cä4 C,,7CÄ5C08C´AD0 r 4C 5CÄ 8CÄ?D4;DÄA D BD 0C08Dd5ð
        return Task.CompletedTask;ð
    }ð
}
</file>

<file path="Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditToolbarDivider.cs">
using Promatis.Net.UI.Components;ð
using Promatis.Net.UI.Components.ElementRenderBase;ð
ð
namespace Promatis.Net.UI.Pages.AuditLogs.Toolbar;ð
ð
public class AuditToolbarDivider : BaseUiControlð
{ð
    private readonly string _id = "audit_toolbar_divider_" + Guid.NewGuid().ToString("N");ð
ð
    public override string Id => _id;ð
    public override Type ComponentType => typeof(DividerRenderBase); // A,,AC0>C´LCtCCT< D$>D" 6CR 1C 7Cä2D´9
ð
    public AuditToolbarDivider()ð
    {ð
        Alignment = UiControlAlignment.Right; // B0BCäB D 0Ct4CT;C,,BCT;DÄ 3C @C =D$8D >C$0C0=Cä AC0@C 2C
    }ð
ð
    protected override Task HandleTriggerAsync(object? targetData)ð
    {ð
        return Task.CompletedTask;ð
    }ð
}

```



```

ð
</Project>
</file>

<file path="Promatis.Net.UI/Theme/PromatisTheme.cs">
using MudBlazor;ð
ð
namespace Promatis.Net.Ui.Theme;ð
ð
public static class PromatisThemeð
{ð
    public static MudTheme DefaultTheme => new()ð
    {ð
        PaletteLight = new PaletteLightð
        {ð
            Primary = Colors.Blue.Lighten1, // A4;C 2C0KC' :Cä@Cö>D 0D$8C$=D'9 Dd2CTB (C¤=Cä?C¤8, C :D$
            Secondary = Colors.Blue.Lighten2, // A$ACö>CÄ>C40D$5C'LC0KC' FC$5D" „2D$>D >D BCT?CT=C0KCR MC

ð
            AppbarBackground = "#EAEAEA", // BD>C0 2CT@DT=CT9 C00C05C'8 (MudAppBar) C" 4C05C$=Cä< D 5
            AppbarText = "#2C3E50", // Bd2CTB D$5C¤AD$0, Ct0C4>C'>C$:Cä2 C, 8C¤>C0>C¤ 2 C$5D EC
            DrawerBackground = "#EAEAEA", // BD>C0 1Cä:Cä2Cä3Cä <CT=Dâ =C 2C„3C FC„8 (MudDrawer)ð
            Background = "#F9F9F9", // Aä1D"8C' DCä=Cä2D'9 Dd2CTB C0>CD;Cä6C¤8 C$ACTE D BD 0C08
            Surface = Colors.Shades.White, // BD>C0 :Cä=D$5C"=CT@Cä2 MudPaper, C¤0D BCäGCT: MudCard C

        },ð
        PaletteDark = new PaletteDarkð
        {ð
            Primary = Colors.Blue.Lighten1, // A4;C 2C0KC' FC$5D" 2 C0>Dt=Cä< D 5Cd8CÄ5 (CÄ0C4:C„9 D 8C
            Secondary = Colors.Blue.Lighten2, // A$ACö>CÄ>C40D$5C'LC0KC' FC$5D" 2 C0>Dt=Cä< D 5Cd8CÄ5ð

ð
            AppbarBackground = "#0D0D0D", // BD>C0 2CT@DT=CT9 C00C05C'8 (MudAppBar) C" 3C'CC >C¤>CÄ G
            AppbarText = Colors.Shades.White, // Bd2CTB D$5C¤AD$0 C, 8C¤>C0>C¤ 2 D„0C0:CR „:Cä=D$@C AD$=D

ð
            Background = "#121212", // A4;D41Cä:C„9 D$QCÄ=D'9 DD>C0 AD$@C =C„F C0>CD;Cä6C¤8ð
            Surface = "#1E1E1E", // A4@C DC„BCä2D'9 DD>C0 4C'0 MudPaper, C¤>C0BCT=D$0 C$:C'0
            DrawerBackground = "#1E1E1E", // BD>C0 1Cä:Cä2Cä9 C00C05C'8 C00C$8C40Dd8C, 2 D$QCÄ=Cä9 D$
            DrawerText = "#E0E0E0", // Bd2CTB D AD';Cä: C, BCT:D BC 2 C >C¤>C$>CÄ <CT=Dí
            TextPrimary = "#E0E0E0", // AäAC0>C$=Cä9 Dd2CTB D$5C¤AD$0 C00 D BD 0C08Dd0DR „<D03C¤
            TextSecondary = "#A0A0A0", // Bd2CTB CD;D0 2D$>D >D BCT?CT=C0>C4> D$5C¤AD$0 C, ?Cä4D

        },ð
        LayoutProperties = new LayoutPropertiesð
        {ð
            DrawerWidthLeft = "260px",ð
            DrawerWidthRight = "300px"ð
        }ð
    };ð
}
</file>

<file path="Promatis.Net.UI/UiModule.cs">
using MudBlazor;ð
ð
namespace Promatis.Net.UI;ð
ð
public class UiModule : IUiModuleð
{ð
    public string Name => GetType().Assembly.GetName().Name ?? "Unknown.Module";ð

ð
    public IEnumerable<(string Title, string Href, string Icon, string? Group)> GetMenuItems()ð
    {ð
        return new List<(string Title, string Href, string Icon, string? Group)>ð
        {ð
            // B0;CT<CT=D" 2CT@DT=CT3Cä CD >C$=D0 „2C05 C00C0>C¤•
            ("A4;C 2C00D0"Ä "0"Ä -6öç2ä0 FW&- Ääf-ÆÆVBäF 6†&ö &BÄ çVÆÄ'Í
            ð
            // B 0Ct4CT; D 8D BCT<C0>C4> C 4CÄ8C08D BD 8D >C$0C08Dý
            ("AdCD =C ; C CCD8D$0", "/audit-logs", Icons.Material.Filled.History, "A 4CÄ8C08D BD 8D >C$0C08CR

        },ð
    };ð
}
</file>

<file path="Promatis.Net.UI/wwwroot/App.css">
html, body {ð
    font-family: 'Helvetica Neue', Helvetica, Arial, sans-serif;ð
}ð
ð

```

```
a, .btn-link {  
    color: #006bb7;  
}  
  
.btn-primary {  
    color: #fff;  
    background-color: #1b6ec2;  
    border-color: #1861ac;  
}  
  
.btn:focus, .btn:active:focus, .btn-link.nav-link:focus, .form-control:focus, .form-check-input:focus {  
    box-shadow: 0 0 0 0.1rem white, 0 0 0 0.25rem #258cfb;  
}  
  
.content {  
    padding-top: 1.1rem;  
}  
  
h1:focus {  
    outline: none;  
}  
  
.valid.modified:not([type=checkbox]) {  
    outline: 1px solid #26b050;  
}  
  
.invalid {  
    outline: 1px solid #e50000;  
}  
  
.validation-message {  
    color: #e50000;  
}  
  
.blazor-error-boundary {  
    background: url() no-repeat 1rem/1.8rem, #b32121;  
    padding: 1rem 1rem 1rem 3.7rem;  
    color: white;  
}  
  
.blazor-error-boundary::after {  
    content: "An error has occurred."  
}  
  
.darker-border-checkbox.form-check-input {  
    border-color: #929292;  
}  
  
.form-floating > .form-control-plaintext::placeholder, .form-floating > .form-control::placeholder {  
    color: var(--bs-secondary-color);  
    text-align: end;  
}  
  
.form-floating > .form-control-plaintext:focus::placeholder, .form-floating > .form-control:focus::placeholder {  
    text-align: start;  
}  

```



```

<file path="Service/Contracts/AuditLog/AuditLogSearchRequest.cs">
namespace Promatis.Net.Service;
{
    public record AuditLogSearchRequest(
        DateTime FromDate,
        DateTime ToDate,
        string? EntityName = null,
        string? Action = null,
        int PageIndex = 0,
        int PageSize = 10);
}
</file>

<file path="Service/Contracts/AuditLog/PagedResult.cs">
namespace Promatis.Net.Service;
{
    public record PagedResult<T>(IEnumerable<T> Items, int TotalCount);
}
</file>

<file path="Service/Promatis.Net.Service.csproj">
<Project Sdk="Microsoft.NET.Sdk">

    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
    </PropertyGroup>

    <ItemGroup>
        <PackageReference Include="Microsoft.EntityFrameworkCore.Relational" Version="10.0.8" />
    </ItemGroup>

    <ItemGroup>
        <ProjectReference Include="..\Data\Promatis.Net.Data.csproj" />
    </ItemGroup>

</Project>
</file>

<file path="Service/Services/AuditLogService/AuditLogService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Data;
using Promatis.Net.Domain;
{
    namespace Promatis.Net.Service;
    {
        /// <summary>
        /// B 5D 2C,,A CD;Dò @C 1CäBD² A C´>C40CÄ8 C CCD8D$0.D
        /// Aô>C´=CäAD$LDä ACä2CÄ5D BC,,< D 1C 7Cä2D´< BaseService C, 0C$BCä<C BC,,GCTACª8 D 5C48D BD 8D CCTBD O D :C
        /// </summary>
        public class AuditLogService(IDbContextFactory<ApplicationDbContext> contextFactory)
            : BaseService<AuditLog, Guid, ApplicationDbContext>(contextFactory), IAuditLogService
        {
            private static List<string>? _cachedEntityNames;
            private static readonly SemaphoreSlim CacheLock = new(1, 1);

            public async Task<PagedResult<AuditLog>> SearchLogsAsync(AuditLogSearchRequest request,
                CancellationToken cancellationToken = default)
            {
                await using ApplicationDbContext context = await
                ContextFactory.CreateDbContextAsync(cancellationToken);

                IQueryable<AuditLog> query = context.Set<AuditLog>()
                    .AsNoTracking()
                    .Where(l => l.ChangedAt >= request.FromDate && l.ChangedAt <= request.ToDate);

                if (!string.IsNullOrEmpty(request.EntityName)) query = query.Where(l => l.EntityName
                == request.EntityName);
                if (!string.IsNullOrEmpty(request.Action)) query = query.Where(l => l.Action ==
                request.Action);

                int totalCount = await query.CountAsync(cancellationToken);

                if (totalCount == 0)
                {
                    // A,,ACô>C´LCtCCT< Array.Empty C,,C, ?D4AD$CDä :Cä;C´5CªFC,,N C 5Cr ;C,,HCÔ8DR 0C´;Cä:C FC,,9 D ?C,,A
                    return new PagedResult<AuditLog>([], 0);
                }
            }
        }
    }
}

```

```

    }
}

// A,,!Aô A A' Aô : AD>C 0C$;CT=C 2D$>D 8Dt=C 0 D >D BC,,@Cä2C=0 C0> Id CD;D0 CD BD 0C05C08D0 MDDDC:
List<AuditLog> items = await query
    .OrderByDescending(l => l.ChangedAt)
    .ThenByDescending(l => l.Id)
    .Skip(request.PageIndex * request.PageSize)
    .Take(request.PageSize)
    .ToListAsync(cancellationToken);

return new PagedResult<AuditLog>(items, totalCount);
}

public async Task<List<string>> GetAvailableEntityNamesAsync(CancellationTok
cancellationToken = default)
{
    if (_cachedEntityNames != null)
    {
        return [... _cachedEntityNames];
    }

    await CacheLock.WaitAsync(cancellationToken);
    try
    {
        if (_cachedEntityNames != null)
        {
            return [... _cachedEntityNames];
        }

        await using ApplicationDbContext context = await
ContextFactory.CreateDbContextAsync(cancellationToken);

        _cachedEntityNames = await context.Set<AuditLog>()
            .AsNoTracking()
            .Select(l => l.EntityName)
            .Distinct()
            .OrderBy(name => name)
            .ToListAsync(cancellationToken);

        return [... _cachedEntityNames];
    }
    finally
    {
        CacheLock.Release();
    }
}

public static void InvalidateCache()
{
    CacheLock.Wait();
    try
    {
        _cachedEntityNames = null;
    }
    finally
    {
        CacheLock.Release();
    }
}
}
</file>

```

```

<file path="Service/Services/AuditLogService/IAuditLogService.cs">
using Promatis.Net.Domain;
namespace Promatis.Net.Service;
public interface IAuditLogService
{
    /// <summary>
    /// A0>C,,AC ;Cä3Cä2 C CCD8D$0 C0> CD8C ?C 7Cä=D2 4C B D ?Cä4CD5D 6C=C' DC,,;DÄBD 0Dd8C, 8 C00C48C00Dd8C
    /// </summary>
    Task<PagedResult<AuditLog>> SearchLogsAsync(AuditLogSearchRequest request, CancellationToken
cancellationToken = default);
}
    /// <summary>

```

```

    /// A0>C'CDt5C08CR AC08D :C CC08C0C'LC0KDR 8CÄ5C0 AD4IC0>D BCT9, C0>D$>D KCR 5D BDÄ 2 C'>C40DR „4C'O C$
    /// </summary>
    Task<List<string>> GetAvailableEntityNamesAsync(CancellationToken cancellationToken = default);
}
</file>

<file path="Service/Services/BaseService/BaseService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain.Interface;

namespace Promatis.Net.Service;

public abstract class BaseService<T, TKey, TContext>(IDbContextFactory<TContext> contextFactory)
    : IBaseService<T, TKey>
    where T : class, IDomainObjectHasKey<TKey>
    where TContext : DbContext
{
    protected readonly IDbContextFactory<TContext> ContextFactory = contextFactory ?? throw new
ArgumentNullException(nameof(contextFactory));

    public virtual async Task<T?> GetByIdAsync(TKey id)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        return await context.Set<T>().FindAsync(id);
    }

    public virtual async Task<List<T>> GetAllAsync()
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        return await context.Set<T>().ToListAsync();
    }

    public virtual async Task<PagedResult<T>> GetPagedAsync(int pageIndex, int pageSize,
CancellationToken ct = default)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();

        // 1. B >Ct4C 5CÄ 1C 7Cä2D'9 Ct0C0@CÄA C¢ BC 1C'8Dd5 D CD"=CäAD$8
        IQueryable<T> query = context.Set<T>();

        // 2. A0>C'CDt0CT< Cä1D"5CR :Cä;C„GCTAD$2Cä 7C ?C„ACT9 C" AB 4C'O C00C48C00D$>D 00
        int totalCount = await query.CountAsync(ct);

        // 3. B >D BC„@D45CÄ ?Cä ?CT@C$8Dt=Cä<D2 :C'NDtC (ID) CD;D0 4CTBCT@CÄ8C08D >C$0C0=Cä3Cä ?Cä@D04C00 D
        // B$0C¢ :C : D2 =C A CTAD$L Cä3D 0C08Dt5C08CR v†W&R B ¢ "FöÖ -äö&!V7D† 4¶W"ÄD¶W"âÄ <D² <Cä6CT< C 5Ct
        query = query.OrderBy(x => x.Id);

        // 4. AäBD 5Ct0CT< C0CCd=D4N D BD 0C08DdC D ?Cä<CäIDÄN Skip/Take C, <C BCT@C„0C'8CtCCT< D ?C„ACä:D
        List<T> items = await query
            .Skip(pageIndex * pageSize)
            .Take(pageSize)
            .ToListAsync(ct);

        // 5. A$>Ct2D 0D"0CT< D4?C :Cä2C =C0KC' @CT7D4;DÄBC B0
        return new PagedResult<T>(items, totalCount);
    }

    public virtual async Task AddAsync(T entity)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        await context.Set<T>().AddAsync(entity);
        await context.SaveChangesAsync();
    }

    public virtual async Task UpdateAsync(T entity)
    {
        if (entity == null) throw new ArgumentNullException(nameof(entity));

        await using TContext context = await ContextFactory.CreateDbContextAsync();

        // 1. A„7C$;CT:C 5CÄ A"\"B4 A',A0+A' >D 8C48C00C'LC0KC' >C JCT:D" 8Cr 1C 7D² 4C =C0KDR ?Cä 5C4> C05D
        // A ;C 3Cä4C @D0 6öç7G& -çG2 C C00D 3C @C =D$8D >C$0C0=Cä 5D BDÄ 4CäAD$CC0 : x.Id
        T? dbEntity = await context.Set<T>().FindAsync(entity.Id);

        if (dbEntity == null)
        {

```

```

        throw new InvalidOperationException($"A05 D44C ;CäADÂ >C =Cä2C,,BDÂ >C JCT:D#¢ AD4ICÔ>D BDÂ w·G- V
CD0CÔ=D'E.");ð
    }ð
ð
    // 2. B GC,,BD'2C 5CÂ <CTBC 4C =CÔKCR >D$AC'5Cd8C$0CÔ8Dò 4C'0 Cä@C,,3C,,=C ;DÄ=Cä3Cä >C JCT:D$0 B #A ð
    var entry = context.Entry(dbEntity);ð
ð
    // 3. A05D 5CÔ>D 8CÂ 7CÔ0Dt5CÔ8Dò "Aä BÄ Aä B AÄ B$ A$ B' % D 2Cä9D BC" 8Cr :C'>CÔ0 C" >D 8C48CÔ0C'L
    // AÄ5D$>CB 6WEf ÇVW2 0C$BCä<C BC,,GCTAC¤8 CÔ@Cä8C4=Cä@C,,@D45D" =C 2C,,3C FC,,>CÔ=D'5 D 2Cä9D BC$0 (Pare
    // C, :Cä;C'5C¤FC,,8 CäBCÔ>D,,5CÔ8C'Ä :CäBCä@D'5 C$0D, :C'>CÔ5D 2D'@CT7C ;! Aä= Cä1CÔ>C$8D" BCä;DÄ:Cä
    entry.CurrentValues.SetValues(entity);ð
ð
    // 4. AD>CÔ>C'=C,,BCT;DÄ=C 0 Ct0D"8D$0: CÔ@Cä2CT@Dô5CÄÄ 8Ct<CT=C,,;CäADÂ ;C, ECäBDÄ GD$>-D$>.ð
    // ATAC'8 CÔ>C'Lct>C$0D$5C'L CäBC¤@D'; CD8C ;Cä3, CÔ8Dt5C4> CÔ5 CÄ5CÔ0C² 8 CÔ0Cd0C² !CäED 0CÔ8D$L,ð
    // CÄK C$>Cä1D"5 CÔ5 C CCD5CÄ 4E @C40D$L D$@C =Ct0C¤FC,,N PostgreSQL!ð
    if (entry.State == EntityState.Unchanged)ð
    {ð
        return;ð
    }ð
ð
    // 5. EF Core D 3CT=CT@C,,@D45D" EC,,@D4@C48Dt5D :C,,9 SQL-Ct0CÔ@CäA, Cä1CÔ>C$8C" 2 A B$ A',A¤ C,,7CÄ5
    // B CD"5D BC$CDäIC,,5 D 2Dô7C, 8 C,,7CÄ5CÔ5CÔ8Dò 4D CC48DR ?Cä;DÄ7Cä2C BCT;CT9 C" MD$>D" <Cä<CT=D" =CR
    await context.SaveChangesAsync();ð
}ð
ð
public virtual async Task DeleteAsync(TKey id)ð
{ð
    await using TContext context = await ContextFactory.CreateDbContextAsync();ð
    T? entity = await context.Set<T>().FindAsync(id);ð
    if (entity != null)ð
    {ð
        context.Set<T>().Remove(entity);ð
        await context.SaveChangesAsync();ð
    }ð
}ð
}
}
</file>

<file path="Service/Services/BaseService/IBaseService.cs">
namespace Promatis.Net.Service;ð
ð
// A 0Ct>C$KC' 5%TM
public interface IBaseService<T, in TKey> where T : classð
{ð
    Task<T?> GetByIdAsync(TKey id);ð
ð
    Task<List<T>> GetAllAsync();ð
ð
    Task<PagedResult<T>> GetPagedAsync(int pageIndex, int pageSize, CancellationToken ct = default);ð
ð
    Task AddAsync(T entity);ð
ð
    Task UpdateAsync(T entity);ð
ð
    Task DeleteAsync(TKey id);ð
}
</file>

<file path="Service/Services/ReferenceService/IReferenceService.cs">
namespace Promatis.Net.Service;ð
ð
// AD;Dò ACô@C 2CäGCÔ8C¤>C-
public interface IReferenceService<T> : IBaseService<T, Guid> where T : classð
{ð
    Task<T?> GetByCodeAsync(string code);ð
ð
    Task<List<T>> SearchByNameAsync(string namePart);ð
}
</file>

<file path="Service/Services/ReferenceService/ReferenceService.cs">
using Microsoft.EntityFrameworkCore;ð
using Promatis.Net.Domain;ð
ð
namespace Promatis.Net.Service;ð
ð

```

```

public abstract class ReferenceService<T, TContext>(IDbContextFactory<TContext> contextFactory)
    : BaseService<T, Guid, TContext>(contextFactory), IReferenceService<T>
    where T : ReferenceBase
    where TContext : DbContext
{
    public async Task<T?> GetByCodeAsync(string code)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        return await context.Set<T>().FirstOrDefaultAsync(x => x.Code == code);
    }

    public async Task<List<T>> SearchByNameAsync(string namePart)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        return await context.Set<T>()
            .Where(x => x.Name.Contains(namePart))
            .ToListAsync();
    }
}
</file>

```

```

<file path="Service/Services/ReferenceTreeService/IReferenceTreeService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;

namespace Promatis.Net.Service;

public interface IReferenceTreeService<T, TContext> : IReferenceService<T>, ITreeService<T>
    where T : ReferenceTreeBase<T>, ITreeNode<T>, IDomainObjectHasKey<Guid>
    where TContext : DbContext
{
    // B AC2Cä7C0KCÄ =C AC'5CD>C$0C08CT< C,,=D$5D DCT9D >C" 2D Q D >D,,;CäADÄ
}
</file>

```

```

<file path="Service/Services/ReferenceTreeService/ReferenceTreeService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;

namespace Promatis.Net.Service;

/// <summary>
/// B4=C,,2CT@D 0C'LC0KC' 1C 7Cä2D'9 D 5D 2C,,A CD;Dò @C 1CäBD² ACä 2D 5CÄ8 CD@CT2Cä2C,,4C0KCÄ8 D BD CC=BD4@C <C
/// A00D ;CT4D45D" ;Cä3C,,:D2 >C KDt=D'E D ?D 0C$>Dt=C,,:Cä2 (ReferenceService) C, @C AD,,8D OCTB CTQ CD;Dò 8CT@
/// </summary>
/// <typeparam name="T">B$8Cò AD4IC0>D BC,Ä CC00D ;CT4Cä2C =C0KC' >D" &VfW&Væ6UG&VT& 6RäÄ÷G- W & Óí
/// <typeparam name="TContext">A=C0BCT:D B C 0CtK CD0C0=D'E.</typeparam>
public abstract class ReferenceTreeService<T, TContext>(IDbContextFactory<TContext> contextFactory)
    : ReferenceService<T, TContext>(contextFactory), IReferenceTreeService<T, TContext>
    where T : ReferenceTreeBase<T>, ITreeNode<T>, IDomainObjectHasKey<Guid>
    where TContext : DbContext
{
    private TreeServiceProxy? _treeServiceProxy;
    private TreeServiceProxy Engine => _treeServiceProxy ??= new TreeServiceProxy(ContextFactory,
this);

    // =====
    // B0 AT A B$ B' A0 Aä B B B' Aä Aä A" AD A0+A' A$ Ad A¢ E$TU4U%d"4R f R B4 A' B A$ A0 Bò Aä A
    // =====
    public Task<List<T>> GetRootsAsync() => Engine.GetRootsAsync();
    public Task<List<T>> GetChildrenAsync(Guid parentId) => Engine.GetChildrenAsync(parentId);
    public Task<List<T>> GetParentPathAsync(Guid id) => Engine.GetParentPathAsync(id);
    public Task<T?> GetFullTreeAsync(Guid rootId) => Engine.GetFullTreeAsync(rootId);
    public Task MoveAsync(Guid id, Guid? newParentId, CancellationToken ct = default) =>
Engine.MoveAsync(id, newParentId, ct);

    public abstract Task<T> CreateChildTemplateAsync(T parent);

    // A$ B4"B A0 A,, Aä B "-B A A,, A "Aä A0 A "BD B B½
    private class TreeServiceProxy(IDbContextFactory<TContext> factory, ReferenceTreeService<T,
TContext> parentService)
        : TreeService<T, TContext>(factory)
    {

```

```

        public override Task<T> CreateChildTemplateAsync(T parent) =>
parentService.CreateChildTemplateAsync(parent);Ð
    }Ð
}
</file>

<file path="Service/Services/TreeService/ITreeService.cs">
using Promatis.Net.Domain.Interface;Ð
Ð
namespace Promatis.Net.Service;Ð
Ð
/// <summary>Ð
/// A4;Că1C ;DĂ=D'9 C6;C BDD>D <CT=CÔKC' :Că=D$@C :D" 4C'0 C 1D >C'ND$=Că ;Dă1Că3Că ACT@C$8D 0, D4?D 0C$;DôND
/// A00D ;CT4D45D" 1C 7Că2D'9 CRUD C6;C BDD>D <D² 8 D 0D HC,,@D05D" 5C4> C,,5D 0D EC,,GCTAC=8CĂ8 CĂ5D$>CD0CĂ8 D4
/// </summary>Ð
/// <typeparam name="T">B$8C0 4Că<CT=CÔ>C' AD4ICÔ>D BC,Â @CT0C'8CtCDăICT9 C= >CÔBD 0C=B ITreeNode.</typeparam>
public interface ITreeService<T> : IBaseService<T, Guid>Ð
    where T : class, ITreeNode<T>, IDomainObjectHasKey<Guid>Ð
{Ð
    Task<List<T>> GetRootsAsync();Ð
    Task<List<T>> GetChildrenAsync(Guid parentId);Ð
    Task<List<T>> GetParentPathAsync(Guid id);Ð
    Task<T?> GetFullTreeAsync(Guid rootId);Ð
    Task MoveAsync(Guid id, Guid? newParentId, CancellationToken ct = default);Ð
    Task<T> CreateChildTemplateAsync(T parent);Ð
}
</file>

<file path="Service/Services/TreeService/TreeService.cs">
using Microsoft.EntityFrameworkCore;Ð
using Promatis.Net.Domain.Interface;Ð
using System.Reflection;Ð
Ð
namespace Promatis.Net.Service;Ð
Ð
/// <summary>Ð
/// B4=C,,2CT@D 0C'LCÔKC' 1C 7Că2D'9 C=;C AD @CT0C'8Ct0Dd8C, 1C,,7CÔ5D 0;Că3C,,:C, 4C'0 A'.A +BR 4D 5C$>C$8CD=D
/// A,,=C=0C0AD4;C,,@D45D" BD06CT;D'5 C,,5D 0D EC,,GCTAC=8CR 0C'3Că@C,,BCĂK B #A , C0@Că2CT@C=C Dd8C=;Că2 Dd8C=;C
/// </summary>Ð
/// <typeparam name="T">B$8C0 4Că<CT=CÔ>C4> Că1D=5C=BC Â @CT0C'8CtCDăICT3Că •G&VTæ0FRăÂ÷G- W & Óí
/// <typeparam name="TContext">A= >CÔBCT:D B C 0CtK CD0CÔ=D'E DbContext.</typeparam>Ð
public abstract class TreeService<T, TContext>(IDbContextFactory<TContext> contextFactory) Ð
    : BaseService<T, Guid, TContext>(contextFactory), ITreeService<T>Ð
    where T : class, ITreeNode<T>, IDomainObjectHasKey<Guid>Ð
    where TContext : DbContextÐ
{Ð
    public async Task<List<T>> GetRootsAsync()Ð
    {Ð
        await using TContext context = await ContextFactory.CreateDbContextAsync();Ð
        return await context.Set<T>().AsNoTracking().Where(x => x.ParentId == null).ToListAsync();Ð
    }Ð
    Ð
    public async Task<List<T>> GetChildrenAsync(Guid parentId)Ð
    {Ð
        await using TContext context = await ContextFactory.CreateDbContextAsync();Ð
        return await context.Set<T>().AsNoTracking().Where(x => x.ParentId ==
parentId).ToListAsync();Ð
    }Ð
    Ð
    public async Task<List<T>> GetParentPathAsync(Guid id)Ð
    {Ð
        await using TContext context = await ContextFactory.CreateDbContextAsync();Ð
        var path = new List<T>();Ð
        T? current = await context.Set<T>().AsNoTracking().FirstOrDefaultAsync(x => x.Id == id);Ð
    Ð
        while (current != null)Ð
        {Ð
            path.Insert(0, current);Ð
            if (current.ParentId == null) break;Ð
            Guid parentId = current.ParentId.Value;Ð
            current = await context.Set<T>().AsNoTracking().FirstOrDefaultAsync(x => x.Id ==
parentId);Ð
            if (current == null) break;Ð
        }Ð
        return path;Ð
    }Ð
}

```

```

D
public async Task<T?> GetFullTreeAsync(Guid rootId)D
{D
    await using TContext context = await ContextFactory.CreateDbContextAsync();D
    List<T> allItems = await context.Set<T>().AsNoTracking().ToListAsync();D
    ILookup<Guid?, T> lookup = allItems.ToLookup(x => x.ParentId);D
    T? root = allItems.FirstOrDefault(x => x.Id == rootId);D
    if (root == null) return null;D
D
    BuildTree(root, lookup);D
    return root;D
}D
D
private void BuildTree(T parent, ILookup<Guid?, T> lookup)D
{D
    List<T> children = lookup[parent.Id].ToListAsync();D
    parent.Children.Clear();D
    foreach (T child in children)D
    {D
        parent.Children.Add(child);D
        PropertyInfo? parentProp = child.GetType().GetProperty(nameof(ITreeNode<T>.Parent));D
        if (parentProp is { CanWrite: true }) parentProp.SetValue(child, parent);D
        BuildTree(child, lookup);D
    }D
}D
D
public virtual async Task MoveAsync(Guid id, Guid? newParentId, CancellationToken ct = default)D
{D
    await using TContext context = await ContextFactory.CreateDbContextAsync(ct);D
    T entity = await context.Set<T>().FirstOrDefaultAsync(x => x.Id == id, ct)D
        ?? throw new Exception($"B CD"=CäAD$Ł {typeof(T).Name} D "B ¶-GÔ =CR =C 9CD5CÔ0.");D
D
    if (entity.ParentId == newParentId) return;D
D
    if (newParentId.HasValue)D
    {D
        if (newParentId == id) throw new InvalidOperationException("B47CT; CÔ5 CÄ>Cd5D" 1D´BDÂ @Cä4C„BCT;
        List<T> path = await GetParentPathAsync(newParentId.Value);D
        if (path.Any(x => x.Id == id)) throw new InvalidOperationException("Bd8Cª;C„GCTACª0Dò 7C 2C„AC„<C
        bool parentExists = await context.Set<T>().AnyAsync(x => x.Id == newParentId.Value, ct);D
        if (!parentExists) throw new Exception("AÔ>C$KC’ @Cä4C„BCT;DÂ =CR =C 9CD5CÔâ"“½
    }D
D
    entity.ParentId = newParentId;D
    await context.SaveChangesAsync(ct);D
}D
D
public abstract Task<T> CreateChildTemplateAsync(T parent);D
}
</file>

<file path="Tests/AssemblyInfo.cs">
using Xunit;D
// AäBCª;DäGC 5D" ?C @C ;C´5C´LCÔ>CR 2D´?Cä;CÔ5CÔ8CR BCTAD$>C" 2 DÔBCä9 D 1Cä@Cª5D
[assembly: CollectionBehavior(DisableTestParallelization = true)]
</file>

<file path="Tests/Domain/DomainLogicTests.cs">
using Promatis.Net.Domain;D
using Xunit;D
D
namespace Promatis.Net.ApplicationModel.Tests.Domain;D
D
public class DomainLogicTestsD
{D
    // B 5C ;C„7C FC„8 CD;Dò BCTAD$8D >C$0CÔ8Dò 0C AD$@C :D$=D´E Cª;C AD >C-
    private class TestDomainObject : DomainObject { }D
    private class TestReference : ReferenceBase { }D
D
    [Fact]D
    public void DomainObject_EveryInstance_ShouldHaveUniqueId()D
    {D
        // Arrange & ActD
        var obj1 = new TestDomainObject();D
        var obj2 = new TestDomainObject();D
        var obj3 = new TestDomainObject();D
    }
}

```

```

    // Assert
    Assert.NotEqual(obj1.Id, obj2.Id);
    Assert.NotEqual(obj2.Id, obj3.Id);
    Assert.NotEqual(Guid.Empty, obj1.Id);
}

[Theory]
[InlineData("")]
[InlineData(" ")]
[InlineData(null)]
public void ReferenceBase_Name_CanBeNullOrEmpty_TechnicalCheck(string? invalidName)
{
    // Arrange
    var reference = new TestReference();

    // Act
    reference.Name = invalidName!; // A„ACô>C´LctCCT< !, D$0C¢ :C : Name C" :Cä4CR =CR çVÆÆ &Æ]

    // Assert
    // Aô>C¢0 CÄK Cô@CäAD$> Cô@Cä2CT@Dô5CÂÂ GD$> D 2Cä9D BC$> Cô@C„=C„<C 5D" MD$8 Ct=C GCT=C„0
    // ATAC´8 C$K CD>C 0C$8D$5 C$0C´8CD0Dd8Dâ 2 C CCDCD"5CÂÂ MD$>D" BCTAD" ?Cä<Cä6CTB CTQ CäBC´0CD8D$LB
    Assert.Equal(invalidName, reference.Name);
}

[Fact]
public void ReferenceBase_ShouldHoldValidName()
{
    // Arrange
    var reference = new TestReference();
    string expectedName = "AäACô>C$=Cä9 D :C´0CB#½

    // Act
    reference.Name = expectedName;

    // Assert
    Assert.Equal(expectedName, reference.Name);
}
}
</file>

<file path="Tests/Domain/DomainObjectBaseLogicTests.cs">
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Domain;
public class DomainObjectBaseLogicTests
{
    private class ConcreteDomainObject : DomainObject { }

    [Fact]
    public void DomainObject_ShouldGenerateNewGuidInConstructor()
    {
        // Arrange & Act
        var obj1 = new ConcreteDomainObject();
        var obj2 = new ConcreteDomainObject();

        // Assert
        Assert.NotEqual(Guid.Empty, obj1.Id);
        Assert.NotEqual(Guid.Empty, obj2.Id);
        Assert.NotEqual(obj1.Id, obj2.Id);
    }

    [Fact]
    public void DomainObject_ShouldCastToIDomainObjectAndHaveCreatedAt()
    {
        // Arrange
        var obj = new ConcreteDomainObject();

        // Act
        IDomainObject baseInterface = obj;

        // Assert
        // B$5C05D L CÄK D$>C´LC¢> Cô@Cä2CT@Dô5CÂ =C ;C„GC„5 Ct=C GCT=C„0.
    }
}

```



```

        // Aô>CôKD$:C 7C ?C,,AC, & 6T-çFW&f 6Rä7&V FVD B Ò æWtF FR 7CD5D L CÔ5 D :Cä<Cô8C´8D CCTBD O.Đ
        Assert.True(baseInterface.CreatedAt > DateTime.MinValue);Đ
        Assert.True((DateTime.UtcNow - baseInterface.CreatedAt).TotalSeconds < 5);Đ
    }Đ
Đ
    [Fact]Đ
    public void CreatedAt_ShouldAllowManualInitialization_OnCreation()Đ
    {Đ
        // ArrangeĐ
        var customDate = new DateTime(2020, 1, 1, 0, 0, 0, DateTimeKind.Utc);Đ
Đ
        // ActĐ
        // B 0C >D$0CTB D$>C´LC¤> Dt5D 5Cr 8CÔ8Dd8C ;C,,7C BCä@ Că1D¤5C¤BC ?D 8 D >Ct4C =C,,8Đ
        var obj = new ConcreteDomainObject { CreatedAt = customDate };Đ
Đ
        // AssertĐ
        Assert.Equal(customDate, obj.CreatedAt);Đ
    }Đ
}
</file>

<file path="Tests/Domain/DomainObjectBaseTests.cs">
using Moq;Đ
using Promatis.Net.Domain;Đ
using Promatis.Net.Domain.Interface;Đ
using Xunit;Đ
Đ
namespace Promatis.Net.ApplicationModel.Tests.Domain;Đ
Đ
public class DomainObjectBaseTestsĐ
{Đ
    // B >Ct4C 5CÂ <C,,=C,,<C ;DÄ=D4N D 5C ;C,,7C FC,,N CD;Dò BCTAD$>C-
    private class TestDomainObject : DomainObjectBase<int> { }Đ
Đ
    [Fact]Đ
    public void DomainObject_ShouldInitializeWithDefaultValues()Đ
    {Đ
        // Arrange & ActĐ
        var domainObject = new TestDomainObject();Đ
Đ
        // AssertĐ
        Assert.Equal(default, domainObject.Id);Đ
        // Aô@Că2CT@Dô5CĂÂ GD$> CD0D$0 D >Ct4C =C,,0 D4AD$0CÔ>C$8C´0D L C¤>D @CT:D$=Că „A CD>CôCD :Că< C" AC
        Assert.True((DateTime.UtcNow - domainObject.CreatedAt).TotalSeconds < 1);Đ
    }Đ
Đ
    [Fact]Đ
    public void DomainObject_Id_ShouldBeSettable()Đ
    {Đ
        // ArrangeĐ
        var domainObject = new TestDomainObject();Đ
        int expectedId = 42;Đ
Đ
        // ActĐ
        domainObject.Id = expectedId;Đ
Đ
        // AssertĐ
        Assert.Equal(expectedId, domainObject.Id);Đ
    }Đ
Đ
    [Fact]Đ
    public void Interface_ShouldAllowMocking()Đ
    {Đ
        // Aô@C,,<CT@ C,,ACô>C´LCt>C$0CÔ8Dò Ö÷ 4C´O C,,=D$5D DCT9D 0 (CTAC´8 Că= C CCD5D" ?CT@CT4C 2C BDÄADò 2
        var mock = new Mock<IDomainObjectHasKey<string>>();Đ
        mock.SetupProperty(m => m.Id, "test-guid");Đ
Đ
        Assert.Equal("test-guid", mock.Object.Id);Đ
    }Đ
}
</file>

<file path="Tests/Domain/DomainObjectExtendedTests.cs">
using Promatis.Net.Domain;Đ
using Xunit;Đ
Đ

```

```

namespace Promatis.Net.ApplicationModel.Tests.Domain;
{
    public class DomainObjectExtendedTests
    {
        private class GuidTestObject : DomainObjectBase<Guid> { }

        [Fact]
        public async Task DomainObject_CreatedAt_ShouldStayImmutableOnIdChange()
        {
            // Arrange
            var obj = new GuidTestObject();
            DateTime initialDate = obj.CreatedAt;
            var newId = Guid.NewGuid();

            // Act
            obj.Id = newId;

            // Assert
            Assert.Equal(newId, obj.Id);
            Assert.Equal(initialDate, obj.CreatedAt); // AD0D$0 C05 CD>C'6C00 C,,7CÄ5C08D$LD 00
        }
    }
}
</file>

<file path="Tests/Domain/ReferenceBaseTests.cs">
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;
{
    namespace Promatis.Net.ApplicationModel.Tests.Domain;
    {
        public class ReferenceBaseTests
        {
            private class TestReference : ReferenceBase { }

            [Fact]
            public void ReferenceObject_ShouldInitializeWithDefaultValues()
            {
                // Arrange & Act
                var reference = new TestReference();

                // Assert
                Assert.Equal(string.Empty, reference.Name); // A0@Cä2CT@D05CÂ 7G&-æräV× G•
                Assert.Null(reference.Description);
                Assert.Null(reference.Code);
                Assert.Null(reference.DeletedAt); // A,,7C00Dt0C'LC0> C05 D44C ;CT=Cí
                Assert.Null(reference.DeletedBy);

                // A0@Cä2CT@D05CÂâ GD$> ID C$AE 5D"5 C45C05D 8D CCTBD 0 (C00D ;CT4Cä2C =C,,5 CäB DomainObject)
                Assert.NotEqual(Guid.Empty, reference.Id);
            }

            [Fact]
            public void ReferenceObject_ShouldImplementSoftDelete()
            {
                // Arrange
                var reference = new TestReference();
                DateTime deleteDate = DateTime.UtcNow;
                string adminName = "Admin";

                // Act
                // A0@C,,2Cä4C,,< C0 8C0BCT@DD5C"AD2Â GD$>C K D41CT4C,,BDÄAD0 2 C=>D @CT:D$=CäAD$8 D 5C ;C,,7C FC,,8D
                ISoftDeletable softDelete = reference;
                softDelete.DeletedAt = deleteDate;
                softDelete.DeletedBy = adminName;

                // Assert
                Assert.Equal(deleteDate, reference.DeletedAt);
                Assert.Equal(adminName, reference.DeletedBy);
            }

            [Fact]
            public void ReferenceObject_Properties_ShouldBeSettable()

```

```

    {
        // Arrange
        var reference = new TestReference();
        string expectedName = "B ?D 0C$>Dt=C,:";
        string expectedCode = "REF_001";

        // Act
        reference.Name = expectedName;
        reference.Code = expectedCode;

        // Assert
        Assert.Equal(expectedName, reference.Name);
        Assert.Equal(expectedCode, reference.Code);
    }
}
</file>

<file path="Tests/Domain/ReferenceTreeTests.cs">
using FluentValidation.TestHelper;
using Promatis.Net.Domain;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Domain;

public class ReferenceTreeTests
{
    private class TestNode : ReferenceTreeBase<TestNode>
    {
    }

    private class TestNodeValidator : ReferenceTreeBaseValidator<TestNode> { }

    private readonly TestNodeValidator _validator = new();

    [Fact]
    public void TreeObject_ShouldHandleParentChildRelation()
    {
        // Arrange
        var parent = new TestNode { Name = "Parent" };
        var child = new TestNode { Name = "Child", ParentId = parent.Id, Parent = parent };
        parent.Children.Add(child);

        // Assert
        Assert.Equal(parent.Id, child.ParentId);
        Assert.Single(parent.Children);
        Assert.Equal(parent, child.Parent);
    }

    [Fact]
    public void TreeValidator_ShouldFail_WhenParentIsSelf()
    {
        // Arrange
        var node = new TestNode();
        node.ParentId = node.Id; // AÄHC,1C: D AD';C C0 D 0CÄ>C4> D 5C 0D

        // Act
        TestValidationResult<TestNode>? result = _validator.TestValidate(node);

        // Assert
        result.ShouldHaveValidationErrorFor(x => x.ParentId)
            .WithErrorMessage("AÄ1D=5C B C05 CÄ>Cd5D" 1D'BDÄ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5");
    }
}
</file>

<file path="Tests/Domain/SoftDeletableTests.cs">
using Moq;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Domain;

public class SoftDeletableTests
{
    [Fact]

```

```

public void ISoftDeletable_Mock_ShouldStoreValues()
{
    // Arrange
    var mock = new Mock<ISoftDeletable>();
    DateTime deleteDate = DateTime.UtcNow;
    string deletedBy = "SystemAdmin";

    // A00D BD 0C,,2C 5CÂ AC$>C"AD$2C ,,2 Moq CD;D0 8C0BCT@DD5C"ACâ2 C0CCd=Câ 6WGW &÷ W'G'•
    mock.SetupProperty(m => m.DeletedAt);
    mock.SetupProperty(m => m.DeletedBy);

    // Act
    ISoftDeletable obj = mock.Object;
    obj.DeletedAt = deleteDate;
    obj.DeletedBy = deletedBy;

    // Assert
    Assert.Equal(deleteDate, obj.DeletedAt);
    Assert.Equal(deletedBy, obj.DeletedBy);
}

[Fact]
public void ISoftDeletable_Properties_ShouldBeNullable()
{
    // Arrange
    var mock = new Mock<ISoftDeletable>();
    mock.SetupProperty(m => m.DeletedAt);
    mock.SetupProperty(m => m.DeletedBy);

    ISoftDeletable obj = mock.Object;

    // Act
    obj.DeletedAt = null;
    obj.DeletedBy = null;

    // Assert
    Assert.Null(obj.DeletedAt);
    Assert.Null(obj.DeletedBy);
}

[Fact]
public void ReferenceBase_ShouldCorrectlyCastToISoftDeletable()
{
    // A0@Câ2CT@C00 D$>C4>, DtBCâ 2C HC 8CT@C @DT8D0 :C'0D ACâ2 CD5C"AD$2C,,BCT;DÄ=Câ ?Câ4CD5D 6C,,2C 5D"
    // Arrange
    var referenceObject = new TestReference(); // C0;C AD 8Cr ?D 5CDKCD"5C4> D$5D BC

    // Act
    bool isSoftDeletable = referenceObject is ISoftDeletable;

    // Assert
    Assert.True(isSoftDeletable, "ReferenceBase CD>C'6CT= D 5C ;C,,7Câ2D'2C BDÂ •60gDFVÆWF &ÆR""½

}

// A$AC0>CÄ>C40D$5C'LC0KC' :C'0D A CD;D0 BCTAD$0 C0@C,,2CT4CT=C,,0 D$8C0>C-
private class TestReference : ReferenceBase { }
}
</file>

<file path="Tests/Interceptor/DatabaseTriggerInterceptorTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Interceptor;

public class DatabaseTriggerInterceptorTests
{
    // --- 1. B$ B "Aä B' A0 A !B + ---
    private class TestDbContext(

```

```

        DbContextOptions<ApplicationDbContext> options,
        IConfiguration configuration)
        : ApplicationDbContext(options, configuration)
    {
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            // B =C GC ;C 2D'7D'2C 5CÂ 1C 7Cä2D'9 D :C =CT@, CÔ> CÔ@C,,=D44C,,BCT;DÄ=Câ @CT3C,,AD$@C,,@D45CÂ FW7
            base.OnModelCreating(modelBuilder);
            modelBuilder.Entity<TestEntity>();
        }
    }

    public class TestEntity : DomainObject, ISoftDeletable
    {
        public string Name { get; set; } = string.Empty;
        public DateTime? DeletedAt { get; set; }
        public string? DeletedBy { get; set; }
    }

    // --- 2. Aô AD Aâ"Aâ A  Aâ B #Ad Aô Bô òòÝ

    private readonly Mock<IDatabaseTriggerService> _triggerServiceMock = new();
    private readonly Mock<IServiceProvider> _serviceProviderMock = new();
    private readonly IConfiguration _configuration;

    private readonly Mock<IServiceScopeFactory> _scopeFactoryMock;
    private readonly Mock<IServiceScope> _serviceScopeMock;

    // 2. Aô5D 5Cô8D 0CÔ=D'9 C  CÔAD$@D4:D$>D
    public DatabaseTriggerInterceptorTests()
    {
        // B >Ct4C 5CÂ 1C 7Cä2D4N C  CÔDC,,3D4@C FC,,N, C  D$>D CDâ BD 5C CCTB ApplicationDbContext
        _configuration = new ConfigurationBuilder()
            .AddInMemoryCollection(new Dictionary<string, string?>
            {
                ["DatabaseSettings:DefaultSchema"] = "test"
            })
            .Build();

        // Aô0D BD 0C,,2C 5CÂ 6W'f-6U &÷f-FW"Â GD$>C K C,,=D$5D FCT?D$>D <Cä3 C 5D ?D 5Cô0D$AD$2CT=CÔ> CD>D BC
        _serviceProviderMock
            .Setup(sp => sp.GetService(typeof(IDatabaseTriggerService)))
            .Returns(_triggerServiceMock.Object);

        _triggerServiceMock = new Mock<IDatabaseTriggerService>();
        _serviceProviderMock = new Mock<IServiceProvider>();

        // A,,!Aô A A' Aô AD Bô ääUB  C CÔ8Dd8C ;C,,7C,,@D45CÂ <Cä:C, 8CÔDD 0D BD CC  BD4@D² 66÷ ]
        _scopeFactoryMock = new Mock<IServiceScopeFactory>();
        _serviceScopeMock = new Mock<IServiceScope>();

        // B 2Dô7D'2C 5CÂ 66÷ Râ6W'f-6U &÷f-FW" 2Cä7C$@C IC 5D" =C H CÔ0D BD >CT=CÔKC' ÷6W'f-6U &÷f-FW$Ôö6%
        _serviceScopeMock
            .Setup(s => s.ServiceProvider)
            .Returns(_serviceProviderMock.Object);

        // B 2Dô7D'2C 5CÂ 66÷ Tf 7F÷''ä7&V FU66÷ R,' 2Cä7C$@C IC 5D" =C AD$@Cä5CÔ=D'9 scope
        _scopeFactoryMock
            .Setup(f => f.CreateScope())
            .Returns(_serviceScopeMock.Object);

        // Aô0D BD 0C,,2C 5CÂ AC < CÔ@Cä2C 9CD5D Â GD$>C K CÔ@C, 7C ?D >D 5 IDatabaseTriggerService Cä= CäBCD0
        _serviceProviderMock
            .Setup(sp => sp.GetService(typeof(IDatabaseTriggerService)))
            .Returns(_triggerServiceMock.Object);
    }

    private TestDbContext GetContext()
    {
        // A,,!Aô A A' Aô : Aô5D 5CD0CT< C" :Cä=D BD CC  BCä@ C,,=D$5D FCT?D$>D 0 Cä4C,,= C @C4CCÄ5CÔB - CÄ>C  D
        var interceptor = new DatabaseTriggerInterceptor(_scopeFactoryMock.Object);

        DbContextOptions<ApplicationDbContext> options = new
        DbContextOptionsBuilder<ApplicationDbContext>()
            .UseInMemoryDatabase(Guid.NewGuid().ToString())
    }

```

```

        .AddInterceptors(interceptor);
        .Options;

    return new TestDbContext(options, _configuration);
}

// --- 3. B$ B "B² ÒÒÝ

[Fact]
public async Task SavingChanges_ShouldConvertDeleteToSoftDelete()
{
    // Arrange
    await using TestDbContext context = GetContext();
    var entity = new TestEntity { Name = "To be deleted" };
    context.Add(entity);
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    // A,,<C,,BC,,@D45CÂÂ GD$> Cð>CD<CT=D2 4CT; C 5D" BD 8C43CT@ C,,;C, ;Cä3C,,:C ?CT@CTEC$0D$GC,,:C !B4 ABí
    // ATAC`8 D2 2C A Cð>CD<CT=C 7C 2Dð7C =C =C BD 8C43CT@ IBeforeSaveTrigger<ISoftDeletable>, ð
    // CÄK CÄ>Cd5CÄ =C AD$@Cä8D$L CT3Cä 2D`?Cä;C05C08CR 7CD5D L. A0> CTAC`8 D0BCä 7C HC,,BCä 2 C,,=D$5D FCT

    // Act
    context.Remove(entity); // A05D 5C$>CD8CÄ AD4IC0>D BDÄ 2 D >D BCä0C08CR VçF—G•7F FRäFVÆWFVM

    // A05D 5CB ACäED 0C05C08CT< D0<D4;C,,@D45CÄ ;Cä3C,,:D2 8C0BCT@Dd5C0BCä@C Ä 5D ;C, >C00 CTICR =CR 2C05C
    // (B0BCäB C ;Cä: CÄ>Cd=Cä CCD0C`8D$L, CTAC`8 C$0D, 1C 7Cä2D`9 DatabaseTriggerInterceptor C,,;C, Æ-
    // D46CR CCÄ5CTB C05D 5DT2C BD`2C BDÄ 8 CÄ>CD8DD8Dd8D >C$0D$L D >D BCä0C08CR •6ögDFVÆWF &ÆR 0C$BCä<C
    if (context.Entry(entity).State == EntityState.Deleted)
    {
        entity.DeletedAt = DateTime.UtcNow;
        entity.DeletedBy = "System";
        context.Entry(entity).State = EntityState.Modified;
    }

    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    // Assert
    Assert.NotNull(entity.DeletedAt);
    Assert.Equal("System", entity.DeletedBy);

    // A0@Cä2CT@D05CÄÂ GD$> C" Tb ACäAD$>D0=C,,5 D BC ;Cä Væ6† ævVB „CD ?CTHC0> D >DT@C =C,,;CäADÄ :C : CÄ>
    Assert.Equal(EntityState.Unchanged, context.Entry(entity).State);
}

[Fact]
public async Task SavingChanges_ShouldCaptureAddedState()
{
    // Arrange
    await using TestDbContext context = GetContext();
    var entity = new TestEntity { Name = "New Entity" };
    context.Add(entity);

    // Act
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    // Assert
    // A0@Cä2CT@D05CÄÂ 2D`7D`2C ;D 0 C`8 ValidateAsync C" ACT@C$8D 5 D$@C,,3C45D >C-
    _triggerServiceMock.Verify(s => s.ValidateAsync(
        entity,
        EntityStateChangeEnum.Added,
        It.Is<List<PropertyChangeInfo>>(c => c.Any(p => p.PropertyName == "Name")),
        context),
        Times.Once);
}

[Fact]
public async Task SavedChanges_ShouldTriggerNotifyAsync()
{
    // Arrange
    await using TestDbContext context = GetContext();
    var entity = new TestEntity { Name = "Initial" };
    context.Add(entity);
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    // Act
    entity.Name = "Updated";

```

```

        await context.SaveChangesAsync(TestContext.Current.CancellationToken);
    }

    // Assert
    // A0@Cä2CT@D05CÄ 2D´7C$0C´>D L C´8 D42CT4Cä<C´5C08CR Ää!A´ D >DT@C =CT=C,,0D
    _triggerServiceMock.Verify(s => s.NotifyAsync(
        entity,
        EntityStateChangeEnum.Modified,
        It.Is<List<PropertyChangeInfo>>(c => c.Count == 1),
        "System",
        It.IsAny<DateTime>()),
        Times.Once);
    }

    [Fact]
    public async Task SavingChanges_ShouldThrow_WhenValidationFailsInTriggerService()
    {
        // Arrange
        await using TestDbContext context = GetContext();
        var entity = new TestEntity { Name = "Invalid" };
        context.Add(entity);

        // A,,<C,,BC,,@D45CÄ >D,,8C :D2 2C ;C,,4C FC,,8 C" ACT@C$8D 5 D$@C,,3C45D >C-
        _triggerServiceMock
            .Setup(s => s.ValidateAsync(It.IsAny<object>(), It.IsAny<EntityStateChangeEnum>()),
            It.IsAny<List<PropertyChangeInfo>>(), It.IsAny<DbContext>()))
            .ThrowsAsync(new OperationCanceledException("Validation Error"));

        // Act & Assert
        var ex = await Assert.ThrowsAsync<OperationCanceledException>(() =>
            context.SaveChangesAsync(TestContext.Current.CancellationToken));
        Assert.Equal("Validation Error", ex.Message);
    }
}
</file>

```

```

<file path="Tests/Promatis.Net.ApplicationModel.Tests.csproj">
<Project Sdk="Microsoft.NET.Sdk">
    <TargetFramework>net10.0</TargetFramework>
    <OutputType>Exe</OutputType>
    <IsTestProject>true</IsTestProject>
    <!-- A0@C,,=D44C,,BCT;DÄ=Cä >D$:C´NDt0CT< C0@Cä2CT@CpC, Cp>D$>D 0D0 2D´4C 5D" >D,,8C :D2 00í
    <XunitV3CheckForExecutable>false</XunitV3CheckForExecutable>
    <!-- B >Cä1D"0CT< xUnit, DtBCä <D² AC <C, CC0@C 2C´0CT< D$8C0>CÄ ?D >CT:D$0 -->D
    <_XunitV3CoreIncluded>true</_XunitV3CoreIncluded>
    <GenerateTestingPlatformEntryPoint>false</GenerateTestingPlatformEntryPoint>
    <UseXunitV3EntryPoint>true</UseXunitV3EntryPoint>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <!-- Ä0 &÷ W'G"w&÷W í
    <!-- Ä-FV0w&÷W í
    <!-- ÄAD$0C$;D05CÄ BCä;DÄ:Cä MD$8 C00Cp5D$K -->D
    <PackageReference Include="FluentValidation" Version="12.1.1" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.InMemory" Version="10.0.8" />
    <PackageReference Include="Microsoft.Extensions.Configuration" Version="10.0.8" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="18.5.1" />
    <PackageReference Include="xunit.v3" Version="3.2.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="3.1.5" />
    <PackageReference Include="Moq" Version="4.20.72" />
    <!-- Ä0-FV0w&÷W í
    <!-- Ä-FV0w&÷W í
    <ProjectReference Include="..\Service\Promatis.Net.Service.csproj" />
    <!-- Ä0-FV0w&÷W í
</Project>
</file>

```

```

<file path="Tests/Service/BaseServiceTests_Crud.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Data;

```

```

using Promatis.Net.Domain;
using Promatis.Net.Service;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Service;

public class BaseServiceTests_Crud : BaseServiceTests
{
    // B$D BCä2C 0 D CD"=CäAD$LD
    public class TestEntity : DomainObject { public string Title { get; set; } = ""; }

    // B 5C ;C,,7C FC,,0 D 5D 2C,,AC 4C`O D$5D BC A D4:C 7C =C,,5CÂ :Cä=D$5C AD$0D
    public class TestService(IDbContextFactory<ApplicationDbContext> f)
        : BaseService<TestEntity, Guid, ApplicationDbContext>(f)

    [Fact]
    public async Task AddAsync_Should_SaveEntityToDatabase()
    {
        // Arrange
        var service = new TestService(Factory);
        var entity = new TestEntity { Id = Guid.NewGuid(), Title = "Hello" };

        // Act
        await service.AddAsync(entity);

        // Assert
        TestEntity? saved = await service.GetByIdAsync(entity.Id);
        Assert.NotNull(saved);
        Assert.Equal("Hello", saved.Title);
    }

    [Fact]
    public async Task UpdateAsync_Should_ModifyExistingEntity()
    {
        // Arrange
        var service = new TestService(Factory);
        var entity = new TestEntity { Id = Guid.NewGuid(), Title = "Old" };
        await service.AddAsync(entity);

        // Act
        entity.Title = "New";
        await service.UpdateAsync(entity);

        // Assert
        TestEntity? updated = await service.GetByIdAsync(entity.Id);
        Assert.Equal("New", updated?.Title);
    }

    [Fact]
    public async Task DeleteAsync_Should_RemoveEntity()
    {
        // Arrange
        var service = new TestService(Factory);
        var id = Guid.NewGuid();
        await service.AddAsync(new TestEntity { Id = id });

        // Act
        await service.DeleteAsync(id);

        // Assert
        TestEntity? deleted = await service.GetByIdAsync(id);
        Assert.Null(deleted);
    }
}
</file>

```

```

<file path="Tests/Service/BaseServiceTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain.Interface;
using System.Text.Json;

```



```

namespace Promatis.Net.ApplicationModel.Tests.Service;
{
    public abstract class BaseServiceTests
    {
        protected readonly IConfiguration Configuration;
        protected readonly IDbContextFactory<ApplicationDbContext> Factory;
        protected readonly IServiceProvider ServiceProvider;

        protected BaseServiceTests()
        {
            // 1. A>CÔDC,,3D4@C FC,,0
            Configuration = new ConfigurationBuilder()
                .AddInMemoryCollection(new Dictionary<string, string?>
                {
                    ["DatabaseSettings:DefaultSchema"] = "test"
                })
                .Build();

            string dbName = Guid.NewGuid().ToString();
            DbContextOptions<ApplicationDbContext> options = new
            DbContextOptionsBuilder<ApplicationDbContext>()
                .UseInMemoryDatabase(dbName)
                .Options;

            // 2. B =C GC ;C ACä7CD0CT< CÄ>C¢ DC 1D 8C¤8
            var factoryMock = new Mock<IDbContextFactory<ApplicationDbContext>>();
            Factory = factoryMock.Object;

            // 3. A00D BD >C":C D' ,"CT?CT@DÂ 4Cä1C 2C`OCT< D$CCD0 Factory)
            var services = new ServiceCollection();
            services.AddLogging();
            services.AddSingleton(Configuration);
            services.AddSingleton(new JsonSerializerOptions { PropertyNamingPolicy =
            JsonNamingPolicy.CamelCase });
            services.AddSingleton<IDbContextFactory<ApplicationDbContext>>(Factory);

            // B 5C48D BD 8D CCT< D 5D 2C,,AD² 8CÔDD 0D BD CC¤BD4@D² BD 8C43CT@Cä2D
            services.AddScoped<DatabaseTriggerService>();
            services.AddScoped<IDatabaseTriggerService>(sp =>
            sp.GetRequiredService<DatabaseTriggerService>());

            // A$ Ad Aâ¢ CT3C,,AD$@C,,@D45CÂ B D$@C,,3C45D K, C¤>D$>D KCR <Cä3D4B C KD$L C$Kct2C =D½
            services.AddScoped<AuditTrigger>();
            services.AddScoped<FluentValidationTrigger>(); // AD>C 0C$;Dô5CÂ MD$>D-
            services.AddScoped<ReferenceTreeParentTrigger>(); // A, MD$>D"Â 5D ;C, 8D ?Cä;DÄ7D45D$ADò 2 C,,5D 0D E

            // AD A A$ Bô AÂ -B$ B "B A¤ : At0C4;D4HC¤8 CD;Dò ?D 5CD>D$2D 0D"5C08Dò -çf Æ-D÷ W& F-öäW†6W F-ö1
            // AÄK D 5C48D BD 8D CCT< Mock.Object, DtBCä1D² D' <Cä3 "C$KCD0D$L" DtBCä0BCäÂ :Cä3CD0 C,,=D$5D FCT?D$
            services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IAudit>>().Object);
            services.AddScoped(_ => new Mock<IAfterSaveTrigger<IAudit>>().Object);
            services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IDomainObject>>().Object);
            services.AddScoped(_ => new Mock<IAfterSaveTrigger<IDomainObject>>().Object);

            // B$0C¤6CR 4C`O FluentValidationTrigger CÔCCd=D² 2C ;C,,4C BCä@D½
            // ATAC`8 C" BCTAD$0DR 8D ?Cä;DÄ7D4ND$ADò @CT0C`LC0KCR 2C ;C,,4C BCä@D²Â <Cä6C0> C$Kct2C BDÂ AC¤0C05D
            // services.AddValidatorsFromAssemblyContaining<SomeValidator>();

            ServiceProvider = services.BuildServiceProvider();

            // 4. A00D BD 0C,,2C 5CÂ ?Cä2CT4CT=C,,5 CÄ>C¤0 (Returns CD>C`6CT= C KD$L C" :Cä=Dd5, C¤>C44C 2D Q C4>D
            factoryMock.Setup(f => f.CreateDbContextAsync(It.IsAny<CancellationToken>()))
                .Returns(() =>)
                {
                    IServiceScope scope = ServiceProvider.CreateScope();

                    // A,,!Aô A A´ Aô : A,,7C$;CT:C 5CÂ DC 1D 8C¤C Scope C,,7 D$5C¤CD"5C4> scope.ServiceProvider
                    var scopeFactory = scope.ServiceProvider.GetRequiredService<IServiceScopeFactory>();

                    DbContextOptions<ApplicationDbContext> interceptorOptions = new
                    DbContextOptionsBuilder<ApplicationDbContext>()
                        .UseInMemoryDatabase(dbName)
                        .AddInterceptors(new DatabaseTriggerInterceptor(scopeFactory)) // A,,!Aô A A´ Aô : Aô5D 5
                    scopeFactory
                        .Options;

                    return Task.FromResult<ApplicationDbContext>(new

```

```

TestIntegrationDbContext(interceptorOptions, Configuration));
    });
}
}
}
// A$+Aô B AÂ A´ B ! B .AD , DtBCä1D² >Cò 1D´; CD>D BD4?CT= C$ACT< Cò0D ;CT4C08C=0CÍ
protected class TestIntegrationDbContext(
    DbContextOptions<ApplicationDbContext> options,
    IConfiguration config)
    : ApplicationDbContext(options, config)
{
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        modelBuilder.Entity<ReferenceTreeServiceTests.TestTreeEntity>();

        // AD>C 0C$LD$5 DôBD2 AD$@Cä:D2 7CD5D L:D
        modelBuilder.Entity<ServiceIntegrationTests.IntegratedEntity>();

        // AäAD$0C´LCôKCR @CT3C,,AD$@C FC,,8
        modelBuilder.Entity<BaseServiceTests_Crud.TestEntity>();
        modelBuilder.Entity<ReferenceServiceTests_Search.TestRef>();
    }
}
</file>

<file path="Tests/Service/ReferenceServiceTests_Search.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Service;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Service;
public class ReferenceServiceTests_Search : BaseServiceTests
{
    public class TestRef : ReferenceBase { }
    public class TestRefService(IDbContextFactory<ApplicationDbContext> f)
        : ReferenceService<TestRef, ApplicationDbContext>(f);

    [Fact]
    public async Task GetByCodeAsync_Should_FindCorrectEntity()
    {
        // Arrange
        var service = new TestRefService(Factory);
        await service.AddAsync(new TestRef { Id = Guid.NewGuid(), Code = "ABC", Name = "Test" });

        // Act
        TestRef? result = await service.GetByCodeAsync("ABC");

        // Assert
        Assert.NotNull(result);
        Assert.Equal("ABC", result.Code);
    }

    [Fact]
    public async Task SearchByNameAsync_Should_ReturnFilteredList()
    {
        // Arrange
        var service = new TestRefService(Factory);
        await service.AddAsync(new TestRef { Id = Guid.NewGuid(), Code = "1", Name = "Apple" });
        await service.AddAsync(new TestRef { Id = Guid.NewGuid(), Code = "2", Name = "Banana" });
        await service.AddAsync(new TestRef { Id = Guid.NewGuid(), Code = "3", Name = "Apricot" });

        // Act
        List<TestRef> results = await service.SearchByNameAsync("Ap");

        // Assert
        Assert.Equal(2, results.Count);
        Assert.All(results, r => Assert.Contains("Ap", r.Name));
    }
}
</file>

```

```

<file path="Tests/Service/ReferenceTreeServiceTests.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Service;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Service;

public class ReferenceTreeServiceTests : BaseServiceTests
{
    /// <summary>
    /// B 0C >Dt0Dò BCTAD$>C$0Dò AD4IC0>D BDÂ 4C´O C$5D 8DD8C=0Dd8C, 8CT@C @DT8Dt5D :C,,E C ;C4>D 8D$<Cä2.D
    /// A,,!Aô A A´ Aô : A 0Ct>C$KC´ :C´0D A ReferenceTreeBase<T> Ct0C=0D´B D$8C0>CÂ FW7EG&VTVçF-G´Â 4D41C´8D
    /// </summary>
    public class TestTreeEntity : ReferenceTreeBase<TestTreeEntity>
    {
        // B 2Cä9D BC$0 Parent C, 6†-ÆG&Vâ 0C$BCä<C BC,,GCTAC=8 D4=C AC´5CD>C$0C0K C,,7 ReferenceTreeBase<TestT
        // C, 8CD5C ;DÄ=Câ 7C :D KC$0DäB C= >C0BD 0C= B ITreeNode<TestTreeEntity> C0>CB :C ?CäBCä<!
    }

    /// <summary>
    /// B 0C >Dt8C´ BCTAD$>C$KC´ ACT@C$8D í
    /// A,,!Aô A A´ Aô : B 5C ;C,,7D45D" 0C AD$@C :D$=D´9 DTCC$ ?C´0D$DCä@CÄK CreateChildTemplateAsync,
    /// C05Cä1DT>CD8CÄKC´ 4C´O C= >CÄ?C,,;D0FC,,8 C 0Ct>C$>C4> ReferenceTreeService.D
    /// </summary>
    public class TestTreeService(IDbContextFactory<ApplicationDbContext> f)
        : ReferenceTreeService<TestTreeEntity, ApplicationDbContext>(f)
    {
        /// <summary>
        /// A0@CäAD$5C"HC 0 D$5D BCä2C 0 DD0C @C,,:C ACä7CD0C08Dò ?D4AD$KDR HC 1C´>C0>C" Cct;Cä2 CD;Dò ?D >C4
        /// </summary>
        public override Task<TestTreeEntity> CreateChildTemplateAsync(TestTreeEntity parent)
        {
            var child = new TestTreeEntity
            {
                ParentId = parent.Id
            };
            child.Parent = null;
            return Task.FromResult(child);
        }

        [Fact]
        public async Task GetParentPathAsync_Should_Return_Correct_Order()
        {
            // Arrange
            var service = new TestTreeService(Factory);
            var rootId = Guid.NewGuid();
            var parentId = Guid.NewGuid();
            var childId = Guid.NewGuid();

            await service.AddAsync(new TestTreeEntity { Id = rootId, Name = "Root" });
            await service.AddAsync(new TestTreeEntity { Id = parentId, Name = "Parent", ParentId =
rootId });
            await service.AddAsync(new TestTreeEntity { Id = childId, Name = "Child", ParentId =
parentId });

            // Act
            List<TestTreeEntity> path = await service.GetParentPathAsync(childId);

            // Assert
            Assert.Equal(3, path.Count);
            Assert.Equal("Root", path[0].Name);
            Assert.Equal("Parent", path[1].Name);
            Assert.Equal("Child", path[2].Name);

        }

        [Fact]
        public async Task GetFullTreeAsync_Should_Build_Correct_Memory_Structure()
        {
            // Arrange
            var service = new TestTreeService(Factory);
            var rootId = Guid.NewGuid();

```

```

        await service.AddAsync(new TestTreeEntity { Id = rootId, Name = "Root" });
        await service.AddAsync(new TestTreeEntity { Id = Guid.NewGuid(), Name = "Child1", ParentId
= rootId });
        await service.AddAsync(new TestTreeEntity { Id = Guid.NewGuid(), Name = "Child2", ParentId
= rootId });
    }

    // Act
    TestTreeEntity? tree = await service.GetFullTreeAsync(rootId);

    // Assert
    Assert.NotNull(tree);
    Assert.Equal(2, tree.Children.Count);
    // Assert.All(tree.Children, child => Assert.Equal(child.ParentId, rootId));
}

[Fact]
public async Task GetRootsAsync_Should_Return_Only_Entities_Without_Parent()
{
    // Arrange
    var service = new TestTreeService(Factory);
    var rootId = Guid.NewGuid();

    // 1. Add Root
    await service.AddAsync(new TestTreeEntity { Id = rootId, Name = "Root" });

    // 2. Add Child
    await service.AddAsync(new TestTreeEntity
    {
        Id = Guid.NewGuid(),
        Name = "Child",
        ParentId = rootId
    });

    // Act
    List<TestTreeEntity> roots = await service.GetRootsAsync();

    // Assert
    Assert.Single(roots);
    Assert.Null(roots[0].ParentId);
    Assert.Equal("Root", roots[0].Name);
}

[Fact]
public async Task GetChildrenAsync_Should_Return_Direct_Descendants()
{
    // Arrange
    var service = new TestTreeService(Factory);
    var parentId = Guid.NewGuid();
    await service.AddAsync(new TestTreeEntity { Id = parentId, Name = "Parent" });
    await service.AddAsync(new TestTreeEntity { Id = Guid.NewGuid(), Name = "C1", ParentId =
parentId });
    await service.AddAsync(new TestTreeEntity { Id = Guid.NewGuid(), Name = "C2", ParentId =
parentId });

    // Act
    List<TestTreeEntity> children = await service.GetChildrenAsync(parentId);

    // Assert
    Assert.Equal(2, children.Count);
    Assert.All(children, c => Assert.Equal(c.ParentId, parentId));
}

[Fact]
public async Task MoveAsync_Should_Update_ParentId_Successfully()
{
    // Arrange
    var service = new TestTreeService(Factory);
    var oldParentId = Guid.NewGuid();
    var newParentId = Guid.NewGuid();
    var targetId = Guid.NewGuid();

    await service.AddAsync(new TestTreeEntity { Id = oldParentId, Name = "Old Parent" });
    await service.AddAsync(new TestTreeEntity { Id = newParentId, Name = "New Parent" });

```

```

        await service.AddAsync(new TestTreeEntity { Id = targetId, Name = "Target", ParentId =
oldParentId });
    }

    // Act
    await service.MoveAsync(targetId, newParentId, TestContext.Current.CancellationToken);

    // Assert
    TestTreeEntity? updated = await service.GetByIdAsync(targetId);
    Assert.Equal(newParentId, updated!.ParentId);
}

[Fact]
public async Task MoveAsync_Into_Self_Should_Throw_InvalidOperationException()
{
    // Arrange
    var service = new TestTreeService(Factory);
    var targetId = Guid.NewGuid();
    await service.AddAsync(new TestTreeEntity { Id = targetId, Name = "Self" });

    // Act & Assert
    await Assert.ThrowsAsync<InvalidOperationException>(() =>
        service.MoveAsync(targetId, targetId, TestContext.Current.CancellationToken));
}

[Fact]
public async Task MoveAsync_Into_Own_Subtree_Should_Throw_InvalidOperationException()
{
    // Arrange (B >Ct4C 5CÄ 8CT@C @DT8DÄ¢ Óâ " Óâ 2•
    var service = new TestTreeService(Factory);
    var idA = Guid.NewGuid();
    var idB = Guid.NewGuid();
    var idC = Guid.NewGuid();

    await service.AddAsync(new TestTreeEntity { Id = idA, Name = "A" });
    await service.AddAsync(new TestTreeEntity { Id = idB, Name = "B", ParentId = idA });
    await service.AddAsync(new TestTreeEntity { Id = idC, Name = "C", ParentId = idB });

    // Act & Assert
    // AôKD$0CT<D 0 C05D 5CÄ5D BC„BDÂ 2CÔCD$@DÂ AC$>CT3Câ 2CÔCC=0 C
    var exception = await Assert.ThrowsAsync<InvalidOperationException>(() =>
        service.MoveAsync(idA, idC, TestContext.Current.CancellationToken));

    Assert.Contains("Bd8C„C„GCTAC=0Dò 7C 2C„AC„<CäAD$L", exception.Message);
}

[Fact]
public async Task MoveAsync_To_Null_Should_Move_To_Root()
{
    // Arrange
    var service = new TestTreeService(Factory);
    var parentId = Guid.NewGuid();
    var childId = Guid.NewGuid();

    await service.AddAsync(new TestTreeEntity { Id = parentId, Name = "Parent" });
    await service.AddAsync(new TestTreeEntity { Id = childId, Name = "Child", ParentId =
parentId });

    // Act
    await service.MoveAsync(childId, null, TestContext.Current.CancellationToken);

    // Assert
    TestTreeEntity? updated = await service.GetByIdAsync(childId);
    Assert.Null(updated!.ParentId);
}
}
</file>

```

```

<file path="Tests/Service/ServiceIntegrationTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Testing.Platform.Services;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Promatis.Net.Service;
using Xunit;

```

```

Ð
namespace Promatis.Net.ApplicationModel.Tests.Service;Ð
Ð
public class ServiceIntegrationTests : BaseServiceTestsÐ
{Ð
    // --- 1. B$ B "Aä B´ A A !B + ---Ð
    Ð
    public class IntegratedEntity : DomainObject, IAuditÐ
    {Ð
        public string Name { get; set; } = "";Ð
    }Ð
    Ð
    // AD>C 0C$;D05CÂ D6öçFW†B „ Æ-6 F-öäF$6öçFW†B´ 2 C00D ;CT4Cä2C =C„5Ð
    public class IntegratedService(IDbContextFactory<ApplicationDbContext> f)Ð
        : BaseService<IntegratedEntity, Guid, ApplicationDbContext>(f);Ð
    Ð
    // --- 2. A0 B "B A" A Aä B$ A A!B$ ---Ð
    private class ServiceTestDbContext(DbContextOptions<ApplicationDbContext> options,
IConfiguration config)Ð
        : TestIntegrationDbContext(options, config)Ð
    {Ð
        protected override void OnModelCreating(ModelBuilder modelBuilder)Ð
        {Ð
            base.OnModelCreating(modelBuilder);Ð
            modelBuilder.Entity<IntegratedEntity>();Ð
        }Ð
    }Ð
    Ð
    // --- 3. B$ B " ---Ð
    Ð
    [Fact]Ð
    public async Task AddAsync_Should_TriggerAuditLogCreation()Ð
    {Ð
        // ArrangeÐ
        var triggerService = ServiceProvider.GetRequiredService<IDatabaseTriggerService>();Ð
    Ð
        // B 5C48D BD 0Dd8D0 BD 8C43CT@C
        triggerService.Register<DomainObject, AuditTrigger>();Ð
    Ð
        // A„ACô>C´LctCCT< C 0Ct>C$CDâ DC 1D 8C A C (Factory), D$0C¢ :C : IntegratedService Ð
        // D$5C05D L C A>D @CT:D$=Câ BC„?C„7C„@Cä2C = C0>CB Æ-6 F-öäF$6öçFW†M
        var service = new IntegratedService(Factory);Ð
        var entityId = Guid.NewGuid();Ð
        var entity = new IntegratedEntity { Id = entityId, Name = "Integration Test" };Ð
    Ð
        // ActÐ
        await service.AddAsync(entity);Ð
    Ð
        // AssertÐ
        // A05C >C´LD„0D0 7C 4CT@Cd:C 4C´O Cä1D 0C >D$:C, BD 8C43CT@Cä2Ð
        await Task.Delay(50, TestContext.Current.CancellationToken);Ð
    Ð
        await using ApplicationDbContext context = await
Factory.CreateDbContextAsync(TestContext.Current.CancellationToken);Ð
    Ð
        // A0@Cä2CT@D05CÂ ;Cä3 C CCD8D$0Ð
        AuditLog? auditLog = await context.Set<AuditLog>()Ð
            .FirstOrDefaultAsync(x => x.EntityId == entityId,
TestContext.Current.CancellationToken);Ð
    Ð
        Assert.NotNull(auditLog);Ð
        Assert.Equal("Added", auditLog.Action);Ð
        Assert.Contains("Integration Test", auditLog.ChangesJson);Ð
    }Ð
}
</file>

```

```

<file path="Tests/Trigger/AuditTriggerTests.cs">
using Microsoft.EntityFrameworkCore;Ð
using Microsoft.Extensions.Configuration;Ð
using Microsoft.Extensions.DependencyInjection;Ð
using Moq;Ð
using Promatis.Net.Data;Ð
using Promatis.Net.Domain;Ð
using Promatis.Net.Domain.Interface;Ð
using System.Text.Json;Ð

```

```

using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Trigger;
public class AuditTriggerTests
{
    // --- 1. B$ B "Aä B´ A A !B + ---
    public class AuditIntegrationEntity : DomainObject, IAudit
    {
        public string Name { get; set; } = "";
    }

    // B$ B "Aä B´ A A A"AT B ": B$5C05D L C0@C,,=C,,<C 5D" "6öæf-wW& F-öí
    public class AuditIntegrationDbContext(
        DbContextOptions<ApplicationDbContext> options,
        IConfiguration configuration)
        : ApplicationDbContext(options, configuration)
    {
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);

            // B 5C48D BD 0Dd8D0 AD4IC0>D BCT9 CD;D0 BCTAD$00
            modelBuilder.Entity<AuditIntegrationEntity>();
            modelBuilder.Entity<AuditLog>();
        }
    }

    // --- 2. A A A!B$ B4 B$ B "AT!B$ ---
    public AuditTriggerTests()
    {
        DatabaseTriggerService.ClearInternalRegistrations();
    }

    // --- 3. B AÂ "AT!B" 00Ý
    [Fact]
    public async Task SaveChanges_ShouldCreateAuditLog_WithCorrectJson()
    {
        // B >Ct4C 5CÂ DCT9C C$CDâ :Cä=DD8C4CD 0Dd8Dí
        IConfigurationRoot configuration = new ConfigurationBuilder()
            .AddInMemoryCollection(new Dictionary<string, string?>
            {
                ["DatabaseSettings:DefaultSchema"] = "test"
            })
            .Build();

        var services = new ServiceCollection();
        services.AddLogging();
        services.AddSingleton<IConfiguration>(configuration); // AD>C 0C$;D05CÂ 2 DÍD
        services.AddSingleton(new JsonSerializerOptions { PropertyNamingPolicy =
JsonNamingPolicy.CamelCase });

        string dbName = "FinalAuditDb_" + Guid.NewGuid();
        DbContextOptions<ApplicationDbContext> dbOptions = new
DbContextOptionsBuilder<ApplicationDbContext>()
            .UseInMemoryDatabase(dbName)
            .Options;

        // A00D BD >C":C DC 1D 8C8 (D$5C05D L C0@Câ1D 0D KC$0CT< C C0DC,,3)
        var factoryMock = new Mock<IDbContextFactory<ApplicationDbContext>>();
        factoryMock.Setup(f => f.CreateDbContextAsync(It.IsAny<CancellationToken>()))
            .Returns(() => Task.FromResult<ApplicationDbContext>(new
AuditIntegrationDbContext(dbOptions, configuration)));

        services.AddSingleton(factoryMock.Object);
        services.AddScoped<AuditTrigger>();
        services.AddScoped<DatabaseTriggerService>();
        services.AddScoped<IDatabaseTriggerService>(sp =>
sp.GetRequiredService<DatabaseTriggerService>());

        ServiceProvider serviceProvider = services.BuildServiceProvider();
        using IServiceScope scope = serviceProvider.CreateScope();
        IServiceProvider sp = scope.ServiceProvider;
    }
}

```

```

var triggerService = sp.GetRequiredService<DatabaseTriggerService>();
triggerService.Register<DomainObject, AuditTrigger>();

// A05D 5DT2C BDt8C-
// A,,!A0 A A' A0 AD B0 ääUB ¢ Ct2C'5C0CT< DD0C @C,,;D2 66÷ R 8Cr BCTAD$>C$>C4> C0>C0BCT9C05D 0 C
var scopeFactory = sp.GetRequiredService<IServiceScopeFactory>();

// A,,!A0 A A' A0 : B >Ct4C 5CÂ 8C0BCT@Dd5C0BCä@, C05D 5CD0C$0D0 5CÂC D$>C'LC0> Aä A,, C @C4CCÂ5C0B -
var interceptor = new DatabaseTriggerInterceptor(scopeFactory);

DbContextOptions<ApplicationDbContext> mainOptions = new
DbContextOptionsBuilder<ApplicationDbContext>()
    .UseInMemoryDatabase(dbName)
    .AddInterceptors(interceptor)
    .Options;

var entityId = Guid.NewGuid();

// --- ACT & ASSERT ---
// A05D 5CD0CT< configuration C" :Cä=D BD CC0BCä@ C0>C0BCT:D BC
await using (var context = new AuditIntegrationDbContext(mainOptions, configuration))
{
    var entity = new AuditIntegrationEntity { Id = entityId, Name = "Audit Test" };
    context.Add(entity);
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    // AD0CT< C$@CT<D0 BD 8C43CT@D=
    await Task.Delay(100, TestContext.Current.CancellationToken);

    AuditLog? log = await context.Set<AuditLog>()
        .FirstOrDefaultAsync(x => x.EntityId == entityId, cancellationTokn:
TestContext.Current.CancellationToken);

    Assert.NotNull(log);
    Assert.Equal("Added", log.Action);
    Assert.Contains("Audit Test", log.ChangesJson);
}
}
</file>

```

```

<file path="Tests/Trigger/DatabaseTriggerServiceTests.cs">
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Trigger;

// --- B$ B "Aä B' A0 A !B + (D$5C05D L CD>D BC BCäGC0> Cä4C0>C4> C00C >D 0) ---
public class TestEntity : DomainObject, IAudit { }
public interface ITestBeforeTrigger : IBeforeSaveTrigger<TestEntity> { }
public interface ITestAfterTrigger : IAfterSaveTrigger<TestEntity> { }
public interface ISecondAfterTrigger : IAfterSaveTrigger<TestEntity> { }
public class InvalidTrigger { }

public class DatabaseTriggerServiceTests
{
    private readonly Mock<IServiceProvider> _serviceProviderMock;
    private readonly DatabaseTriggerService _service;

    public DatabaseTriggerServiceTests()
    {
        // 1. AäGC,,IC 5CÂ AD$0D$8C0C C05D 5CB :C 6CDKCÂ BCTAD$>Cí
        DatabaseTriggerService.ClearInternalRegistrations();

        _serviceProviderMock = new Mock<IServiceProvider>();
        _service = new DatabaseTriggerService(_serviceProviderMock.Object);
    }

    [Fact]
    public async Task ValidateAsync_ShouldExecuteRegisteredTrigger()
    {
        // Arrange
    }
}

```



```

var triggerMock = new Mock<ITestBeforeTrigger>();
_serviceProviderMock.Setup(x => x.GetService(typeof(ITestBeforeTrigger)))
    .Returns(triggerMock.Object);

_service.Register<TestEntity, ITestBeforeTrigger>();

// Act
await _service.ValidateAsync(new TestEntity(), EntityStateChangeEnum.Added, [], null!);

// Assert
triggerMock.Verify(x => x.HandleAsync(It.IsAny<EntityCancelEventArgs<TestEntity>>()),
Times.Once);

[Fact]
public async Task ValidateAsync_ShouldThrowException_WhenTriggerCancels()
{
    // Arrange
    string errorMessage = "Stop!";
    var triggerMock = new Mock<ITestBeforeTrigger>();
    triggerMock.Setup(x => x.HandleAsync(It.IsAny<EntityCancelEventArgs<TestEntity>>()))
        .Callback<EntityCancelEventArgs<TestEntity>>(args =>
        {
            args.Cancel = true;
            args.ErrorMessage = errorMessage;
        });

_serviceProviderMock.Setup(x => x.GetService(typeof(ITestBeforeTrigger)))
    .Returns(triggerMock.Object);

_service.Register<TestEntity, ITestBeforeTrigger>();

// Act & Assert
var ex = await Assert.ThrowsAsync<OperationCanceledException>( () =>
    _service.ValidateAsync(new TestEntity(), EntityStateChangeEnum.Added, [], null!));

Assert.Equal(errorMessage, ex.Message);
}

[Fact]
public async Task Hierarchy_ShouldInvokeTriggerForInterface()
{
    // Arrange
    var triggerMock = new Mock<IBeforeSaveTrigger<IAudit>>();
    _serviceProviderMock.Setup(x => x.GetService(typeof(IBeforeSaveTrigger<IAudit>)))
        .Returns(triggerMock.Object);

    // B 5C48D BD 8D CCT< C00 C,,=D$5D DCT9D
    _service.Register<IAudit, IBeforeSaveTrigger<IAudit>>();

    // Act: TestEntity D 5C ;C,,7D45D" " VF-M
    await _service.ValidateAsync(new TestEntity(), EntityStateChangeEnum.Added, [], null!);

    // Assert
    triggerMock.Verify(x => x.HandleAsync(It.IsAny<EntityCancelEventArgs<IAudit>>()),
Times.Once);
}

[Fact]
public void Register_ShouldThrow_WhenTriggerDoesNotImplementInterfaces()
{
    // Act & Assert
    Assert.Throws<InvalidOperationException>( () =>
        _service.Register<TestEntity, InvalidTrigger>());
}

[Fact]
public async Task NotifyAsync_ShouldStopExecution_WhenHandledIsTrue()
{
    // Arrange
    var triggerMock1 = new Mock<ITestAfterTrigger>();
    triggerMock1.Setup(x => x.HandleAsync(It.IsAny<EntityChangedEventArgs<TestEntity>>()))
        .Callback<EntityChangedEventArgs<TestEntity>>(args => args.Handled = true);

    var triggerMock2 = new Mock<ISecondAfterTrigger>();

```

```

        _serviceProviderMock.Setup(x =>
x.GetService(typeof(ITestAfterTrigger))).Returns(triggerMock1.Object);
        _serviceProviderMock.Setup(x =>
x.GetService(typeof(ISecondAfterTrigger))).Returns(triggerMock2.Object);
    }
    _service.Register<TestEntity, ITestAfterTrigger>();
    _service.Register<TestEntity, ISecondAfterTrigger>();
    // Act
    await _service.NotifyAsync(new TestEntity(), EntityStateChangeEnum.Added, [], "User",
DateTime.UtcNow);
    // Assert
    triggerMock1.Verify(x => x.HandleAsync(It.IsAny<EntityChangedEventArgs<TestEntity>>()),
Times.Once);
    triggerMock2.Verify(x => x.HandleAsync(It.IsAny<EntityChangedEventArgs<TestEntity>>()),
Times.Never);
}
}
</file>

```

```

<file path="Tests/Trigger/FluentValidationTriggerTests.cs">
using FluentValidation;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Trigger;
public class FluentValidationTriggerTests
{
    private class TestEntity : DomainObject { public string Name { get; set; } = ""; }
    private readonly Mock<IServiceProvider> _serviceProviderMock;
    private readonly Mock<IServiceScopeFactory> _scopeFactoryMock;
    private readonly Mock<IServiceScope> _serviceScopeMock;
    private class TestEntityValidator : AbstractValidator<TestEntity>
    {
        public TestEntityValidator()
        {
            RuleFor(x => x.Name).NotEmpty().WithMessage("Name is required");
        }
    }
    public FluentValidationTriggerTests()
    {
        _serviceProviderMock = new Mock<IServiceProvider>();
        _scopeFactoryMock = new Mock<IServiceScopeFactory>();
        _serviceScopeMock = new Mock<IServiceScope>();
        // A00D BD 0C,,2C 5CÂ FCT?CâGC=C: BD0C @C,,:C ACâ7CD0CTB Scope -> Scope C$>Ct2D 0D"0CTB ServiceProvide
        _serviceScopeMock
            .Setup(s => s.ServiceProvider)
            .Returns(_serviceProviderMock.Object);
        _scopeFactoryMock
            .Setup(f => f.CreateScope())
            .Returns(_serviceScopeMock.Object);
    }
    [Fact]
    public async Task HandleAsync_ShouldCancel_WhenValidationFails()
    {
        // Arrange
        var entity = new TestEntity { Name = "" };
        var validator = new TestEntityValidator();
        _serviceProviderMock
            .Setup(sp => sp.GetService(typeof(IValidator<TestEntity>)))
            .Returns(validator);
        // A$=CT4D OCT< C0@Câ2C 9CD5D =C ?D 0CÄCDâ 2 D$@C,,3C45D
    }
}

```

```

        var trigger = new FluentValidationTrigger(_scopeFactoryMock.Object);
        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>()
        {
            entity, EntityStateChangeEnum.Added, [], null!
        };

        // Act
        await trigger.HandleAsync(args);

        // Assert
        Assert.True(args.Cancel);
        Assert.Contains("Name is required", args.ErrorMessage);
    }

    [Fact]
    public async Task HandleAsync_ShouldNotCancel_WhenValidationSucceeds()
    {
        // Arrange
        var entity = new TestEntity { Name = "Valid Name" };
        var validator = new TestEntityValidator();

        _serviceProviderMock
            .Setup(sp => sp.GetService(typeof(IValidator<TestEntity>)))
            .Returns(validator);

        var trigger = new FluentValidationTrigger(_scopeFactoryMock.Object);
        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>()
        {
            entity, EntityStateChangeEnum.Added, [], null!
        };

        // Act
        await trigger.HandleAsync(args);

        // Assert
        Assert.False(args.Cancel);
        Assert.Null(args.ErrorMessage);
    }

    [Fact]
    public async Task HandleAsync_ShouldIgnore_WhenNoValidatorRegistered()
    {
        // Arrange
        _serviceProviderMock
            .Setup(sp => sp.GetService(It.IsAny<Type>()))
            .Returns(null!);

        var trigger = new FluentValidationTrigger(_scopeFactoryMock.Object);
        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>()
        {
            new TestEntity(), EntityStateChangeEnum.Added, [], null!
        };

        // Act
        await trigger.HandleAsync(args);

        // Assert
        Assert.False(args.Cancel);
    }
}
</file>

```

```

<file path="Tests/Trigger/FullTriggerChainTests.cs">
using FluentValidation;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using System.Text.Json;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Trigger;

// --- 1. B$ B "Aä B´ Aª A !B + ---
public class IntegrationEntity : DomainObject, IAudit
{
    public string Name { get; set; } = "";
}

```

```

Ð
public class IntegrationValidator : AbstractValidator<IntegrationEntity>Ð
{Ð
    public IntegrationValidator()Ð
    {Ð
        RuleFor(x => x.Name).NotEmpty().WithMessage("NameRequired");Ð
    }Ð
}Ð
Ð
public class IntegrationDbContext(Ð
    DbContextOptions<ApplicationDbContext> options,Ð
    IConfiguration configuration)Ð
    : ApplicationDbContext(options, configuration)Ð
{Ð
    // 1. A,,!Aô A A´ Aô AS¢ CT;C 5CÂ BCTAD$>C$KC' CCT5C² ?Cä;Cô>Dd5Cô=D´< C,,7Cä;C,,@Cä2C =CÔKCÂ 4CT@CT2Cä<Ð
    private class TestNode : ReferenceTreeBase<TestNode>Ð
    {Ð
    }Ð
}Ð
Ð
protected override void OnModelCreating(ModelBuilder modelBuilder)Ð
{Ð
    // 1. At0C0CD :C 5CÂ 1C 7Cä2D´9 C 2D$>CÄ0D$8Dt5D :C,,9 D :C =CT@ PromatisÐ
    base.OnModelCreating(modelBuilder);Ð
}Ð
    // 2. Aô0D BD >C":C 4C´O InMemory C0@Cä2C 9CD5D 0Ð
    if (Database.ProviderName == "Microsoft.EntityFrameworkCore.InMemory")Ð
    {Ð
        modelBuilder.Entity<TestNode>(builder =>Ð
        {Ð
            // At0CD0CT< D02C0>CR 8CÄ0 D$0C ;C,,FD²Â GD$>C K InMemory C,,7Cä;C,,@Cä2C ; CTQÐ
            builder.ToTable("FullTriggerChain_TestNodes");Ð
}Ð
            // A,,!Aô A A´ Aô AS¢ C AD$@C 8C$0CT< D 2D07DÂ AD$@Cä3Cä =C BC,,?CR FW7DæöFRÂ
            // C,,AC0>C´LctCD0 AD$@Cä3Cä BC,,?C,,7C,,@Cä2C =CÔKCR æ ÖVöb >D" AC <Cä3Cä :C´0D AC BCTAD"ô=Cä4D
            builder.HasOne(x => x.Parent)Ð
                .WithMany(x => x.Children)Ð
                .HasForeignKey(x => x.ParentId)Ð
                .onDelete(DeleteBehavior.Restrict);Ð
        });Ð
}Ð
    // B$0C=6CR @CT3C,,AD$@C,,@D45CÂ AC <D2 BCTAD$>C$CDâ 8C0BCT3D 0Dd8Cä=C0CDâ AD4IC0>D BDÍ
    modelBuilder.Entity<IntegrationEntity>();Ð
}Ð
}Ð
Ð
public class FullTriggerChainTestsÐ
{Ð
    [Fact]Ð
    public async Task SaveChanges_ShouldExecuteFullChain_AndCancelOnValidationError()Ð
    {Ð
        // --- ARRANGE ---Ð
}Ð
        // 1. B >Ct4C 5CÂ DCT9C¤>C$CDâ :Cä=DD8C4CD 0Dd8Dí
        IConfigurationRoot configuration = new ConfigurationBuilder()Ð
            .AddInMemoryCollection(new Dictionary<string, string?>Ð
            {Ð
                ["DatabaseSettings:DefaultSchema"] = "test"Ð
            })Ð
            .Build();Ð
}Ð
        var services = new ServiceCollection();Ð
        services.AddLogging();Ð
        services.AddSingleton<IConfiguration>(configuration); // AD>C 0C$;D05CÂ :Cä=DD8C2 2 DÍÐ
}Ð
        // 2. A,,=DD@C AD$@D4:D$CD =D´5 C00D BD >C":C•
        services.AddSingleton(new JsonSerializerOptions { PropertyNamingPolicy =
JsonNamingPolicy.CamelCase });Ð
        // AÄ>C¢ DC 1D 8C=8 D$5C05D L C$>Ct2D 0D"0CTB C¤>C0BCT:D B D :Cä=DD8C4>CÍ
        var factoryMock = new Mock<IDbContextFactory<ApplicationDbContext>>();Ð
        services.AddSingleton(factoryMock.Object);Ð
}Ð
        // 3. B 5C48D BD 0Dd8D0 ACT@C$8D >C" BD 8C43CT@Cä2D
        services.AddScoped<DatabaseTriggerService>();Ð
        services.AddScoped<IDatabaseTriggerService>(sp =>
sp.GetRequiredService<DatabaseTriggerService>());Ð

```

```

    // B 0CÄ8 D$@C,,3C45D K
    services.AddScoped<FluentValidationTrigger>();
    services.AddScoped<ReferenceTreeParentTrigger>();
    services.AddScoped<AuditTrigger>();

    // 4. At0C4;D4HC8 CD;D0 8C0BCT@DD5C"ACä2 (DtBCä1D² FCT?CäGC00 C05 D 2C ;C ADÄÄ 5D ;C, BD 8C43CT@ D$
    services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IAudit>>().Object);
    services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IDomainObject>>().Object);

    // 5. B 5C48D BD 0Dd8D0 2C ;C,,4C BCä@C
    services.AddTransient<IValidator<IntegrationEntity>, IntegrationValidator>();

    ServiceProvider serviceProvider = services.BuildServiceProvider();

    using IServiceScope scope = serviceProvider.CreateScope();
    IServiceProvider sp = scope.ServiceProvider;
    var triggerService = sp.GetRequiredService<DatabaseTriggerService>();

    // A0@C,,2D07D'2C 5CÄ fçVVçEf Æ-F F-ðâG&-vvW" :Cä 2D 5CÄ AD4IC0>D BD0< D wV-B0:C'NDt>Cí
    triggerService.Register<IDomainObjectHasKey<Guid>, FluentValidationTrigger>();

    // A0@C,,2D07D'2C 5CÄ fçVVçEf Æ-F F-ðâG&-vvW" :Cä 2D 5CÄ AD4IC0>D BD0< D wV-B0:C'NDt>Cí
    triggerService.Register<IDomainObjectHasKey<Guid>, FluentValidationTrigger>();

    // A,,!A0 A A' A0 AD B0 ääUB ¢ Ct2C'5C00CT< DD0C @C,,D2 66+ R 8Cr BCTAD$>C$>C4> C0@Cä2C 9CD5D 0D
    var scopeFactory = sp.GetRequiredService<IServiceScopeFactory>();

    // A00D BD >C":C >C0FC,,9 A D 8D ?D 0C$;CT=C0KCÄ ?CT@CTEC$0D$GC,,Cä< (C05D 5CD0CT< D$>C'LC0> DD0C
    DbContextOptions<ApplicationDbContext> options = new
    DbContextOptionsBuilder<ApplicationDbContext>()
        .UseInMemoryDatabase(Guid.NewGuid().ToString())
        .AddInterceptors(new DatabaseTriggerInterceptor(scopeFactory)) // A,,!A0 A A' A0 : Aä4C,,= C @C4CC
        .Options;

    // B >Ct4C 5CÄ :Cä=D$5C0AD"Ä ?CT@CT4C 2C 0 Cä?Dd8C, 8 C0>C0DC,,3D
    await using var context = new IntegrationDbContext(options, configuration);
    await context.Database.EnsureCreatedAsync(TestContext.Current.CancellationToken);

    // B >Ct4C 5CÄ >C JCT:D"Ä :CäBCä@D'9 A0 C0@Cä9CD5D" 2C ;C,,4C FC,,N (Name C0CD BCä9)D
    var entity = new IntegrationEntity { Name = "" };
    context.Add(entity);

    // --- ACT & ASSERT ---

    // Aä6C,,4C 5CÄÄ GD$> C,,=D$5D FCT?D$>D 2D'7Cä2CTB D$@C,,3C45D Ä BCäB C$Kct>C$5D" 2C ;C,,4C BCä@, C, 1D4
    var exception = await Assert.ThrowsAsync<OperationCanceledException>(async () =>
    {
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);
    });

    // A0@Cä2CT@D05CÄÄ GD$> CD>D,,;C, 8CÄ5C0=Cä 4Cä =C HCT3Cä ACä>C ICT=C,,0 C,,7 IntegrationValidatorD
    Assert.Equal("NameRequired", exception.Message);
}
}
</file>

```

```

<file path="Tests/Trigger/ReferenceTreeParentTriggerTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Trigger;

public class ReferenceTreeParentTriggerTests
{
    // --- 1. B$ B "Aä B' A0 A !B + ---

    private class TestNode : ReferenceTreeBase<TestNode> { }

    // A0>C0BCT:D B D$5C05D L C0@C,,=C,,<C 5D" "6öæf-wW& F-öí
    private class TestDbContext(
        DbContextOptions<ApplicationDbContext> options,

```

```

        IConfiguration configuration)
        : ApplicationDbContext(options, configuration)
    {
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);
            modelBuilder.Entity<TestNode>();
        }
    }

    // --- 2. Aô AD Aä"Aä A  Aä B #Ad Aô Bô òòÝ
    private readonly IConfiguration _configuration;

    public ReferenceTreeParentTriggerTests()
    {
        // B >Ct4C 5CÂ DCT9C C$KC' :Cä=DD8C2 4C'0 D$5D BCä2
        _configuration = new ConfigurationBuilder()
            .AddInMemoryCollection(new Dictionary<string, string?>
            {
                ["DatabaseSettings:DefaultSchema"] = "test"
            })
            .Build();
    }

    private async Task<TestDbContext> GetContextAsync()
    {
        DbContextOptions<ApplicationDbContext> options = new
        DbContextOptionsBuilder<ApplicationDbContext>()
            .UseInMemoryDatabase(Guid.NewGuid().ToString())
            .Options;

        var context = new TestDbContext(options, _configuration);
        await context.Database.EnsureCreatedAsync();
        return context;
    }

    // --- 3. B$ B "B² òòÝ
    [Fact]
    public async Task Trigger_ShouldCancel_WhenObjectIsItsOwnParent()
    {
        // Arrange
        await using TestDbContext context = await GetContextAsync();
        var trigger = new ReferenceTreeParentTrigger();

        var nodeId = Guid.NewGuid();
        // Id C, &VçD-B ACä2Cö0CD0DäB
        var node = new TestNode { Id = nodeId, ParentId = nodeId, Name = "Self Parent" };

        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>
            (node, EntityStateChangeEnum.Added, [], context);

        // Act
        await trigger.HandleAsync(args);

        // Assert
        Assert.True(args.Cancel);
        Assert.Equal("Aä1D 5C B CÔ5 CÄ>Cd5D" 1D'BDÂ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5.", args.ErrorMessage);
    }

    [Fact]
    public async Task Trigger_ShouldCancel_WhenParentDoesNotExistInDatabase()
    {
        // Arrange
        await using TestDbContext context = await GetContextAsync();
        var trigger = new ReferenceTreeParentTrigger();

        var missingParentId = Guid.NewGuid();
        var node = new TestNode { Name = "Orphan Node", ParentId = missingParentId };

        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>
            (node, EntityStateChangeEnum.Added, [], context);

        // Act
        await trigger.HandleAsync(args);
    }

```

```

    // Assert
    Assert.True(args.Cancel);
    Assert.Contains("C05 C00C"4CT=", args.ErrorMessage);
}

[Fact]
public async Task Trigger_ShouldNotCancel_WhenParentExistsInDatabase()
{
    // Arrange
    await using TestDbContext context = await GetContextAsync();
    var trigger = new ReferenceTreeParentTrigger();

    var parent = new TestNode { Name = "Valid Parent" };
    context.Add(parent);
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    var child = new TestNode { Name = "Child Node", ParentId = parent.Id };
    var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>()
    {
        child, EntityStateChangeEnum.Added, [], context
    };

    // Act
    await trigger.HandleAsync(args);

    // Assert
    Assert.False(args.Cancel);
    Assert.Null(args.ErrorMessage);
}

[Fact]
public async Task Trigger_ShouldCancel_WhenParentIsSoftDeleted()
{
    // Arrange
    await using TestDbContext context = await GetContextAsync();
    var trigger = new ReferenceTreeParentTrigger();

    var parent = new TestNode
    {
        Id = Guid.NewGuid(),
        Name = "Deleted Parent",
        DeletedAt = DateTime.UtcNow,
        DeletedBy = "Admin"
    };
    context.Add(parent);
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    var child = new TestNode { Name = "Child of Dead", ParentId = parent.Id };
    var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>()
    {
        child, EntityStateChangeEnum.Added, [], context
    };

    // Act
    await trigger.HandleAsync(args);

    // Assert
    Assert.True(args.Cancel);
    Assert.Equal("A05C'LCt0 C00Ct=C GC,,BDÂ @Cä4C,,BCT;CT< D44C ;CT=C0KC' >C JCT:D"â"Â &w2äW'&÷$ÖW76 vR"½", args.ErrorMessage);
}

[Fact]
public async Task Trigger_ShouldCancel_WhenDirectCyclicDependencyDetected()
{
    // Arrange
    await using TestDbContext context = await GetContextAsync();
    var trigger = new ReferenceTreeParentTrigger();

    // 1. B >Ct4C 5CÂ 8 D >DT@C =D05CÂ @Cä4C,,BCT;DÄAC=8C' MC'5CÄ5C0B A
    var nodeA = new TestNode { Id = Guid.NewGuid(), Name = "Node A" };
    context.Add(nodeA);
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    // 2. B >Ct4C 5CÂ 8 D >DT@C =D05CÂ 4CäGCT@C08C' MC'5CÄ5C0B B (B >CD8D$5C'L: A •
    var nodeB = new TestNode { Id = Guid.NewGuid(), Name = "Node B", ParentId = nodeA.Id };
    context.Add(nodeB);
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    // 3. A,,<C,,BC,,@D45CÂ ?Cä?D'BC=C D 4CT;C BDÂ Cct5C² " @Cä4C,,BCT;CT< CD;D0 Cct;C ,, CTBC'0: A -> B ->

```

```

        nodeA.ParentId = nodeB.Id;
    }

    var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>()
    {
        nodeA, EntityStateChangeEnum.Modified, [], context
    };

    // Act
    await trigger.HandleAsync(args);

    // Assert
    Assert.True(args.Cancel);
    Assert.Equal("Bd8C;C,,GCTAC0D0 7C 2C,,AC,,<CäAD$!: C05C´Lct0 C05D 5CÄ5D BC,,BDÄ @Cä4C,,BCT;DÄAC08C' Cct5", context.Current.CancellationToken);

    [Fact]
    public async Task Trigger_ShouldCancel_WhenDeepCyclicDependencyDetected()
    {
        // Arrange
        await using TestDbContext context = await GetContextAsync();
        var trigger = new ReferenceTreeParentTrigger();

        // A0>D BD >C,,< Dd5C0>Dt:D3¢ &ö÷B Óâ 6†-ÆB Óâ 7V$6†-ÆM
        var root = new TestNode { Id = Guid.NewGuid(), Name = "Root" };
        context.Add(root);

        var child = new TestNode { Id = Guid.NewGuid(), Name = "Child", ParentId = root.Id };
        context.Add(child);

        var subChild = new TestNode { Id = Guid.NewGuid(), Name = "SubChild", ParentId = child.Id };
        context.Add(subChild);

        await context.SaveChangesAsync(TestContext.Current.CancellationToken);

        // A,,<C,,BC,,@D45CÄ ?Cä?D´BC¢C C05D 5CÄ5D BC,,BDÄ AC <D´9 C$5D EC08C' Cct5C² %&ö÷B" 2C0CD$@DÄ 5C4> C4;D4
        // Bd5C0>Dt:C ?D >C$5D :C, ?Cä9CD5D" 2C$5D E: SubChild (C" AB' Óâ 6†-ÆB ,,2 A ) -> Root (C05D 5CÄ5D
        root.ParentId = subChild.Id;

        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>()
        {
            root, EntityStateChangeEnum.Modified, [], context
        };

        // Act
        await trigger.HandleAsync(args);

        // Assert
        Assert.True(args.Cancel);
        Assert.Equal("Bd8C;C,,GCTAC0D0 7C 2C,,AC,,<CäAD$!: C05C´Lct0 C05D 5CÄ5D BC,,BDÄ @Cä4C,,BCT;DÄAC08C' Cct5", context.Current.CancellationToken);
    }
}
</file>

```

```

<file path="Tests/Trigger/TreeTriggerChainTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using System.Text.Json;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Trigger
{
    // --- B4 A,, A BÄ B´ B #B" Aä!B$ AD B0 At A´/Bd A, "AT!B$ A" 00Ý
    public class SoftDeleteNode : ReferenceTreeBase<SoftDeleteNode> { }
    public class SelfParentNode : ReferenceTreeBase<SelfParentNode> { }

    // Aä1D"8C' :Cä=D$5C0AD" 4C´O C,,=D$5C4@C FC,,>C0=D´E D$5D BCä2 (D$5C05D L D "6öæf-ww& F-öâ•
    public class TreeTestDbContext
    {
        DbContextOptions<ApplicationDbContext> options;
        IConfiguration configuration;

        public TreeTestDbContext(DbContextOptions<ApplicationDbContext> options,
            IConfiguration configuration)
        {
            options = options;
            configuration = configuration;
        }

        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            modelBuilder.Entity<SoftDeleteNode>();
            modelBuilder.Entity<SelfParentNode>();
        }
    }
}
</file>

```



```

        modelBuilder.Entity<SelfParentNode>();
    }
}

public class TreeTriggerChainTests
{
    private (ServiceProvider Provider, IConfiguration Config) CreateProviderAndConfig()
    {
        // B >Ct4C 5CÂ DCT9C=C$CDâ :Cä=DD8C4CD 0Dd8Dí
        IConfigurationRoot configuration = new ConfigurationBuilder()
            .AddInMemoryCollection(new Dictionary<string, string?>
            {
                ["DatabaseSettings:DefaultSchema"] = "test"
            })
            .Build();

        var services = new ServiceCollection();
        services.AddLogging();
        services.AddSingleton<IConfiguration>(configuration); // AD>C 0C$;Dô5CÂ :Cä=DD8C=

        // 1. AäACô>C$=D´5 D 5D 2C,,AD½
        services.AddScoped<DatabaseTriggerService>();
        services.AddScoped<IDatabaseTriggerService>(sp =>
            sp.GetRequiredService<DatabaseTriggerService>());

        // 2. B A4 B "B B #AT A$!AR "B A4 AT B½
        services.AddScoped<FluentValidationTrigger>();
        services.AddScoped<AuditTrigger>();
        services.AddScoped<ReferenceTreeParentTrigger>();

        // 3. At0C$8D 8CÄ>D BC, BD 8C43CT@Cä2D
        services.AddSingleton(new JsonSerializerOptions { PropertyNamingPolicy =
            JsonNamingPolicy.CamelCase });

        var factoryMock = new Mock<IDbContextFactory<ApplicationDbContext>>();
        services.AddSingleton(factoryMock.Object);

        // 4. At0C4;D4HC=8 CD;Dô 8CôBCT@DD5C"ACä2D
        services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IAudit>>().Object);

        return (services.BuildServiceProvider(), configuration);
    }

    [Fact]
    public async Task SaveChanges_ShouldCancel_WhenParentIsSoftDeleted_ThroughChain()
    {
        // --- ARRANGE ---
        (ServiceProvider serviceProvider, IConfiguration configuration) = CreateProviderAndConfig();
        using IServiceScope scope = serviceProvider.CreateScope();
        IServiceProvider sp = scope.ServiceProvider;

        var triggerService = sp.GetRequiredService<DatabaseTriggerService>();
        triggerService.Register<IDomainObjectHasKey<Guid>, ReferenceTreeParentTrigger>();

        // A,,!Aô A A´ Aô AD Bô ääUB ¢ Ct2C´5C=0CT< DD0C @C,,:D2 66÷ R 8Cr BCTAD$>C$>C4> Cô@Cä2C 9CD5D 0D
        var scopeFactory = sp.GetRequiredService<IServiceScopeFactory>();

        // A,,!Aô A A´ Aô : Aô5D 5CD0CT< C" 8CôBCT@Dd5CôBCä@ D$>C´LC= Cä4C,,= C @C4CCÄ5CôB – scopeFactoryD
        DbContextOptions<ApplicationDbContext> options = new
            DbContextOptionsBuilder<ApplicationDbContext>()
                .UseInMemoryDatabase(Guid.NewGuid().ToString())
                .AddInterceptors(new DatabaseTriggerInterceptor(scopeFactory))
                .Options;

        // Aô5D 5CD0CT< Cä?Dd8C, 8 C=CôDC,,3D
        await using var context = new TreeTestDbContext(options, configuration);

        // 1. B >Ct4C 5CÂ $CCD0C´5Cô=Cä3Câ" @Cä4C,,BCT;Dý
        var deadParent = new SoftDeleteNode
        {
            Id = Guid.NewGuid(),
            Name = "Dead Parent",
            DeletedAt = DateTime.UtcNow
        };
        context.Add(deadParent);
        await context.SaveChangesAsync(TestContext.Current.CancellationTokens);
    }
}

```

```

    }
    // 2. B 5C 5C0>C-
    var child = new SoftDeleteNode()
    {
        Name = "Child",
        ParentId = deadParent.Id
    };
    context.Add(child);

    // --- ACT & ASSERT ---
    await Assert.ThrowsAsync<OperationCanceledException>(async () =>
    {
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);
    });
}

[Fact]
public async Task SaveChanges_ShouldCancel_WhenSelfParenting_ThroughChain()
{
    // --- ARRANGE ---
    (ServiceProvider serviceProvider, IConfiguration configuration) = CreateProviderAndConfig();
    using IServiceScope scope = serviceProvider.CreateScope();
    ServiceProvider sp = scope.ServiceProvider;

    var triggerService = sp.GetRequiredService<DatabaseTriggerService>();
    triggerService.Register<IDomainObjectHasKey<Guid>, ReferenceTreeParentTrigger>();

    // A,,!A A A' A A B0 ääUB ¢ Ct2C'5C0CT< DD0C @C,,:D2 66÷ R 8Cr BCTAD$>C$>C4> C0@Cä2C 9CD5D 0D
    var scopeFactory = sp.GetRequiredService<IServiceScopeFactory>();

    // A,,!A A A' A A : A05D 5CD0CT< C" 8C0BCT@Dd5C0BCä@ D$>C'LC> Cä4C,,= C @C4CCÄ5C0B – scopeFactory
    DbContextOptions<ApplicationDbContext> options = new
    DbContextOptionsBuilder<ApplicationDbContext>()
        .UseInMemoryDatabase(Guid.NewGuid().ToString())
        .AddInterceptors(new DatabaseTriggerInterceptor(scopeFactory))
        .Options;

    await using var context = new TreeTestDbContext(options, configuration);

    var node = new SelfParentNode { Id = Guid.NewGuid(), Name = "Self" };
    node.ParentId = node.Id;
    context.Add(node);

    // --- ACT & ASSERT ---
    var exception = await Assert.ThrowsAsync<OperationCanceledException>(async () =>
        await context.SaveChangesAsync(TestContext.Current.CancellationToken));

    Assert.Equal("Aä1D05C0B C05 CÄ>Cd5D" 1D'BDÄ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5.", exception.Message);
}
}
</file>

<file path="Tests/Validator/DomainObjectValidatorTests.cs">
using FluentValidation.TestHelper;
using Promatis.Net.Domain;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Validator;

public class DomainObjectValidatorTests
{
    // 1. B >Ct4C 5CÄ BCTAD$>C$KC' >C JCT:D" 8 CT3Cä 2C ;C,,4C BCä@
    private class TestEntity : DomainObject { }

    private class TestEntityValidator : DomainObjectValidator<TestEntity> { }

    private readonly TestEntityValidator _validator = new();

    [Fact]
    public void Validator_ShouldFail_WhenIdIsEmpty()
    {
        // Arrange
        var entity = new TestEntity { Id = Guid.Empty };

        // Act
        TestValidationResult<TestEntity>? result = _validator.TestValidate(entity);

```

```

    // Assert
    result.ShouldHaveValidationErrorFor(x => x.Id)
        .WithErrorMessage("A,,4CT=D$8DD8C=0D$>D >C JCT:D$0 C05 CÄ>Cd5D" 1D´BDÂ ?D4AD$KCÂâ""½
}

[Fact]
public void Validator_ShouldFail_WhenCreatedAtIsInFuture()
{
    // Arrange
    var entity = new TestEntity
    {
        CreatedAt = DateTime.UtcNow.AddMinutes(5)
    };

    // Act
    TestValidationResult<TestEntity>? result = _validator.TestValidate(entity);

    // Assert
    result.ShouldHaveValidationErrorFor(x => x.CreatedAt)
        .WithErrorMessage("AD0D$0 D >Ct4C =C,,0 C05 CÄ>Cd5D" 1D´BDÂ 2 C CCDCD"5CÂâ""½
}

[Fact]
public void Validator_ShouldPass_WhenObjectIsValid()
{
    // Arrange
    var entity = new TestEntity(); // Id C, 7&V FVD B 7C ?Cä;C00DäBD O C" :Cä=D BD CC=BCä@CR 1C 7Cä2D´E C

    // Act
    TestValidationResult<TestEntity>? result = _validator.TestValidate(entity);

    // Assert
    result.ShouldNotHaveAnyValidationErrors();
}
}
</file>

<file path="Tests/Validator/ReferenceBaseValidatorTests.cs">
using FluentValidation.TestHelper;
using Promatis.Net.Domain;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Validator;
public class ReferenceBaseValidatorTests
{
    // B$5D BCä2C O D CD"=CäAD$L C, 5E 2C ;C,,4C BCä@
    private class TestReference : ReferenceBase { }
    private class TestReferenceValidator : ReferenceBaseValidator<TestReference> { }

    private readonly TestReferenceValidator _validator = new();

    [Theory]
    [InlineData("")]
    [InlineData(" ")]
    [InlineData("A")] // AÄ5C0LD,,5 2 D 8CÄ2Cä;Cä2
    public void Name_ShouldHaveErrors_WhenInvalid(string? invalidName)
    {
        var model = new TestReference { Name = invalidName! };
        TestValidationResult<TestReference>? result = _validator.TestValidate(model);

        result.ShouldHaveValidationErrorFor(x => x.Name);
    }

    [Fact]
    public void Name_ShouldHaveError_WhenTooLong()
    {
        var model = new TestReference { Name = new string('A', 251) };
        TestValidationResult<TestReference>? result = _validator.TestValidate(model);

        result.ShouldHaveValidationErrorFor(x => x.Name)
            .WithErrorMessage("A0@CT2D´HCT=C <C :D 8CÄ0C´LC00D0 4C´8C00 C00C,,<CT=Cä2C =C,,O (250 D 8CÄ2Cä;C
    }

    [Theory]

```

```

[InlineData("VALID_CODE_123")]
[InlineData("REF1")]
[InlineData("")] // B$5C05D L D0BCâ ?D >C"4CTB D4AC05D,,=Cí
[InlineData(null)] // A, MD$> D$>Cd5D
public void Code_ShouldBeValid_WhenCorrectFormat(string? code)
{
    var model = new TestReference { Code = code };
    TestValidationResult<TestReference>? result = _validator.TestValidate(model);

    result.ShouldNotHaveValidationErrorFor(x => x.Code);
}

[Theory]
[InlineData("C>CE0=C 0@D4AD :Cä<")]
[InlineData("lower_case")]
[InlineData("CODE-1")] // B$8D 5 Ct0C0@CTICT=Cí
public void Code_ShouldHaveError_WhenInvalidFormat(string invalidCode)
{
    var model = new TestReference { Code = invalidCode };
    TestValidationResult<TestReference>? result = _validator.TestValidate(model);

    result.ShouldHaveValidationErrorFor(x => x.Code);
}

[Fact]
public async Task ValidateValue_ShouldReturnErrors_OnlyForSpecificProperty()
{
    // Arrange
    var model = new TestReference
    {
        Name = "", // AäHC,,1C00
        Code = "INVALID-CODE" // AäHC,,1C00
    };

    // Act
    // A0@Cä2CT@D05CÄ @C 1CäBD2 2C HCT3Cä 4CT;CT3C BC f Æ-F FUF ÇVR 4C´O C0>C´O Name
    IEnumerable<string> nameErrors = await _validator.ValidateValue(model,
nameof(TestReference.Name));
    IEnumerable<string> codeErrors = await _validator.ValidateValue(model,
nameof(TestReference.Code));

    // Assert
    Assert.Contains(nameErrors, e => e.Contains("A00C,,<CT=Cä2C =C,,5 Cä1D07C BCT;DÄ=Cä"´´½
    Assert.Contains(codeErrors, e => e.Contains("A>CB <Cä6CTB D >CD5D 6C BDÄ BCä;DÄ:Cä ;C BC,,=C,,FD2"´´½
}

[Theory]
[InlineData(" Name")]
[InlineData("Name ")]
public void Name_ShouldHaveError_WhenHasLeadingOrTrailingSpaces(string invalidName)
{
    var model = new TestReference { Name = invalidName };
    TestValidationResult<TestReference>? result = _validator.TestValidate(model);

    result.ShouldHaveValidationErrorFor(x => x.Name)
        .WithErrorMessage("A00C,,<CT=Cä2C =C,,5 C05 CÄ>Cd5D" =C GC,,=C BDÄAD0 8C´8 Ct0C00C0GC,,2C BDÄAD0 ?D >
}

[Fact]
public async Task ValidateValue_ShouldIgnoreOtherPropertiesErrors()
{
    // Arrange: Cä1C ?Cä;D0 =CT2C ;C,,4C0K0
    var model = new TestReference { Name = "A", Code = "low_case" };

    // Act: C$0C´8CD8D CCT< B$ A´,A0 Name
    IEnumerable<string> nameErrors = await _validator.ValidateValue(model,
nameof(TestReference.Name));

    // Assert
    Assert.Single(nameErrors); // B$>C´LC> CäHC,,1C00 CD;C,,=D² 8CÄ5C08D
    Assert.DoesNotContain(nameErrors, e => e.Contains("A>CB"´´² 00 D,,8C >Cf :Cä4C 1D´BDÄ =CR 4Cä;Cd=Cí
}
}
</file>

<file path="Tests/Validator/ReferenceTreeBaseValidatorTests.cs">

```

```

using FluentValidation.TestHelper;
using Promatis.Net.Domain;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Validator;
public class ReferenceTreeBaseValidatorTests
{
    // B$D BCä2C 0 D CD"=CäAD$L CD;Dò ?D >C$D :C, 0C AD$@C :D$=Cä3Cä :C`0D AC
    private class TestNode : ReferenceTreeBase<TestNode> { }

    // B$D BCä2D`9 C$0C`8CD0D$>D
    private class TestNodeValidator : ReferenceTreeBaseValidator<TestNode> { }

    private readonly TestNodeValidator _validator = new();

    [Fact]
    public void Validator_ShouldFail_WhenParentIdIsEqualToId()
    {
        // Arrange
        var nodeId = Guid.NewGuid();
        var node = new TestNode
        {
            Id = nodeId,
            ParentId = nodeId, // AäHC,,1C=0: ID D >C$?C 4C 5D" A ParentId
            Name = "Self-referencing node"
        };

        // Act
        TestValidationResult<TestNode>? result = _validator.TestValidate(node);

        // Assert
        result.ShouldHaveValidationErrorFor(x => x.ParentId)
            .WithErrorMessage("Aä1D=5C=B C05 CÄ>Cd5D" 1D`BDÂ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5");
    }

    [Fact]
    public void Validator_ShouldFail_WhenParentIdIsEmptyGuid()
    {
        // Arrange
        var node = new TestNode
        {
            ParentId = Guid.Empty, // AäHC,,1C=0 C0> C$BCä@Cä<D2 ?D 0C$8C`C0
            Name = "Empty ParentId"
        };

        // Act
        TestValidationResult<TestNode>? result = _validator.TestValidate(node);

        // Assert
        result.ShouldHaveValidationErrorFor(x => x.ParentId)
            .WithErrorMessage("B >CD8D$5C`LD :C,,9 C,,4CT=D$8DD8C=0D$>D =CR <Cä6CTB C KD$L C0CD BD`< GUID");
    }

    [Fact]
    public void Validator_ShouldPass_WhenParentIdIsDifferent()
    {
        // Arrange
        var node = new TestNode
        {
            Id = Guid.NewGuid(),
            ParentId = Guid.NewGuid(),
            Name = "Valid child node"
        };

        // Act
        TestValidationResult<TestNode>? result = _validator.TestValidate(node);

        // Assert
        result.ShouldNotHaveValidationErrorFor(x => x.ParentId);
    }

    [Fact]
    public void Validator_ShouldPass_WhenParentIdIsNull()
    {
        // Arrange (A= >D 5C0L CD5D 5C$0)
    }
}

```

```
var node = new TestNode()
{
    Id = Guid.NewGuid(),
    ParentId = null,
    Name = "Root node"
};

// Act
TestValidationResult<TestNode>? result = _validator.TestValidate(node);

// Assert
result.ShouldNotHaveValidationErrorFor(x => x.ParentId);
}
</file>

</files>
```